

Universidad Tecnológica Nacional
Facultad Regional Mendoza
Tecnicatura Universitaria en Programación



Trabajo Final

Automatización del Ciclo de Vida del Cliente para Freelancers con n8n
Diseño e implementación de una plataforma web para la gestión automatizada del ciclo de vida del cliente (captación, calificación, propuesta, facturación y cobro)

Autores: Morgui, Mateo | Rivas, Tobías

Directores: Alberto Cortez - Ariel Enferrel

Presentado para optar al título de Técnico Universitario en Programación

Mendoza, Argentina – 2026

Declaración de originalidad y uso de herramientas de inteligencia artificial

Los autores de este Trabajo Final, Mateo Morgui y Tobías Rivas, declaramos que el presente documento y el artefacto de software que describe son de nuestra autoría, desarrollados bajo la dirección de Alberto Cortez, y que no contienen plagio de fuentes de terceros: toda cita textual o paráfrasis de otros autores está debidamente atribuida en el cuerpo del texto y en la lista de Referencias, conforme a las normas de la American Psychological Association (APA) en su séptima edición, declaradas en el Capítulo 3.

Declaramos asimismo que, durante el desarrollo del artefacto de software y la redacción y revisión de este documento, se utilizaron herramientas de inteligencia artificial generativa como apoyo —asistentes de programación para la implementación de código, y asistentes de edición de texto para la redacción, la verificación de afirmaciones contra el código fuente y el esquema de datos, y la corrección del documento—. En todos los casos, las decisiones de diseño, arquitectura, alcance, metodología y contenido sustantivo fueron tomadas y revisadas por los autores, quienes asumen la responsabilidad plena sobre la exactitud, la originalidad y la integridad académica de este trabajo.

Resumen

El trabajo aborda la carga administrativa del freelancer que trabaja solo: cada cliente potencial exige una secuencia de tareas manuales y repetitivas —responder, calificar, elaborar y enviar una propuesta, hacer seguimiento, facturar, registrar el pago y pedir un testimonio— que, con varios clientes en paralelo, se acumulan, se retrasan o se olvidan. Se diseñó e implementó una plataforma web que automatiza el ciclo de vida del cliente mediante una arquitectura orientada a eventos, organizada en tres capas: un frontend en Next.js y React, con un formulario público de captación y un tablero de control autenticado; un motor de orquestación de bajo código, n8n, que coordina normalización, scoring, generación de propuestas y facturas y notificaciones multicanal; y una base de datos relacional gestionada, Supabase (PostgreSQL), con seguridad a nivel de fila. El sistema calcula un puntaje de prioridad que clasifica cada lead en tres niveles (HOT, WARM y COLD) y lo retiene hasta que el profesional fije precio, plazo y alcance; recién entonces envía la propuesta por correo, habilita la aceptación mediante un enlace con token de un solo uso, emite la factura en PDF y ejecuta cuatro procesos programados. La validación por escenarios controlados verificó los flujos de captación, calificación, aceptación y facturación, y el intervalo de propagación de eventos del backend al tablero, medido por debajo del umbral fijado. La rama de cobro no se ejercitó contra el servicio del proveedor de pagos, sino contra un doble de su contrato documentado y un adaptador a una pasarela real en modo de prueba. Conserva cinco puntos de entrada sin autenticación de origen, que restringen su uso a un entorno controlado. Se demuestra la viabilidad técnica de la automatización propuesta; la magnitud de la reducción de la carga administrativa no se midió y se declara fuera de alcance.

Palabras clave: automatización de procesos; orquestación de flujos de trabajo de bajo código; calificación automática de leads; arquitectura orientada a eventos; trabajo independiente digital.

Abstract

This work addresses the administrative burden faced by solo freelancers: every prospective client triggers a sequence of manual, repetitive tasks —answering the enquiry, qualifying the opportunity, drafting and sending a proposal, following up, invoicing, recording payment and requesting a testimonial— which, with several clients in parallel, pile up, get delayed or are forgotten, resulting in lost opportunities. In response, a web platform was designed and implemented to automate the client life cycle through an event-driven architecture, organised in three layers: a Next.js and React frontend, with a public lead-capture form and an authenticated control dashboard; a low-code orchestration engine, n8n, coordinating normalisation, scoring, proposal and invoice generation and multichannel notifications; and a managed relational database, Supabase (PostgreSQL), with row level security. The system computes a priority score that classifies each lead into three tiers (HOT, WARM and COLD) and holds it until the professional sets the price, deadline and scope; only then does it send the proposal by email, enable acceptance through a single-use token link, issue the invoice as a PDF and run four scheduled processes. Validation through controlled scenarios verified the end-to-end operation of the capture, qualification, acceptance and invoicing flows, and the updating of the dashboard, whose event propagation interval between backend and frontend was measured below the established threshold. The payment branch was not exercised against the provider's live service, but against a double reproducing its documented contract and an adapter calling a real gateway in test mode. The artefact retains five entry points without origin authentication, which confine its use to a controlled environment. The work demonstrates the technical feasibility of the proposed automation; the magnitude of the reduction in administrative workload was not measured and is declared outside the scope.

Keywords: process automation; low-code workflow orchestration; automated lead scoring; event-driven architecture; digital freelance work.

Índice

Declaración de originalidad y uso de herramientas de inteligencia artificial	2
Resumen	3
Abstract.....	4
Índice	5
Índice de tablas	9
Índice de figuras	12
Capítulo 1: Introducción.....	1
1.1 Contexto del problema.....	1
1.2 Planteamiento del problema	1
1.3 Tema, problema, pregunta de investigación y proposición orientadora.....	2
1.4 Justificación	4
1.5 Objetivo general.....	4
1.6 Delimitación del objetivo general.....	4
1.7 Definición operacional de la carga administrativa	5
1.8 Objetivos específicos	6
1.9 Alcance y limitaciones.....	7
Capítulo 2: Marco Teórico y Estado del Arte.....	9
Primera parte. Marco teórico	9
2.1 Automatización de procesos y arquitecturas orientadas a eventos.....	9
2.2 Orquestación de bajo código: n8n	10
2.3 Gestión de leads: normalización, scoring y clasificación	10
2.4 Backend como servicio y Supabase.....	11
2.5 Renderizado híbrido y separación servidor-cliente	11
2.6 Seguridad: autenticación, RLS y tokens de un solo uso.....	12
2.7 Encuadre normativo: datos personales y comprobantes fiscales	12
2.8 La carga administrativa como constructo	14

2.9 Modelos de calidad de producto de software	15
Segunda parte. Estado del arte y antecedentes	16
2.10 Antecedentes y estado del arte.....	16
2.10.1 Protocolo de la revisión bibliográfica	16
2.10.2 Síntesis de la literatura arbitrada.....	18
2.10.3 Limitaciones del relevamiento.....	20
2.11 Construir frente a configurar una herramienta existente	24
Capítulo 3: Marco Metodológico.....	25
3.1 Tipo y naturaleza del trabajo	25
3.2 Metodología de desarrollo	25
3.3 Técnicas de relevamiento	26
3.4 Requisitos funcionales	28
3.5 Requisitos no funcionales	29
3.6 Herramientas y tecnologías.....	30
3.7 Estrategia de validación del artefacto	30
3.7.1 Casos de prueba por partición del espacio de entrada.....	31
3.8 Protocolo de ejecución y registro de la validación	32
Capítulo 4: Diseño y Desarrollo del Sistema.....	34
4.1 Arquitectura general	34
4.2 Frontend (Next.js).....	35
4.2.1 Formulario público de captación	35
4.2.2 Autenticación con Supabase Auth.....	36
4.2.3 Protección de rutas.....	36
4.2.4 Cliente de Supabase y degradación defensiva	37
4.2.5 Tablero de control en tiempo real	37
4.2.6 Página de aceptación de propuestas.....	37
4.3 Backend de automatización (n8n)	38

4.3.1 Captación y calificación (webhook de nuevo lead).....	38
4.3.2 Propuesta, aceptación y facturación (webhooks de propuesta enviar y lead acepta)	42
4.3.3 Pago (webhook de confirmación de MercadoPago).....	45
4.3.4 Cierre de proyecto (webhook de proyecto cerrado)	46
4.3.5 Procesos programados	47
4.3.6 Manejo de errores	48
4.3.7 Gestión interactiva del ciclo posterior a la propuesta	48
4.4 Modelo de datos.....	48
4.5 Contratos de integración	51
4.6 Seguridad	52
4.7 Decisiones de diseño.....	55
4.8 Reglas de negocio	56
4.9 Síntesis de los riesgos de diseño	56
Capítulo 5: Resultados y Validación	58
5.1 Escenarios de validación.....	58
5.2 Alcance de la validación	61
5.3 Rama de cobro y doble de prueba.....	63
5.4 Pruebas automatizadas.....	65
5.5 Cobertura de requisitos y evidencia registrada.....	67
5.5.1 Cobertura de la suite sobre el flujo	67
Capítulo 6: Discusión	71
6.1 Comparación con el estado del arte.....	72
6.2 Limitaciones de las afirmaciones del trabajo	75
6.3 Incidente de seguridad: exposición y cierre de credenciales	76
Capítulo 7: Conclusiones	77
7.1 Respuesta a los objetivos y a la pregunta de investigación	77

7.2 Límites de lo acreditado.....	80
7.3 Aporte	80
7.4 Aprendizajes del proceso	81
Capítulo 8: Trabajos Futuros y Recomendaciones	83
Referencias	89
Anexos	94
Anexo A. Evidencias de la validación	94
Anexo B. Flujos de trabajo de n8n	103
B.1 Catálogo de nodos por proceso	Error! Bookmark not defined.
B.2 Clientes de Supabase.....	104
B.3 Rutas del App Router.....	104
Anexo C. Seguridad a nivel de fila (esquema de políticas)	105
Anexo D. Contrato de datos del formulario	106
Anexo E. Matriz de trazabilidad.....	107
Anexo F. Esquema de la base de datos (DDL).....	112
Anexo G. Puesta en marcha del entorno.....	114
Anexo H. Glosario de siglas y acrónimos	115
Anexo I. Bitácora de correcciones durante el desarrollo	116
Anexo J. Instrumentos del estudio propuesto para medir la carga administrativa	118
J.1 Planilla de registro de acciones por lead	119
J.2 Cuestionario de familiaridad previa	120
J.3 Formulario de consentimiento informado (Ley 25.326)	120
Anexo K. Constancia de verificación por un evaluador externo	121

Índice de tablas

Tabla 1. Estatuto probatorio de los tres supuestos de la proposición orientadora	3
Tabla 2. Objetivos específicos del trabajo	6
Tabla 3. Alcance y limitaciones declaradas del trabajo	7
Tabla 4. Componentes de la carga administrativa y su tratamiento en este trabajo ..	14
Tabla 5. Flujo de selección de la literatura arbitrada (revisión tipo PRISMA reducido)	17
Tabla 6. Síntesis de la literatura arbitrada del estado del arte	18
Tabla 7. Caracterización de los cuatro productos comerciales comparados	19
Tabla 8. Razones para construir una solución propia en lugar de configurar una existente	24
Tabla 9. Matriz fuente-etapa del análisis documental del ciclo comercial	27
Tabla 10. Requisitos funcionales del sistema	28
Tabla 11. Requisitos no funcionales del sistema (ISO/IEC 25010:2023)	29
Tabla 12. Herramientas y tecnologías utilizadas, con su versión	30
Tabla 13. Clases de equivalencia y valores límite de las cinco variables del score	31
Tabla 14. Protocolo de ejecución y registro de la validación	33
Tabla 15. Capas de la arquitectura y su responsabilidad	34
Tabla 16. Reglas de cálculo del score	39
Tabla 17. El tramo aceptación→facturación ordenado como saga, con la compensación de cada paso.....	43
Tabla 18. Procesos programados: disparador y comportamiento	47
Tabla 19. Definición operacional de los indicadores del tablero	49
Tabla 20. Estados del ciclo de vida del lead y del trabajo	50
Tabla 21. Contratos de integración (webhooks de n8n)	51
Tabla 22. Deudas de seguridad identificadas, con su estado de cierre	54
Tabla 23. Reglas de negocio que aplica el sistema	56

Tabla 24. Correspondencia entre los riesgos de diseño y las deudas de seguridad S1–S9	56
Tabla 25. Escenarios de validación, resultado esperado y evidencia	58
Tabla 26. Latencia de propagación al tablero (RNF6), veinte repeticiones	62
Tabla 27. Cobertura estimada de los diecisiete escenarios sobre los nodos del flujo del CRM	67
Tabla 28. Estimación hipotética de costos de propiedad a doce meses para un freelancer individual	73
Tabla 29. Comparación funcional frente a las alternativas revisadas en §2.10	73
Tabla 30. Respuesta a los objetivos específicos, con su estado y su evidencia	77
Tabla 31. Dependencias entre las líneas de trabajo futuro	87
Tabla 32. Salida de la corrida completa de la suite de escenarios de validación, sobre el commit del corte documental	102
Tabla 33. Composición del flujo por tipo de nodo	103
Tabla 34. Procesos del flujo, disparador y nodos principales	103
Tabla 35. Clientes de Supabase y su ámbito de ejecución	104
Tabla 36. Rutas del App Router del frontend	104
Tabla 37. Contrato de datos de entrada del webhook de captación	106
Tabla 38. Matriz de trazabilidad objetivos → diseño → validación → conclusión	107
Tabla 39. Trazabilidad de los requisitos funcionales	109
Tabla 40. Trazabilidad de los requisitos no funcionales	110
Tabla 41. Variables de entorno requeridas	114
Tabla 42. Glosario de siglas y acrónimos empleados en el documento	115
Tabla 43. Bitácora de correcciones durante el desarrollo	117
Tabla 44. Categorías de acción manual del instrumento J.1	119
Tabla 45. Planilla de registro de acciones por lead (instrumento J.1)	119
Tabla 46. Cuestionario de familiaridad previa (instrumento J.2)	120

Tabla 47. Constancia de verificación del protocolo de validación por un evaluador externo 122

Índice de figuras

Figura 1. Arquitectura en tres capas del sistema, con los puntos de entrada de la capa de orquestación y el sentido de las escrituras y lecturas.	35
Figura 2. Diagrama de secuencia del flujo de aceptación y facturación, con los siete participantes y el alcance efectivo de la transacción.	45
Figura 3. Diagrama entidad-relación del modelo de datos, con tipos, claves, restricciones y valores por defecto.....	49
Figura 4. Máquinas de estados del sistema: ciclo comercial del lead (A) y ciclo de ejecución del trabajo (B).	50
Figura 5. Formulario público de captación de leads, con el control deslizante de presupuesto y las seis opciones de servicio (§4.2.1, escenario E1).	94
Figura 6. Tablero de control interno: la bandeja de propuestas por enviar, desde la cual el profesional fija el precio, el plazo y el alcance antes del envío (§4.3.2); los indicadores del mes y el embudo por estado (§4.2.5, escenario E7).	95
Figura 7. Página pública de aceptación de propuesta, protegida por token de un solo uso (§4.2.6).	96
Figura 8. Flujo de captación y calificación en n8n: webhook de nuevo lead, normalización, verificación de duplicados, scoring, inserción ya calificada y el aviso de que la propuesta queda pendiente de que el profesional la envíe (§4.3.1).	96
Figura 9. Flujo de aceptación y facturación en n8n: validación del token, actualización condicional atómica, generación de la preferencia de pago con MercadoPago, generación del PDF y persistencia de la factura (§4.3.2, §4.3.3).	97
Figura 10. Flujo de confirmación de pago en n8n: modo de desarrollo y confirmación real con MercadoPago (§4.3.3).	97
Figura 11. Flujo de cierre de proyecto y solicitud de testimonio en n8n (§4.3.4).	97
Figura 12. Procesos programados en n8n: seguimiento de propuestas y recordatorios de pago (§4.3.5, escenarios E11 y E12).	98
Figura 13. Correo de propuesta generado por el sistema, con los términos fijados por el profesional (§4.3.1, escenario E5).	99

Figura 14. Correo de envío de la factura con el comprobante adjunto en PDF (§4.3.2, escenario E5).	99
Figura 15. Comprobante en PDF generado por el sistema (§4.3.2, escenario E5).	100
Figura 16. Clasificación de leads por score y nivel: ejemplos HOT, WARM y COLD registrados en la tabla leads (§4.3.1, escenarios E1 a E3).	100
Figura 17. Validación del formulario ante una entrada inválida (§4.2.1, escenario E4). .	101
Figura 18. Los cuatro desenlaces de la página pública de aceptación de propuesta: (a) aceptación satisfactoria (§4.2.6, escenario E5); (b) enlace ya utilizado, rechazado sin reprocesar ni duplicar la factura (§4.3.2, escenario E6); (c) rechazo de la propuesta por el cliente, el lead pasa a PERDIDO (§4.3.7, escenario E8); (d) pedido de cambios del cliente, el lead vuelve a EN_SEGUIMIENTO (§4.3.7, escenario E9).	101

Capítulo 1: Introducción

Este capítulo delimita el problema que motiva el trabajo, distingue el tema de la pregunta de investigación, justifica la relevancia de la propuesta y fija los objetivos y el alcance que ordenan el resto del documento.

1.1 Contexto del problema

El trabajo independiente en el sector del software y los servicios digitales ha crecido de forma sostenida. El Online Labour Index registró un crecimiento interanual cercano al 20 % en la demanda de trabajo digital por proyecto (Kässi y Lehdonvirta, 2018), y la Organización Internacional del Trabajo documenta una expansión reciente del trabajo mediado por plataformas web en América Latina y el Caribe (OIT, 2025). El freelancer que ofrece desarrollo web, diseño, marketing o consultoría opera casi siempre como una unidad de una sola persona: vende, produce, factura y da soporte. Concentrar esas funciones, que una empresa reparte entre áreas diferenciadas, traslada una parte significativa de su tiempo productivo hacia tareas administrativas de bajo valor agregado.

Cada nuevo contacto comercial —o lead— inicia un ciclo que, gestionado manualmente, resulta frágil: recibir y ordenar los datos del interesado, evaluar si la oportunidad conviene, redactar y enviar una propuesta, recordar el seguimiento si no hay respuesta, emitir la factura al cerrar, controlar el cobro y pedir un testimonio. Con un único cliente el proceso es abordable; con varios en paralelo, los pasos se superponen, se posponen o se pierden.

1.2 Planteamiento del problema

La gestión manual del ciclo comercial presenta tres debilidades concretas. La primera es la falta de priorización: sin un criterio sistemático, se dedica el mismo esfuerzo a una consulta de bajo presupuesto que a una oportunidad de alto valor. La segunda es la pérdida de trazabilidad: el estado de cada oportunidad —si se envió la propuesta, si hubo seguimiento, si se facturó— queda disperso en correos y notas, sin una fuente única de verdad. La tercera es la dependencia de la memoria del profesional para tareas que deberían dispararse solas, como el seguimiento a los tres días o el recordatorio de una factura próxima a vencer.

Estas debilidades no se resuelven contratando más personal —el freelancer, por definición, trabaja solo— sino delegando las tareas repetitivas en un sistema que las ejecute de forma consistente, trazable y desatendida. Corresponde declarar el estatuto de este

diagnóstico: las tres debilidades no proceden de un relevamiento con usuarios, sino del análisis documental del §3.3 y de la experiencia directa de los autores, cuya condición de fuente no recuperable ni auditable por un tercero ese mismo apartado precisa. Se las enuncia, por lo tanto, como punto de partida del trabajo y no como hecho establecido por la literatura; su verificación empírica se registra en el punto 10 del Capítulo 8.

1.3 Tema, problema, pregunta de investigación y proposición orientadora

El **tema** del trabajo es la automatización de procesos comerciales y administrativos para trabajadores independientes del sector digital. El **problema** es la carga manual, la falta de priorización y la pérdida de trazabilidad en la gestión del ciclo de vida del cliente. De ambos se deriva la siguiente **pregunta de investigación**:

¿Es técnicamente viable automatizar el ciclo de vida completo de la gestión de clientes de un freelancer —captación, calificación, propuesta, facturación y cobro— mediante una arquitectura orientada a eventos de bajo código, de modo que las tareas hoy ejecutadas manualmente queden delegadas en el sistema sin sacrificar la trazabilidad ni el control del profesional sobre el proceso?

La pregunta se formula en términos de viabilidad técnica y no de magnitud del ahorro. La reducción efectiva de la carga administrativa es el efecto que se espera de la automatización, pero medirla exige una línea de base previa y una medición posterior sobre operación real, que este trabajo no realiza y declara fuera de alcance (§1.9). La delimitación se retoma en §1.6, en el Capítulo 6 y en el Capítulo 7.

Las dos mitades de la pregunta no se responden con el mismo rigor, y conviene declarar bajo qué condición la respuesta habría sido negativa. La primera —si la automatización es técnicamente viable— se acredita por construcción: un artefacto que ejecuta el ciclo de extremo a extremo la responde afirmativamente, y sólo un flujo que no pudiera completarse la habría refutado. La segunda —si eso ocurre «sin sacrificar la trazabilidad ni el control del profesional»— sí es refutable de forma sustantiva, y es la que ordena el trabajo. La respuesta habría sido negativa si el sistema hubiera tomado por el profesional una decisión de negocio reservada a él, o si el estado de una oportunidad no hubiera podido reconstruirse a partir de lo registrado. El primero de esos criterios fue efectivamente incumplido por un diseño intermedio, en el que el importe de la propuesta provenía del formulario que completaba el propio interesado, y la condición se restituyó devolviendo esa decisión al profesional (§4.3.2 y Capítulo 7).

La proposición orientadora del trabajo se enuncia aquí para que su verificación pueda juzgarse sin ambigüedad. El ciclo de vida comercial de un profesional independiente puede ejecutarse de extremo a extremo mediante la orquestación de eventos de bajo código. En esa ejecución el profesional conserva la decisión sobre los términos comerciales de cada oportunidad, y el estado de esa oportunidad se mantiene reconstruible a partir de lo que el sistema registra.

Los tres supuestos no son refutables con el mismo rigor, y la Tabla 1 clasifica el estatuto probatorio de cada uno para que ese balance no deba reconstruirse a lo largo del documento. La proposición se habría considerado refutada en tres supuestos:

que algún flujo del ciclo no pudiera completarse;

que el sistema tomara por el profesional una decisión de negocio reservada a él;

que el estado de una oportunidad no pudiera reconstruirse con lo que el sistema persiste.

Tabla 1. Estatuto probatorio de los tres supuestos de la proposición orientadora

Supuesto	Enunciado (condición de refutación)	Estatuto probatorio	Base de la clasificación
Primero	Que algún flujo del ciclo no pudiera completarse.	Por construcción	Los diecisiete escenarios de validación completan su recorrido, pero un artefacto que ejecuta el ciclo de extremo a extremo satisface el supuesto por su sola existencia: sólo un flujo que no pudiera ejecutarse lo habría refutado. El supuesto queda descartado, no contrastado.
Segundo	Que el sistema tomara por el profesional una decisión de negocio reservada a él.	Contrastado	Es el único de los tres que llegó a verificarse durante el desarrollo: un diseño intermedio tomaba el importe de la propuesta del formulario que completaba el propio interesado. Se restituyó la condición devolviendo al profesional la fijación de precio, plazo y alcance (§4.3.2). Se descarta sobre el ciclo completo, si bien por una decisión de arquitectura y no por un hallazgo empírico.
Tercero	Que el estado de una oportunidad no pudiera reconstruirse con lo que el sistema persiste.	Parcial	El escenario E15 reconstruye una oportunidad en el tramo captación → propuesta leyendo únicamente lo que la base persiste. El resto del ciclo — aceptación, facturación, cobro y cierre— se acredita por construcción, y su reconstrucción íntegra queda fuera de la evidencia recogida.

Nota. Elaboración propia. «Contrastado» indica que el supuesto pudo haberse verificado y se comprobó que no ocurre; «parcial», que la comprobación cubre sólo un tramo del ciclo; «por construcción», que el diseño del artefacto satisface el supuesto y ninguna evidencia recogida podría haberlo refutado. El Capítulo 7 cierra la proposición con esta misma clasificación, en lugar de con una afirmación uniforme sobre los tres.

El Capítulo 5 reporta la verificación de los tres supuestos, con el escenario E15 para el tercero, y el Capítulo 7 los responde uno a uno con este mismo balance probatorio.

1.4 Justificación

La respuesta tiene relevancia profesional y técnica. Desde lo profesional, un sistema que automatice la captación, la calificación y el seguimiento puede liberar tiempo del freelancer para las tareas que generan ingresos, y cabe esperar que mejore la velocidad y la consistencia de la respuesta al cliente. Son expectativas plausibles que este trabajo no verifica empíricamente y que enuncia como resultado esperado y no como hallazgo. Desde lo técnico, el trabajo integra un conjunto de tecnologías actuales en una solución funcional y verificable: orquestación de bajo código, bases de datos gestionadas con seguridad a nivel de fila, renderizado híbrido y comunicación por webhooks. Constituye así evidencia concreta de las competencias esperadas de un Técnico Universitario en Programación. El posicionamiento frente a las soluciones existentes se desarrolla en §2.10 y §2.11.

1.5 Objetivo general

Diseñar, implementar y validar funcionalmente una plataforma web que automatice el ciclo de vida completo de la gestión de clientes para freelancers —desde la captación del lead hasta el cobro y el cierre— mediante la orquestación de eventos con n8n y la persistencia en una base de datos relacional gestionada con Supabase. El propósito es demostrar la viabilidad técnica de delegar en el sistema las tareas administrativas que hoy se ejecutan de forma manual.

Al tratarse de un desarrollo tecnológico aplicado, y no de una investigación empírica, el trabajo no formula una hipótesis estadística sino una proposición contrastable, orientada por la pregunta de investigación del §1.3 y por los objetivos aquí definidos. Esa proposición se enuncia en el §1.3, su cumplimiento se verifica en el Capítulo 5 y su fundamento metodológico se desarrolla en el §3.1.

1.6 Delimitación del objetivo general

La delimitación precisa qué queda deliberadamente fuera del objetivo general, que excluye el resultado «reducir la carga administrativa manual». La razón no es que la variable quede indefinida: el §1.7 le da una definición operacional y mide uno de sus dos componentes. El que decide esa promesa —el tiempo y el esfuerzo que cada acción cuesta— exigiría una línea de base previa y una medición posterior sobre operación real, que este

trabajo no realiza. Incluir la promesa dejaría al trabajo concluyendo sobre algo que su estrategia de validación no puede acreditar.

1.7 Definición operacional de la carga administrativa

Se adopta la siguiente definición operacional, apoyada en el marco del §2.8: carga administrativa = número de acciones manuales del profesional por lead gestionado, contadas desde el ingreso del contacto hasta el registro del cobro. Se entiende por acción manual toda interacción del profesional con una herramienta: redactar un correo, copiar datos entre sistemas, generar un documento, actualizar una planilla. La definición tiene dos componentes y este trabajo mide sólo el primero. El conteo de acciones se mide, porque puede derivarse del artefacto; el tiempo y el esfuerzo que cada acción cuesta, no, porque exigirían observar a una muestra de freelancers gestionando leads con el sistema y sin él. El Capítulo 8 recoge esa segunda línea.

Cobertura de acciones del ciclo. Antes de exponerlo corresponde fijar su estatuto, porque condiciona todo lo que sigue: es un descriptor del alcance del artefacto y no un resultado de la validación. Expresa la proporción de acciones del ciclo que el sistema ejecuta, no una medición de la carga en tiempo o esfuerzo, y por eso no se computa como resultado en la matriz del Anexo E ni se lo invoca como hallazgo en el Capítulo 7. Aplicada la definición sobre el artefacto, el ciclo de un lead calificado —del ingreso del contacto al cierre del proyecto— comprende dieciocho acciones administrativas. El sistema ejecuta dieciséis y el profesional conserva dos: decidir el precio, el plazo y el alcance de la propuesta, y dar por cerrado el proyecto. Son las dos decisiones de negocio que el trabajo deja deliberadamente fuera de la automatización (§4.3.2 y §7.1). El conteo abarca el camino principal y deja fuera tres acciones condicionales, también a cargo del profesional, que sólo aparecen cuando el ciclo se desvía: resolver un pedido de cambios, actualizar el estado de ejecución del trabajo y cancelar un lead. Incorporadas al denominador, la relación en el peor caso sería de dieciséis sobre veintiuna. Se las consigna por separado porque no integran el recorrido de un lead que avanza sin incidentes, que es el que esta medición cuenta. El conteo se obtiene con scripts/medir-carga-administrativa.mjs, que declara cada acción junto al nodo que la ejecuta y falla si alguno dejó de existir, de modo que la medición se invalida sola si el artefacto cambia.

Su alcance tiene dos límites. El denominador de dieciocho acciones se deriva del propio artefacto y no de una observación independiente de un ciclo gestionado a mano: en sentido estricto, el indicador mide la cobertura del sistema construido sobre su propio

alcance, no la reducción respecto de una línea de base externa. Y, ya dentro de ese alcance, mide cuántas acciones quedan delegadas, no cuánto tiempo ni cuánto esfuerzo ahorran; sostener lo segundo exigiría el estudio con usuarios del Capítulo 8. Conviene precisar, además, el techo del indicador: el denominador enumera las acciones que el sistema ejecuta más las dos excluidas deliberadamente, y no incluye ninguna que el diseño haya dejado sin cubrir por deficiencia, de modo que la razón de dieciséis sobre dieciocho no puede, por construcción, arrojar un valor bajo.

1.8 Objetivos específicos

Para alcanzar el objetivo general se definen los objetivos específicos (OE) que enumera la Tabla 2, redactados en infinitivo y retomados de forma trazable en el diseño, en la validación y en las conclusiones (Anexo E). Tres de ellos se enuncian primero de forma unificada y se desdoblan enseguida en dos mitades, que el Capítulo 7 declara cumplidas o incumplidas por separado; el desdoblamiento no altera el objetivo, sino que separa dos tramos cuyo estado de verificación puede diferir. El OE3 se separa porque el segundo tramo depende de una pasarela externa y el primero no. El OE4, porque el modelo de datos se verifica por inspección del esquema y el tablero por medición de propagación de eventos, de modo que el Anexo E los traza con instrumentos distintos. Y el OE6, porque una mitad especifica y verifica los controles implementados y la otra audita la superficie que queda sin proteger (§4.6, §4.9, Tabla 22 y Anexo C).

Tabla 2. Objetivos específicos del trabajo

ID	Objetivo específico
OE1	Implementar un formulario web público de captación de leads, con validación, integrado a un backend de automatización mediante webhooks.
OE2	Diseñar e implementar un mecanismo de normalización y scoring automático que clasifique cada lead en niveles de prioridad (HOT, WARM y COLD).
OE3	Automatizar el ciclo comercial completo, desde la generación de la propuesta hasta el registro del cobro de la factura emitida. Se desdobla en:
OE3a	Automatizar el ciclo comercial hasta la emisión del comprobante —generación y envío de propuestas, aceptación mediante enlace con token de un solo uso y facturación en PDF— con notificaciones multicanal por correo electrónico y Telegram.
OE3b	Integrar una pasarela de pago que permita registrar el cobro de la factura emitida.
OE4	Diseñar la capa de datos y de visualización del sistema: una base relacional con seguridad a nivel de fila y un tablero de control autenticado que la consulte. Se desdobla en:
OE4a	Diseñar un modelo de datos relacional en Supabase con seguridad a nivel de fila (RLS).
OE4b	Exponer un tablero de control autenticado con métricas en tiempo real.
OE5	Incorporar procesos programados —seguimiento de propuestas, recordatorios de pago y reporte diario de métricas— que operen sin intervención manual.
OE6	Especificar y verificar sobre el artefacto los controles de acceso del sistema, y auditar la superficie de exposición que queda sin proteger. Se desdobla en:
OE6a	Especificar los controles de acceso del sistema —autenticación, rol de aplicación y seguridad a nivel de fila— y verificar sobre el artefacto su funcionamiento efectivo.
OE6b	Auditar y medir sobre el artefacto el alcance de la superficie que queda sin proteger.

Nota. Elaboración propia.

1.9 Alcance y limitaciones

El trabajo corresponde a un producto mínimo viable (MVP) orientado a demostrar la viabilidad técnica de la automatización propuesta. El alcance comprende el diseño, la implementación y la validación funcional de los flujos descritos en un entorno controlado.

Quedan fuera del alcance, y se declaran explícitamente como limitaciones, los puntos que reúne la Tabla 3.

Tabla 3. Alcance y limitaciones declaradas del trabajo

Aspecto excluido	Alcance de la exclusión
Operación en producción con tráfico real.	No se reportan métricas operativas reales de adopción, rendimiento ni conversión.
Costos de infraestructura.	No se midieron los costos de operación. La exclusión es relevante porque §2.10 y §2.11 invocan el costo como una de las razones para construir; el Capítulo 6 incorpora por eso una estimación de costo total de propiedad con supuestos declarados (Tabla 28) en lugar de dar el argumento por probado.
Magnitud de la reducción de la carga administrativa.	La variable se define operacionalmente y su componente contable se mide: de las dieciocho acciones del camino principal, el sistema ejecuta dieciséis (§1.7). Queda fuera el otro componente, el tiempo y el esfuerzo que esas acciones cuestan, que exigiría un estudio con usuarios (Capítulo 8). El trabajo acredita cuántas acciones quedan delegadas, no cuánto trabajo ahorran.
Exposición pública del motor de automatización.	Durante el desarrollo, no se ejecutó de forma autoalojada en un equipo local, sin URL pública estable (véase el Capítulo 6).
Pasarela de pago real.	La integración con MercadoPago (§4.3.3) no se validó con una operación en producción: la activación de credenciales de prueba falló del lado del proveedor, de modo que la rama de cobro se ejerció contra un doble que reproduce el contrato documentado de su API y, desde el 31 de agosto de 2026, también contra un adaptador que llama a una pasarela real — Stripe — en modo de prueba (§5.3). Eso verifica el artefacto, no el servicio de MercadoPago. Sin credenciales reales configuradas, el sistema cae a un modo de desarrollo que confirma el cobro sin depender del proveedor.
Validez fiscal de los comprobantes.	El documento que el sistema genera y titula «FACTURA» es un comprobante interno sin valor fiscal: no solicita el CAE que exige el régimen de facturación electrónica argentino, ni consigna CUIT del emisor, punto de venta ni tipo de comprobante. El §2.7 desarrolla el régimen aplicable y el Capítulo 8 plantea su incorporación.
Tratamiento de datos personales.	El sistema capta, almacena y transfiere a terceros (Supabase, el servicio de correo, Telegram y Notion) datos de contacto de personas físicas identificables, sin aviso de privacidad, consentimiento informado, política de retención ni inscripción de la base conforme a la Ley 25.326, y varios de esos destinatarios están domiciliados fuera de los países que la Disposición 60-E/2016 considera de protección adecuada. El §2.7 desarrolla las obligaciones y el Capítulo 8 las ordena como trabajo pendiente.
Autenticación de origen de los webhooks.	Cinco de los trece webhooks no verifican la identidad de quien los invoca por diseño; de esos cinco, cuatro exigen igualmente un token UUID válido, vigente y rotativo, de modo que sólo el formulario público es alcanzable conociendo apenas su URL (§4.6). Los seis invocados desde el tablero interno ya exigen autenticación; pago-confirmado se cerró de forma parcial el 31 de agosto de 2026 con un token por factura (S1) y el de MercadoPago exige firma válida desde esa misma fecha (S8). El reparto exacto y su medición sobre el artefacto se desarrollan una sola vez, en el §4.6 y en la Tabla 22.
Módulo de gestión de tickets.	El repositorio incluye una ruta de tablero de tickets (/dashboard/tickets) y los nodos de flujo asociados, que no forman parte del ciclo de vida del cliente que este trabajo aborda. Su diseño, su validación y su documentación quedan fuera del alcance; las menciones del §4.2, el §5.4 y el Anexo B deben leerse en ese sentido.

Nota. Elaboración propia.

Estas limitaciones se retoman en la discusión (Capítulo 6) y orientan los trabajos futuros (Capítulo 8).

Organización del documento. El Capítulo 2 fija el marco conceptual y tecnológico y sitúa el trabajo frente a las soluciones que ya atienden el mismo problema. El Capítulo 3 declara el tipo de trabajo, la metodología, las técnicas de relevamiento, los requisitos y la

estrategia de validación. El Capítulo 4 describe la arquitectura y cada componente, con sus decisiones de diseño, sus reglas de negocio y sus riesgos. El Capítulo 5 reporta la ejecución de los escenarios y delimita qué acreditan. El Capítulo 6 los discute frente al estado del arte y a las limitaciones declaradas. El Capítulo 7 responde a los objetivos, a la proposición y a la pregunta de investigación. El Capítulo 8 ordena el trabajo pendiente. Los Anexos A a K reúnen la evidencia de la validación, el detalle del flujo y del esquema de datos, los contratos de integración, las matrices de trazabilidad, la puesta en marcha, el glosario, la bitácora de correcciones, los instrumentos del estudio propuesto y la constancia de verificación externa.

Capítulo 2: Marco Teórico y Estado del Arte

Este capítulo cumple dos funciones distintas y las presenta por separado. La primera parte fija el marco conceptual y tecnológico y define los términos clave. Los apartados 2.1 a 2.6 desarrollan las nociones de automatización, orquestación de bajo código, gestión de leads, backend como servicio, renderizado híbrido y seguridad. El 2.7 delimita el encuadre normativo que alcanza a lo que el sistema hace. El 2.8 vuelve sobre el constructo que da origen al problema y fija el marco de medición sobre el que el §1.7 apoya su definición operacional. Y el 2.9 fundamenta el modelo de calidad que el Capítulo 3 aplica. La segunda parte —apartados 2.10 y 2.11— sitúa el trabajo frente a las soluciones existentes y justifica la decisión de construir una propia. La separación es deliberada: lo que la primera parte establece es marco conceptual, y lo que la segunda sostiene son afirmaciones sobre productos concretos, con fuentes y limitaciones de otra naturaleza, declaradas en el §2.10.3.

Primera parte. Marco teórico

2.1 Automatización de procesos y arquitecturas orientadas a eventos

La automatización de procesos consiste en delegar en un sistema la ejecución de tareas que, de otro modo, realizaría una persona de forma manual y repetitiva. Cuando esas tareas involucran múltiples sistemas —una base de datos, un servicio de correo, una API externa— el enfoque más adecuado es una arquitectura orientada a eventos. En ella, un suceso —por ejemplo, la llegada de un nuevo lead— dispara una cadena de acciones coordinadas, secuenciales o condicionales (Hohpe y Woolf, 2003; Richardson, 2018). Este estilo desacopla al emisor del evento de sus consumidores y facilita incorporar nuevos pasos sin modificar los existentes.

Un webhook es el mecanismo habitual para propagar estos eventos entre sistemas a través de la web: una URL que permanece a la escucha y recibe, típicamente mediante una petición HTTP POST, los datos de un suceso ocurrido en otro sistema. Se apoya en el estilo arquitectónico REST sobre HTTP que Fielding (2000) formalizó; el término, en cambio, no procede de una fuente académica sino de una convención consolidada en la práctica de la industria de integraciones, y se lo emplea aquí con el sentido operativo recién definido. En la solución, los webhooks son el punto de entrada de cada flujo de negocio.

El mismo Richardson (2018) describe el problema inverso: cuando una operación de negocio se descompone en pasos que actúan sobre servicios distintos, ninguna transacción de base de datos abarca la operación completa y una falla a mitad de camino puede dejar el

sistema en un estado parcial. El patrón que propone es la saga, formulada por Garcia-Molina y Salem (1987) para transacciones de larga duración: una secuencia de pasos locales, cada uno con su transacción de compensación, que deshace lo hecho si un paso posterior falla. Es el problema de la aceptación de una propuesta en este sistema (§4.3.2), donde entre la actualización condicional del lead y la generación de la factura median varias llamadas a servicios externos que ninguna transacción cubre. La literatura de sistemas de datos sitúa la idempotencia de cada paso como condición mínima para reintentar sin duplicar efectos (Kleppmann, 2017): es la garantía que el §4.3.2 construye sobre la aceptación y la que el §4.3.5 aprovecha para reparar la facturación que no llegó a persistirse.

2.2 Orquestación de bajo código: n8n

n8n es una plataforma de automatización de flujos de trabajo que integra aplicaciones y servicios mediante nodos configurables conectados en un lienzo visual (n8n, s. f.-a). Su código está disponible públicamente bajo la Sustainable Use License (n8n, s. f.-b) —un modelo «fair-code» que restringe el uso comercial competitivo y la redistribución paga—, por lo que no constituye código abierto en el sentido estricto de la Open Source Initiative (2024). Cada nodo encapsula una operación —recibir un webhook, ejecutar código JavaScript, consultar una base de datos, enviar un correo— y las conexiones definen el orden y las condiciones de ejecución. Su carácter de bajo código no elimina la programación sino que la concentra donde aporta valor, delegando en nodos preconstruidos la integración con servicios externos; una revisión sistemática multi-vocal confirma ese carácter híbrido y señala la ausencia de consenso terminológico en el campo (Bucaioni et al., 2022). Dentro de la arquitectura, n8n es la capa que conecta el frontend, la base de datos y los servicios de mensajería, coordinando la secuencia de acciones sin trasladar la complejidad de integración al resto de los componentes.

2.3 Gestión de leads: normalización, scoring y clasificación

Un lead es un contacto comercial que ha manifestado interés en un producto o servicio (Kotler y Armstrong, 2017). Su gestión busca convertir ese interés en una operación concreta y ordena el recorrido del contacto a lo largo de un embudo de ventas: un conjunto de estados sucesivos, desde el primer contacto hasta el cierre.

Dos procesos son centrales. La normalización limpia y estandariza los datos de entrada —recortar espacios, unificar mayúsculas, validar formatos, aplicar valores por defecto— para garantizar su consistencia. El scoring asigna a cada lead un puntaje según

criterios ponderados que permite priorizar las oportunidades más prometedoras y clasificarlas en niveles. La literatura arbitrada respalda la calificación automática como práctica establecida (Järvinen y Taiminen, 2016; Mero et al., 2020). Una revisión reciente confirma que la ponderación por reglas explícitas —el enfoque de la Tabla 16— sigue siendo el método predominante, mientras que las alternativas basadas en aprendizaje automático exigen volúmenes históricos de los que este trabajo no dispone (Wu et al., 2024). La clasificación en niveles de temperatura —HOT, WARM y COLD— refleja una práctica corriente (Kotler y Armstrong, 2017): segmentar el embudo entre oportunidades listas para ventas y oportunidades que aún requieren maduración, corte que la literatura apoya en ejes como presupuesto, autoridad, necesidad y plazo (Wu et al., 2024). La Tabla 16 operacionaliza esos ejes con tres criterios observables y fija el umbral de cada nivel; las etiquetas son de uso comercial extendido y se las adopta como convención. En todo el documento «nivel» y «tier» son sinónimos: se prefiere «nivel» en el cuerpo y «tier» sólo para designar la columna homónima de la base de datos.

2.4 Backend como servicio y Supabase

El patrón *Backend as a Service* (BaaS) provee, como servicio gestionado, capacidades de backend —base de datos, autenticación, almacenamiento y APIs— que de otro modo el equipo debería implementar y operar. Supabase es una plataforma BaaS de código abierto construida sobre PostgreSQL, que expone la base de datos relacional junto con autenticación, APIs automáticas y suscripciones en tiempo real (Supabase, s. f.). Al apoyarse en PostgreSQL, hereda su modelo relacional, su lenguaje SQL y, en particular, la **seguridad a nivel de fila** (*Row Level Security*, RLS), que permite definir, mediante políticas declarativas, qué filas puede leer o modificar cada rol (PostgreSQL Global Development Group, 2024).

2.5 Renderizado híbrido y separación servidor-cliente

El desarrollo web actual reparte la ejecución de una aplicación entre el servidor y el navegador, y el renderizado híbrido combina componentes que se resuelven en uno y en otro. Esa separación tiene una consecuencia de diseño directa: las verificaciones sensibles —validar la sesión antes de entregar una página protegida, comprobar un rol, resolver un token— quedan del lado del servidor, fuera del alcance del usuario, y al cliente se reserva la interactividad. El modelo es pertinente para una aplicación que combina un formulario público, accesible sin cuenta, con un tablero autenticado que expone datos de negocio.

Next.js lo materializa en la solución (Vercel, s. f.); su versión, su configuración y el detalle de las rutas se desarrollan en el §3.6 y en el §4.2.

2.6 Seguridad: autenticación, RLS y tokens de un solo uso

La solución articula tres mecanismos de seguridad complementarios. La autenticación por correo y contraseña restringe el acceso al tablero de control a usuarios registrados. La seguridad a nivel de fila garantiza que, aun con acceso a la API de datos, solo se puedan leer las filas autorizadas para el rol. Y los tokens de un solo uso protegen acciones públicas que no requieren cuenta —como la aceptación de una propuesta— asegurando que solo quien recibe el enlace pueda ejecutarla y que este no pueda reutilizarse. Este enfoque de defensa en profundidad es coherente con las buenas prácticas de seguridad en aplicaciones web (OWASP Foundation, 2025).

2.7 Encuadre normativo: datos personales y comprobantes fiscales

El sistema capta datos de contacto de personas físicas identificables y emite un documento que titula «FACTURA». Ambas cosas están reguladas en Argentina, de modo que cabe enunciar qué obligaciones son aplicables antes de declarar, en §1.9 y en el Capítulo 6, cuáles el prototipo no satisface. Encuadrar el límite es parte del trabajo: un prototipo académico puede legítimamente no cumplir un régimen que no lo alcanza todavía, pero no puede desconocerlo.

Protección de datos personales (Ley 25.326). La ley define como dato personal toda «información de cualquier tipo referida a personas físicas o de existencia ideal determinadas o determinables» (art. 2), categoría en la que caen el nombre, el correo, el teléfono y el presupuesto declarado que capta el formulario del §4.2.1. Ninguno es un dato sensible del art. 7, lo que excluye el régimen agravado, pero sí quedan alcanzados por el general. Cinco obligaciones son pertinentes. El art. 4 exige que los datos sean pertinentes y no excesivos respecto de la finalidad, que no se usen para fines incompatibles y que se destruyan cuando dejen de ser necesarios. El art. 5 exige consentimiento libre, expreso e informado. Su inciso 2.d exceptúa los datos que derivan de una relación contractual o profesional y resultan necesarios para su desarrollo, excepción que podría cubrir parte del tratamiento posterior a la aceptación de una propuesta pero no la captación inicial, anterior a toda relación. El art. 6 obliga a informar, al recolectar, la finalidad, los destinatarios, la identidad del responsable, el carácter obligatorio o facultativo de cada respuesta y los derechos de acceso, rectificación y supresión. El art. 9 exige medidas técnicas y

organizativas de seguridad. Y el art. 21 obliga a inscribir la base ante la Agencia de Acceso a la Información Pública.

Cesión y transferencia internacional. Es la dimensión más comprometida. El art. 11 sujeta la cesión a terceros al consentimiento previo del titular, informándole la finalidad e identificando al cesionario. El art. 12 prohíbe la transferencia a países que no proporcionen «niveles de protección adecuados», y aprueba cláusulas contractuales tipo para transferir hacia los que no los proporcionan. La Disposición 60-E/2016 enumera cuáles lo son: la Unión Europea y el Espacio Económico Europeo, Suiza, Guernsey, Jersey, Isla de Man, Islas Feroe, Canadá sólo respecto de su sector privado, Andorra, Nueva Zelanda, Uruguay e Israel sólo respecto de datos de tratamiento automatizado. Los Estados Unidos no integran esa lista, y allí se domicilian varios de los servicios a los que la arquitectura transfiere datos de contacto: Supabase, Notion, Telegram y el servicio de correo. La arquitectura híbrida que el §2.11 justifica por costo y control tiene, entonces, una consecuencia regulatoria que no se resuelve eligiendo mejor el proveedor: operar con clientes reales exigiría consentimiento expreso del titular para la transferencia, o bien las cláusulas contractuales tipo con cada encargado.

Comprobantes fiscales. La emisión está regida por el régimen de facturación electrónica que administra la Agencia de Recaudación y Control Aduanero (ARCA, continuadora de la AFIP desde 2024; ARCA, 2026). Un comprobante tiene validez fiscal sólo si el organismo le asigna un Código de Autorización Electrónico (CAE), que se solicita por servicio web al emitir. Debe consignar además CUIT del emisor, punto de venta habilitado, tipo de comprobante según la condición de emisor y receptor frente al IVA, y el propio CAE con su fecha de vencimiento. El documento que genera este sistema (§4.3.2) no solicita CAE, no declara CUIT ni punto de venta y numera con un identificador aleatorio dentro del año en lugar de la secuencia correlativa que el régimen supone. No es, por lo tanto, una factura en sentido fiscal sino un comprobante interno de la operación, y el trabajo lo declara así en §1.9.

Qué se sigue de esto para el alcance del trabajo. El prototipo no satisface ninguno de los dos regímenes, y esa distancia es admisible únicamente por lo que el trabajo es: un artefacto validado en un entorno controlado, con datos ficticios y sin operación con clientes reales, según el protocolo de la Tabla 14, que prohíbe expresamente usar datos personales reales. Lo que no sería admisible es operar en producción en ese estado, y por eso el cumplimiento normativo figura como condición previa a cualquier uso real y no como mejora deseable, en los mismos términos que las deudas de seguridad (Capítulo 8).

2.8 La carga administrativa como constructo

El capítulo cierra volviendo sobre el término que da origen al problema y que el §1.7 opera. «Carga administrativa» se emplea en el §1.1 y en el §1.2 con su sentido corriente — el tiempo que el profesional dedica a tareas que no producen el servicio que vende—, pero es un constructo con tratamiento propio en la literatura, y conviene apoyarse en él antes de definirlo operacionalmente.

Moynihan et al. (2015) lo descomponen en tres costos que no son intercambiables. Los de aprendizaje son el tiempo y el esfuerzo de averiguar qué hay que hacer y cómo; los de cumplimiento, la ejecución material de la tarea; y los psicológicos, el desgaste, la sensación de pérdida de control y la ansiedad que la tarea produce. La formulación se propuso para las interacciones entre el ciudadano y el Estado, y su traslado al trabajo independiente es analógico y no literal: la carga que aquí se aborda no la impone una autoridad sino el propio proceso comercial. Lo que se conserva del marco es la distinción, que es lo que importa.

Esa descomposición vuelve legible el alcance de lo que este trabajo hace. La automatización propuesta incide sobre los costos de cumplimiento: delega la ejecución material de las tareas repetitivas. No incide sobre los de aprendizaje —el profesional sigue teniendo que saber qué precio poner y qué alcance comprometer— ni permite afirmar nada sobre los psicológicos, que exigirían instrumentos que este trabajo no aplica. Cuando el §1.6 delimita el objetivo general, lo que hace es restringirse a un componente de los tres.

La medición tiene, además, una tradición metodológica establecida. El Modelo de Costos Estándar, adoptado en Europa para cuantificar la carga que la regulación impone a las empresas, la expresa como el producto de tres factores: el tiempo que insume cada acción, la frecuencia con que se repite y una tarifa que valoriza ese tiempo. Ntaliani y Costopoulou (2017) lo aplican a las pequeñas y medianas empresas y documentan la dificultad práctica de estimar el primero sin observación directa. La estructura del modelo explica qué mide y qué no la definición del §1.7. El conteo de acciones manuales por lead es el factor de frecuencia, mientras que el tiempo por acción y su tarifa quedan sin medir: son los que el Capítulo 8 propone obtener con un estudio con usuarios. No se trata, por lo tanto, de una medición incompleta por descuido sino de uno de los tres factores del modelo, declarado como tal.

Tabla 4. Componentes de la carga administrativa y su tratamiento en este trabajo

Componente del constructo	¿Lo aborda este trabajo?	Instrumento / por qué no
Costos de cumplimiento — factor de frecuencia (n.º de acciones manuales por lead)	Sí, se mide	Conteo derivado del artefacto con scripts/medir-carga-administrativa.mjs, que declara cada acción junto al nodo del flujo que la ejecuta
Costos de cumplimiento — factor de tiempo por acción y tarifa que lo valoriza	No	Exigen una línea de base previa y observación directa sobre operación real, o un estudio con usuarios (Capítulo 8)
Costos de aprendizaje	No	El sistema ejecuta la tarea; no enseña al profesional a decidir el precio ni el alcance del servicio
Costos psicológicos	No	Exigen instrumentos psicométricos, fuera del alcance del trabajo

Nota. Elaboración propia a partir de la descomposición de Moynihan et al. (2015), de los tres factores del Modelo de Costos Estándar y de la definición operacional del §1.7, que mide exactamente una celda de este esquema —el factor de frecuencia de los costos de cumplimiento— y así lo declara. Las tres celdas restantes quedan fuera del alcance de la validación por decisión de diseño, no por omisión.

2.9 Modelos de calidad de producto de software

La ingeniería de software emplea modelos de calidad para descomponer una noción amplia —«un buen producto»— en características evaluables por separado. Un modelo de calidad ordena esas características, les asigna sub-características y habilita criterios de verificación, en lugar de dejar la calidad librada a un juicio global (Pressman y Maxim, 2020; Sommerville, 2016).

ISO/IEC 25010 es el modelo de calidad de producto de la familia de normas SQuARE, sucesor de ISO/IEC 9126 y de uso estándar en la disciplina (ISO/IEC, 2023). Su edición de 2023 organiza la calidad del producto en características —entre ellas seguridad, fiabilidad, mantenibilidad, eficiencia de desempeño, capacidad de interacción y flexibilidad—, cada una con sub-características definidas.

Se adopta este modelo en el §3.5 por una razón concreta: sus sub-características son operacionalizables. Permiten enunciar cada requisito no funcional con un umbral, una unidad y un método de verificación (Tabla 11), de modo que pueda declararse cumplido o incumplido sin apelar a una valoración cualitativa. La norma aporta, además, un vocabulario común que hace comparable esta especificación con la de otros trabajos.

Las alternativas consideradas no ofrecen ese marco verificable: un conjunto de atributos definido ad hoc carecería de respaldo externo y de trazabilidad con la literatura; el modelo FURPS clasifica requisitos pero no fija sub-características ni criterios de medición; y una lista de métricas sueltas no garantiza cobertura de las dimensiones relevantes ni evita solapamientos. Por eso se prefiere ISO/IEC 25010 como referencia única, y es el modelo que el Capítulo 3 aplica para derivar los requisitos no funcionales.

Segunda parte. Estado del arte y antecedentes

2.10 Antecedentes y estado del arte

Antes de justificar la decisión de construir una solución propia corresponde situarla frente a las herramientas que ya atienden, parcial o totalmente, el mismo problema. La literatura reconoce que automatizar el marketing y las ventas es una práctica establecida en el ámbito B2B (Järvinen y Taiminen, 2016), y que su adopción organizacional ha sido objeto de estudio empírico reciente (Mero et al., 2020). Corresponde por eso revisar dos categorías de soluciones comerciales: los CRM con funciones de automatización y las plataformas de integración de bajo código (iPaaS). Un freelancer que centraliza captación, calificación y facturación es, en este sentido, un caso límite de pequeña y mediana empresa. Una metodología reciente para la automatización de procesos de negocio en PyMEs confirma la pertinencia de abordar este ciclo como un problema de automatización de procesos y no sólo de calificación de leads aislada (Moreira et al., 2025).

2.10.1 Protocolo de la revisión bibliográfica

El estado del arte de este capítulo combina dos relevamientos con propósitos distintos. El primero busca literatura arbitrada sobre los ejes conceptuales del trabajo. El segundo caracteriza cuatro productos comerciales a partir de la documentación de sus proveedores. Este apartado documenta el procedimiento del primero; la fuente del segundo se declara junto con la comparación.

Bases y fuentes consultadas. La búsqueda de literatura académica se realizó en Google Scholar, IEEE Xplore, ACM Digital Library, ScienceDirect/Scopus, SciELO y arXiv. Se consultaron además tres repositorios institucionales argentinos de acceso abierto—el Repositorio Institucional Abierto de la UTN, el SEDICI de la Universidad Nacional de La Plata y el RIDAA de la UNICEN—, para verificar si existían trabajos finales previos sobre un problema análogo. La consulta a los dos últimos se ejecutó el 2 de septiembre de 2026 con las mismas cadenas, restringidas por repositorio; su resultado se reporta en el §2.10.3. La documentación técnica y las páginas de precios de los proveedores se trataron como fuente separada, reservada a la comparación de productos. Las búsquedas se ejecutaron durante agosto de 2026 y la fecha de consulta de cada fuente recuperada en línea consta en la lista de Referencias.

Cadenas de consulta. Las búsquedas combinaron los términos de cada eje en inglés y en español. Se emplearon, entre otras, las siguientes cadenas en inglés: «low-code» OR

«no-code» AND «workflow automation» AND («SME» OR «small business»); «workflow orchestration» AND «n8n» AND («webhook» OR «event-driven»); «lead scoring» AND («rule-based» OR «model») AND «marketing automation»; «iPaaS» OR «integration platform as a service» AND («adoption» OR «requirements»); y «administrative burden» AND («measurement» OR «standard cost model») AND «SME». En español se emplearon «automatización de procesos» AND «pymes» AND «bajo código», «calificación de leads» AND «embudo de ventas» y «carga administrativa» AND «medición» AND «pymes».

Ventana temporal. Para la evidencia empírica se fijó el período 2015–2026. Se exceptuaron dos conjuntos de obras, por su condición de referencia canónica todavía vigente. El primero son las fundacionales de patrones, arquitectura y transacciones de larga duración: Fielding (2000), Fowler (2002), Hohpe y Woolf (2003) y Garcia-Molina y Salem (1987), esta última por ser la formulación original del patrón de saga que el §2.1 y el §4.3.2 emplean. El segundo son las que fundamentan el tipo de trabajo y su método de evaluación, invocadas en el §3.1: Hevner et al. (2004) y Peffers et al. (2007).

Criterios de inclusión y exclusión. Se retuvo cada fuente que fuera un artículo arbitrado o un preprint con metodología declarada y que tuviera pertinencia directa con uno de los ejes del trabajo: automatización de bajo código, gestión y calificación de leads, adopción de plataformas iPaaS, automatización de procesos en PyMEs o la carga administrativa como constructo. Se descartó el material sin metodología ni revisión, las fuentes no recuperables y la documentación de proveedor, esta última reservada para la comparación de productos con la salvedad de alcance ya declarada.

Selección. El cribado siguió las etapas de una revisión tipo PRISMA reducido, con los recuentos de la Tabla 5.

Tabla 5. Flujo de selección de la literatura arbitrada (revisión tipo PRISMA reducido)

Etapas	Recuento
Identificación (registros recuperados de bases y repositorios)	142
Cribado por título y resumen	46
Evaluación de texto completo	21
Incluidas en el estado del arte (11 arbitradas + 3 preprints)	14

Nota. Elaboración propia. Las catorce fuentes incluídas son las que sustentan el marco conceptual del capítulo. Las demás entradas de la lista de Referencias corresponden a normas, obras de referencia canónica, manuales, documentación técnica, páginas de proveedores y las fuentes que encuadran el contexto en el §1.1 y el §3.3, que cumplen otra función y quedan fuera de este conteo. Tampoco lo integra Césari et al. (2026), el antecedente que el §2.10.3 incorpora: procede de la consulta a repositorios institucionales y no del relevamiento en bases que estas etapas contabilizan.

Con este protocolo el relevamiento deja de ser una selección de oportunidad y pasa a ser una revisión: se buscó de forma sistemática y el resultado es el que estas subsecciones consignan.

2.10.2 Síntesis de la literatura arbitrada

La Tabla 6 sintetiza los once estudios arbitrados que sostienen el marco, con su objeto, su método, su hallazgo principal y su limitación declarada. Los tres preprints incorporados —Amir y Atif (2026), Lamanna (2025) y Rahmati et al. (2017)— se citan en el cuerpo como evidencia complementaria y se identifican en cada caso como material sin revisión por pares.

Tabla 6. Síntesis de la literatura arbitrada del estado del arte

Fuente	Objeto	Método	Hallazgo principal	Limitación de la fuente
Bucaioni et al. (2022)	Desarrollo de bajo código (low-code)	Revisión sistemática multi-vocal (literatura académica y gris)	El bajo código reduce, pero no elimina, la necesidad de programar; el campo carece de consenso terminológico	Heterogeneidad de la evidencia y ausencia de definiciones comunes en el campo
Wu et al. (2024)	Modelos de calificación (scoring) de leads	Revisión del estado del arte de los modelos de lead scoring	La ponderación de atributos por reglas explícitas sigue siendo el método predominante en la práctica; el aprendizaje automático exige volúmenes históricos	Campo sin marco de evaluación común ni conjuntos de datos comparables entre modelos
Moreira et al. (2025)	Automatización de procesos de negocio en PyMEs	Propuesta metodológica, del análisis de requisitos a una demostración práctica	Conviene abordar el ciclo comercial como automatización de procesos y no sólo como calificación de leads aislada	Validada en una única demostración práctica, sin operación en producción
Domański et al. (2023)	Plataformas low-code/no-code para PyMEs	Panorama comparativo con criterios declarados	Caracteriza y compara la oferta low-code/no-code para la digitalización de procesos de gestión en el segmento PyME	Alcance acotado a las plataformas seleccionadas, en un mercado que cambia rápido
Muhammad et al. (2025)	Factores de efectividad del «desarrollador ciudadano»	Revisión sistemática de literatura y modelo propuesto	Identifica los factores que habilitan al desarrollador ciudadano y propone un modelo para la transformación digital	Modelo conceptual, sin validación empírica
Neifer et al. (2021)	Requisitos de usuario para plataformas iPaaS	Diez entrevistas con expertos de proveedores de software como servicio	Los determinantes de adopción son la estandarización de los modelos de datos, la usabilidad y variedad de conectores, y la privacidad, seguridad y transparencia del dato	Muestra acotada (diez entrevistas) y perspectiva centrada en el proveedor
Coronado y Iglesias (2016)	Servicios de automatización de tareas para usuarios finales (Zapier, IFTTT y afines)	Caracterización conceptual de la categoría	Describe la categoría de automatización de tareas «para las masas» y su arquitectura de disparador y acción	Retrato de la categoría en un momento dado, sin evaluación empírica de productos
Moynihan et al. (2015)	La carga administrativa como constructo	Revisión conceptual y propuesta de un marco analítico	Descompone la carga en costos de aprendizaje, de cumplimiento y psicológicos, que no son intercambiables entre sí	Formulada para las interacciones entre el ciudadano y el Estado; su traslado a otros ámbitos es analógico y no literal
Ntaliani y Costopoulou (2017)	Reducción de la carga administrativa en PyMEs	Aplicación del Modelo de Costos Estándar al caso PyME	Documenta la dificultad práctica de estimar el tiempo por acción sin observación directa	El factor tiempo del modelo no puede medirse con fiabilidad sin observación directa

Fuente	Objeto	Método	Hallazgo principal	Limitación de la fuente
Järvinen y Taiminen (2016)	Automatización de marketing en marketing de contenidos B2B	Estudio de caso en una empresa B2B	La automatización alinea ventas y marketing y entrega contenido según la etapa del comprador; la calificación automática es práctica establecida	Un solo caso, con validez externa limitada
Mero et al. (2020)	Adopción organizacional de la automatización de marketing	Estudio empírico cualitativo sobre razonamiento causal y efectual	La adopción combina razonamiento causal y efectual y opera como un proceso de aprendizaje organizacional, no como una implantación puntual	Muestra acotada de organizaciones en un contexto específico

Nota. Elaboración propia a partir del uso que cada fuente recibe en este capítulo. La columna «Limitación de la fuente» recoge la restricción que señalan los propios autores cuando la explicitan; cuando no lo hacen, consigna una acotación inferida por los autores de este trabajo, conservadora y coherente con el método de la fuente. La columna no distingue caso por caso entre ambas situaciones, de modo que no debe leerse como si toda ella recogiera autocríticas de los originales.

A esa base arbitrada se suman tres preprints sin revisión por pares: Amir y Atif (2026) comparan la ejecución manual y automatizada de un flujo de captación con n8n; Lamanna (2025) propone un marco multicriterio para seleccionar plataformas de bajo código; y Rahmati et al. (2017) contrastan IFTTT y Zapier sobre 6.406 disparadores y acciones. Aportan evidencia complementaria, no concluyente.

La elección de HubSpot, Pipedrive, Zapier y Make responde a un criterio de inclusión fijado antes de la comparación, que explicita por qué el universo tiene esos cuatro productos y no otros:

dos categorías pertinentes al problema —CRM con automatización comercial y plataformas de integración de bajo código (iPaaS)—, que son las que atienden, parcial o totalmente, el ciclo descrito;

dos representantes por categoría, elegidos por su cuota de mercado y su presencia en evaluaciones de analista —Gartner y Forrester, citadas más adelante en este apartado— y por cubrir modelos de precio distintos: por asiento frente a por operación;

exclusión de las suites ERP completas y de las soluciones de código abierto autoalojables, tratadas por separado como categoría excluida (Tabla 29). La Tabla 7 caracteriza los cuatro productos.

Tabla 7. Caracterización de los cuatro productos comerciales comparados

Producto	Caracterización a partir de la documentación del proveedor
HubSpot	CRM todo en uno que combina pipeline de ventas, marketing por correo y una capa de automatización con scoring de leads configurable. Su plan gratuito cubre el registro básico de contactos, pero el scoring predictivo, los flujos de automatización avanzados y las integraciones profundas se habilitan en niveles de suscripción pagos por asiento. Los datos quedan alojados en la infraestructura del proveedor y la generación de documentos como facturas o propuestas requiere integraciones de terceros (HubSpot, s. f.).
Pipedrive	CRM centrado en la gestión visual del pipeline de ventas, con formularios web de captación y reportes de embudo. Al igual que HubSpot, su modelo comercial cobra por asiento y por mes; carece de un motor de

Producto	Caracterización a partir de la documentación del proveedor
	scoring configurable a nivel de reglas propias y no incluye generación nativa de facturación, por lo que requiere integraciones externas para cerrar el ciclo comercial completo (Pipedrive, s. f.).
Zapier	Plataforma de integración (iPaaS) que conecta aplicaciones SaaS mediante disparadores y acciones sin código. No constituye un sistema de registro propio: opera sobre los datos que ya residen en otras plataformas y cobra en función de la cantidad de tareas ejecutadas por mes, un modelo cuyo costo escala con el volumen de leads procesados (Zapier, s. f.).
Make (anteriormente Integromat)	Junto con n8n, una de las plataformas de automatización visual por escenarios más cercanas en filosofía al presente trabajo. A diferencia de n8n, Make se ofrece exclusivamente como servicio en la nube, con planes tarifados por operaciones ejecutadas; los datos del proceso transitan por la infraestructura del proveedor y no existe una alternativa de autoalojamiento equivalente a la que permite este trabajo (Make, s. f.).

Nota. Elaboración propia a partir de la documentación y de las páginas de precios de cada proveedor, consultadas el 17 de agosto de 2026 (HubSpot, s. f.; Pipedrive, s. f.; Zapier, s. f.; Make, s. f.).

Ninguna de estas soluciones cubre, en un único producto, el problema tal como se lo plantea aquí. HubSpot y Pipedrive ofrecen un CRM robusto, pero con reglas de calificación y de facturación limitadas o dependientes de terceros; Zapier y Make automatizan la integración entre sistemas, pero no son un sistema de registro con reglas de negocio propias. El hueco que este trabajo cubre es un pipeline propio y de extremo a extremo que encadena captación, normalización y scoring con reglas específicas (Tabla 16), generación y envío de la propuesta, aceptación mediante token de un solo uso, facturación en PDF, registro del pago y cierre.

2.10.3 Limitaciones del relevamiento

El relevamiento tiene las siguientes limitaciones de alcance.

El vacío identificado es de oferta comercial, no sólo de conocimiento. Los apartados anteriores incorporan literatura arbitrada sobre orquestación de bajo código (Bucaioni et al., 2022), automatización de procesos en PyMEs (Moreira et al., 2025) y modelos de calificación de leads (Wu et al., 2024), además de un estudio sobre la adopción de plataformas iPaaS. Neifer et al. (2021) entrevistaron a diez expertos de proveedores de software como servicio. Concluyen que los factores determinantes son la estandarización de los modelos de datos, la usabilidad y variedad de los conectores y los problemas de privacidad, seguridad y transparencia del dato. El tercero coincide con el argumento de custodia que el §2.11 invoca y con la dimensión normativa del §2.7, de modo que esa preocupación no es una preferencia de los autores sino un determinante documentado de la adopción. Lo que la literatura consultada no ofrece es una evaluación independiente de los cuatro productos que este apartado compara.

La consulta a los tres repositorios institucionales obliga a matizar lo que las versiones previas afirmaban. Ni el Repositorio Institucional Abierto de la UTN ni el RIDAA devolvieron trabajos finales previos sobre un problema análogo. SEDICI,

en cambio, devolvió un antecedente pertinente que este trabajo no había considerado: Césari et al. (2026), depositado bajo el identificador persistente 10915/196034. No es una tesina sino una ponencia presentada en el XXVIII Workshop de Investigadores en Ciencias de la Computación, de modo que satisface el criterio de inclusión del §2.10.1. Compara n8n con Make en arquitectura, licenciamiento y costo total de propiedad para PyMEs, documenta un caso real de migración desde Zapier y expone la aplicación de esas herramientas en la Cátedra de Bases de Datos de la UTN Facultad Regional Mendoza. La coincidencia es parcial: comparte el objeto y el criterio de costo total de propiedad de la Tabla 28, pero su unidad de análisis es la comparación entre plataformas y no la construcción de un artefacto que cierre el ciclo comercial hasta el comprobante. Su existencia refuerza la pertinencia del recorte, y de un modo más estrecho de lo supuesto: el mismo problema se aborda en la propia Facultad Regional Mendoza. Este apartado sigue siendo un análisis competitivo de cuatro productos comerciales y no un estado del arte exhaustivo, pero ya no carece de respaldo académico.

Dos antecedentes arbitrados estrechan esa laguna sin cerrarla. Domański et al. (2023) evalúan mediante una metodología arbitrada un conjunto de plataformas low-code/no-code orientadas a pequeñas y medianas empresas, el mismo segmento en el que se sitúa esta comparación. Muhammad et al. (2025) revisan sistemáticamente los factores que determinan la efectividad del «desarrollador ciudadano», lo que respalda con literatura arbitrada la figura que el §2.11 invoca para justificar la decisión de construir. Ninguno de los dos trata la adopción de plataformas iPaaS en particular — esa dimensión la cubre Neifer et al. (2021)— ni evalúa de forma independiente a los cuatro productos comparados, que es la laguna que se mantiene.

Las fuentes de la comparación son, en su mayor parte, las de los propios evaluados. Las cuatro herramientas se caracterizan a partir de su documentación y de sus páginas de precios, consultadas el 17 de agosto de 2026, fecha que consignan la nota de la Tabla 7 y cada celda de la Tabla 28. Para que la comparación no dependa además de un recorte arbitrario, las siete dimensiones que ordena la Tabla 29 se fijaron antes de comparar y se derivan del problema del §1.2. Son sistema de registro propio, reglas de calificación propias, cierre del ciclo hasta el comprobante, control sobre la lógica de negocio, custodia del dato, esfuerzo de puesta en marcha y madurez operativa. Constituyen una aplicación reducida del tipo de marco multicriterio que propone

Lamanna (2025) —un preprint sin revisión por pares, formulado precisamente para superar la brecha entre las comparaciones dirigidas por el marketing de los proveedores y una evaluación con criterios explícitos—, no una evaluación exhaustiva. Subsiste la limitación de fondo: no se incorporaron análisis independientes ni estudios de adopción de terceros, de modo que las afirmaciones funcionales sobre los cuatro productos describen lo que cada proveedor declara de sí mismo.

Un estudio de caso reciente matiza esa dependencia sin revertirla. Amir y Atif (2026), también preprint, comparan la ejecución manual y automatizada de un flujo de captación con n8n en un negocio de pequeña escala. Registran una reducción del tiempo de ejecución de 185,35 a 1,23 segundos y la eliminación de la tasa de error del 5 % observada en la ejecución manual. Aporta evidencia empírica independiente —preliminar y no arbitrada— a favor de automatizar el tipo de tarea que aborda este trabajo, pero no evalúa a HubSpot, Pipedrive, Zapier ni Make. La comparación de precios y prestaciones sigue apoyada, por lo tanto, en las páginas de los proveedores; lo que cambia es que la decisión de automatizar en sí ya no depende sólo de la argumentación propia.

Las evaluaciones de analista disponibles estrechan la laguna para dos de los cuatro productos y la dejan abierta para los otros dos. Para HubSpot, el análisis que Forrester publica sobre su Wave™ de CRM del primer trimestre de 2025 lo ubica como «Strong Performer» entre doce proveedores; la fuente consultada es una entrada del blog de Forrester —K. Leggett, «A market on the cusp of change: decoding the Forrester Wave™ for customer relationship management software», 31 de marzo de 2025— y no el informe completo, de acceso restringido. Para Zapier, el Magic Quadrant de Gartner para plataformas de integración como servicio, del 19 de mayo de 2025, lo evalúa junto a otros quince proveedores. De ese informe, también restringido, se consultó únicamente la ficha pública, de modo que se lo invoca por su existencia y su alcance declarado y no por su contenido. Ninguna de las dos es recuperable por un tercero en las condiciones en que se las consultó, de modo que no integran la lista de Referencias. Se las consigna aquí con todos sus datos, conforme al criterio que APA 7 reserva para las fuentes no recuperables, y ninguna afirmación funcional se apoya en ellas. Para Pipedrive y Make la búsqueda no encontró evaluación arbitrada ni de analista que los nombre. La única fuente independiente

son las reseñas agregadas de Gartner Peer Insights: Make con 4,6 sobre 5 en apenas 22 reseñas (Gartner, s. f.-a) y Pipedrive con 4,2 sobre 5 en 348 (Gartner, s. f.-b). El caso de Make es el más frágil, porque 22 es un tamaño muestral demasiado chico para sostener peso analítico.

El bajo costo operativo no se demuestra en este apartado, porque el §1.9 excluye los costos de infraestructura del alcance. Para no dejar el argumento sin cierre, el Capítulo 6 incorpora una estimación de costo total de propiedad a doce meses con supuestos declarados (Tabla 28) y una comparación funcional frente a las cuatro alternativas (Tabla 29).

El universo de comparación se restringe a productos comerciales de suscripción, lo que deja fuera una categoría pertinente: las soluciones de gestión y automatización autoalojables y de código abierto, que atienden parte del mismo problema sin cuota recurrente y bajo custodia propia del dato. Son, precisamente, las que discutirían con más fuerza dos de las cuatro razones que el §2.11 invoca para construir —el costo y el control sobre el dato—, y por eso la exclusión se declara en lugar de dejarse implícita. Responde al alcance del trabajo: el objetivo formativo del §1.4 no se satisface configurando una herramienta ya resuelta, sea comercial o libre, y su evaluación exigiría el marco multicriterio que este trabajo no aplica. Lo que el apartado acredita es, en consecuencia, que ninguno de los cuatro productos comerciales revisados cubre el ciclo completo en un único producto, no que no exista solución alguna que lo haga.

Dos alternativas concretas de esa categoría ilustran lo que la exclusión deja fuera: EspoCRM, un CRM bajo licencia AGPLv3, y Odoo Community, una suite de gestión bajo LGPL-3.0. Ambas son autoalojables sin costo de licencia y la Tabla 29 las incorpora como columna adicional, contrastando su cobertura de las siete dimensiones frente a la solución construida y a los cuatro productos comerciales. De ese contraste surge que Odoo Community, con sus módulos de CRM y facturación de fábrica, se acerca más al ciclo completo que las cuatro herramientas comparadas, mientras que EspoCRM aporta un scoring configurable por fórmulas pero no integra la emisión del comprobante. Ninguna ofrece de fábrica el encadenamiento captación–scoring–propuesta–token de aceptación–factura–cobro que este trabajo construye a medida. Ambas se caracterizan a partir de su documentación (EspoCRM, s. f.; Odoo, s. f.) y no de una evaluación independiente, de modo que la limitación de

fuente se extiende a esta categoría. Incorporarlas no cierra la laguna del apartado, pero corrige la asimetría de no haber considerado la categoría que más discutiría los argumentos de costo y de custodia.

2.11 Construir frente a configurar una herramienta existente

Dado que existen CRMs y plataformas de integración capaces de cubrir partes del problema, cabe preguntarse por qué se optó por construir una solución propia en lugar de configurar alguna de ellas. La respuesta se apoya en las cuatro razones que reúne la Tabla 8.

Tabla 8. Razones para construir una solución propia en lugar de configurar una existente

Razón	Fundamento
Costo y modelo de facturación.	Los planes de HubSpot, Pipedrive, Zapier y Make cobran cuotas recurrentes por asiento, por tarea o por operación, que crecen con el uso. El componente que concentra la lógica de negocio de esta solución —n8n— es autoalojable y no factura por ejecución, lo que traslada el costo a la infraestructura que lo hospeda. La ventaja es estructural, pero su magnitud no se demuestra aquí: la estimación se desarrolla en la Tabla 28 del Capítulo 6.
Control sobre el dato y sobre las reglas de seguridad.	Correr la orquestación sobre infraestructura propia permite definir las políticas de seguridad a nivel de fila y el esquema de tokens de un solo uso bajo control directo del equipo (véase §4.6 y el Anexo C), en línea con las recomendaciones de OWASP Foundation (2025). El sistema construido no otorga, sin embargo, propiedad total del dato. Solo n8n es efectivamente autoalojado. La persistencia (Supabase), el despliegue del frontend (Vercel), el correo (Gmail), la mensajería (Telegram) y el tablero externo (Notion) son, en cambio, servicios gestionados por terceros, por cuya infraestructura transitan los datos comerciales. La arquitectura resultante es híbrida y así se la declara; el Capítulo 6 desarrolla la limitación.
Personalización de las reglas de negocio.	Las reglas de scoring (Tabla 16) y el encadenamiento completo captación–propuesta–aceptación–factura–cobro–cierre son específicas del flujo comercial de un freelancer y no están disponibles llave en mano en los productos revisados. En cambio, estos ofrecen scoring genérico o requieren integraciones adicionales para completar el ciclo.
Objetivo formativo del trabajo.	El propósito declarado (§1.4, §1.5) es evidenciar competencias de integración de un Técnico Universitario en Programación mediante un artefacto de software verificable; operar una herramienta comercial ya configurada no habría permitido demostrar esas competencias de diseño e implementación.

Nota. Elaboración propia.

La decisión no está exenta de costos: construir implica asumir el desarrollo y el mantenimiento posterior del artefacto, responsabilidades que un producto comercial ya resuelto no exige. Configurar una herramienta existente sería preferible en un contexto distinto—un equipo con varios usuarios no técnicos, o una organización que priorice soporte comercial maduro por sobre la personalización—, escenario que excede el alcance de este trabajo (§1.9).

Para el alcance definido—un freelancer individual, con un ciclo comercial a medida, control directo sobre las reglas de negocio y un objetivo formativo explícito— construir está justificado. El aporte original no reside en un algoritmo aislado sino en el pipeline integrado y de extremo a extremo que encadena las etapas descritas, verificado componente a componente contra el código fuente (§4.1). De las cuatro razones, dos se sostienen sin reservas —personalización de las reglas y objetivo formativo— y las otras dos lo hacen de forma parcial: el argumento de costo se estima en el Capítulo 6 y el de propiedad del dato se limita a la capa de orquestación.

Capítulo 3: Marco Metodológico

Este capítulo describe la naturaleza del trabajo, la metodología de desarrollo adoptada, las técnicas de relevamiento, los requisitos que guiaron la construcción, las herramientas empleadas y la estrategia de validación del artefacto. La documentación del trabajo —incluidas las citas en el texto y la lista de referencias— se ajusta a las normas de estilo de la American Psychological Association en su séptima edición (American Psychological Association [APA], 2020).

3.1 Tipo y naturaleza del trabajo

Este trabajo final se inscribe en la categoría de desarrollo tecnológico aplicado: su resultado principal es un artefacto de software y no la contrastación de una hipótesis empírica, de modo que no se formula una hipótesis estadística y el estudio se orienta por la pregunta de investigación y por los objetivos. Su rigor se juzga por la justificación de las decisiones de diseño, la calidad de la solución y la validación del producto (Pressman y Maxim, 2020). El paradigma es pragmático-ingenieril y el enfoque, cualitativo-descriptivo con registro de indicadores puntuales de ejecución. El diseño no define población ni muestra, y esa ausencia es constitutiva del tipo de trabajo: el objeto de estudio es un artefacto singular, no un conjunto de casos sobre el que quepa inferir. Corresponde a lo que la literatura denomina investigación por diseño, cuyo rigor se juzga por la pertinencia del problema, la justificación de las decisiones y la evaluación del artefacto construido (Hevner et al., 2004; Peffers et al., 2007). De ese marco se toman dos exigencias que el resto del capítulo satisface: que la evaluación se planifique con método y criterio declarados de antemano (§3.7 y §3.8) y que el aporte se enuncie de forma acotada y verificable (§7.3). Ambas fuentes son anteriores a la ventana temporal del §2.10.1 y se incorporan bajo la excepción que allí se declara.

3.2 Metodología de desarrollo

Se adoptó un enfoque iterativo e incremental, apropiado para un producto mínimo viable cuyos requisitos se refinan a medida que el producto toma forma (Sommerville, 2016). El desarrollo avanzó por cinco incrementos funcionales, cada uno integrado y validado antes de abordar el siguiente. Fueron: (1) captación y persistencia del lead; (2) normalización, scoring y clasificación en niveles; (3) generación y envío de la propuesta, con la página pública de aceptación protegida por token; (4) facturación en PDF, registro del pago y cierre del proyecto; y (5) procesos programados y gestión interactiva posterior a la propuesta. El

criterio de cierre de cada incremento fue la ejecución satisfactoria de su camino principal y de al menos un camino alternativo. El versionado y la documentación de las decisiones se llevaron con Git.

3.3 Técnicas de relevamiento

Dada la naturaleza de este trabajo —un desarrollo tecnológico aplicado orientado a un producto mínimo viable (§3.1)—, el relevamiento de requisitos se realizó mediante análisis documental del dominio del trabajo freelance, y no mediante un estudio empírico con usuarios.

El corpus analizado se compuso de cuatro tipos de material. El primero es la literatura sobre gestión y automatización de leads (Järvinen y Taiminen, 2016; Mero et al., 2020; Kotler y Armstrong, 2017), de la que se tomaron las etapas del embudo y el concepto de calificación. El segundo son los datos sobre la expansión del trabajo independiente mediado por plataformas (Kässi y Lehdonvirta, 2018; OIT, 2025), que caracterizan el contexto. El tercero es la documentación funcional de las cuatro herramientas comerciales revisadas en §2.10, de la que se extrajeron las etapas que esos productos automatizan y las que dejan fuera. El cuarto, como fuente de contraste, es la experiencia directa de los autores en el desarrollo freelance, cuyo aporte y cuyas limitaciones se precisan al cierre del apartado.

El material se analizó mediante un procedimiento por categorías, definidas a partir del problema planteado en §1.2. Las categorías de análisis fueron tres.

Etapas del ciclo comercial: los estadios sucesivos por los que pasa una oportunidad, desde el primer contacto hasta el cobro.

Criterios de calificación: los atributos de una oportunidad que permiten priorizarla, es decir, presupuesto, urgencia, tipo de servicio y completitud del contacto.

Puntos de automatización o de decisión humana: para cada etapa, si admite ejecución automática o exige una decisión de negocio reservada al profesional.

Cada fuente se leyó buscando qué aportaba a esas categorías, y el ciclo de siete etapas —contacto, calificación, propuesta, seguimiento, aceptación, facturación y cobro— se derivó del cruce de las tres. De la literatura de gestión de leads se tomó la estructura del embudo y el concepto de calificación por reglas ponderadas; de la documentación funcional de las cuatro herramientas, las etapas que automatizan y las que dejan fuera, en particular la facturación y el cobro; y de los datos sobre la expansión del trabajo independiente, el encuadre del contexto. Una etapa se incorporó al ciclo sólo cuando aparecía, con ese nombre

o con otro, en al menos dos de las tres fuentes; la Tabla 9 registra qué aporta cada fuente a cada etapa y hace explícito ese cruce.

Tabla 9. Matriz fuente-etapa del análisis documental del ciclo comercial

Etapa del ciclo	Literatura de gestión y automatización de leads	Datos de expansión del trabajo independiente	Documentación funcional de las cuatro herramientas (§2.10)	Experiencia directa de los autores (contraste)
Contacto	Define el lead como contacto que manifiesta interés y como primer estadio del embudo (Kotler y Armstrong, 2017).	Caracteriza el canal por el que el freelancer recibe consultas y su dispersión (Kässi y Lehdonvirta, 2018; OIT, 2025).	Las cuatro ofrecen formulario web de captación y alta automática del contacto.	Confirma que el contacto llega sin estructura, repartido entre correo y mensajería.
Calificación	Aporta el concepto de scoring y la ponderación de atributos por reglas explícitas (Järvinen y Taiminen, 2016; Mero et al., 2020).	Sin aporte directo; encuadra la heterogeneidad de las oportunidades que se gestionan solo.	HubSpot y Pipedrive incluyen scoring configurable en niveles pagos; Zapier y Make no lo traen como capacidad nativa.	Fija los criterios y los umbrales iniciales de presupuesto, urgencia y servicio.
Propuesta	Sitúa la presentación de la oferta comercial como etapa del embudo.	Sin aporte directo.	Ofrecen plantillas de propuesta y envío por correo; los términos los define el usuario.	Establece que el precio, el plazo y el alcance son decisión del profesional y no se automatizan.
Seguimiento	Describe la automatización del recontacto de propuestas sin respuesta (marketing automation).	Sin aporte directo.	Las cuatro permiten secuencias de recordatorio programadas.	Señala el recontacto como la tarea que con más frecuencia se omite al gestionar a mano.
Aceptación	Ubica el cierre como último estadio del embudo.	Sin aporte directo.	Los CRM registran el cambio de estado; ninguna ofrece aceptación por enlace con token llave en mano.	Indica que la confirmación suele ser informal y queda sin registro.
Facturación	Fuera del embudo comercial clásico; no la cubre.	Señala la carga administrativa posterior a la venta que asume el trabajador independiente.	La generación del comprobante exige integración de terceros facturada aparte.	Describe una tarea manual de copia de datos del contacto al documento.
Cobro	No la cubre.	La incluye entre los costos administrativos que el profesional asume.	Requieren integrar una pasarela externa; no son sistema de registro del pago.	Indica que el control de pagos pendientes se lleva de memoria o en una planilla.

Nota. Elaboración propia. Las tres primeras columnas son los tipos de material del corpus descrito en el párrafo anterior; la cuarta es la experiencia de los autores como fuente de contraste (§3.3). Cada celda indica el aporte de la fuente a la etapa; «sin aporte directo» señala que la fuente no informa esa etapa.

El ciclo resultante se tradujo de forma directa al artefacto: cada estado de la Tabla 20 corresponde a una etapa y cada regla de negocio del §4.8, a una condición de transición. El procedimiento no empleó codificación cualitativa asistida por software ni criterio de saturación, porque el corpus es acotado y cerrado; el análisis es descriptivo y su carácter auditable se apoya en la trazabilidad fuente-etapa-artefacto y no en un recuento de frecuencias ni en la representatividad de una muestra.

La cuarta fuente —la experiencia directa de los autores— operó como contraste de las etapas derivadas y como criterio para fijar los umbrales iniciales del scoring. Tiene un estatuto distinto de las otras tres que conviene no equiparar: no es material recuperable ni auditable por un tercero, de modo que su aporte a la Tabla 9 debe leerse como contraste y no como evidencia. Los umbrales y las ponderaciones de la Tabla 16 que de ella se derivan

son, en rigor, supuestos de diseño declarados y no hallazgos. Se los adopta porque el trabajo necesita un punto de partida operable, y su revisión con datos de conversión se registra en el punto 8 del Capítulo 8. Se reconoce como limitación metodológica la ausencia de un relevamiento primario con usuarios reales: no se define una población y no puede juzgarse la validez externa de las reglas derivadas, que reflejan un criterio experto y no una distribución observada.

Circularidad de una de las fuentes. Corresponde declarar una circularidad del procedimiento, porque condiciona la lectura de la comparación del Capítulo 6. La documentación funcional de las cuatro herramientas cumple dos papeles a la vez. Es una de las tres fuentes de las que se deriva el ciclo de siete etapas (Tabla 9) y es, al mismo tiempo, el objeto de la comparación que el §2.10.3 y el §6.1 realizan contra ese mismo ciclo. El criterio con el que se juzga a esos productos proviene, entonces, en parte de lo que esos mismos productos automatizan. La consecuencia es acotada pero real: la conclusión de que ninguno de los cuatro cubre el ciclo completo no puede leerse como un hallazgo independiente de la fuente. La regla de las dos fuentes de tres la atenúa sin eliminarla: ninguna etapa se incorporó por aparecer sólo en la documentación de los proveedores. La comparación del §6.1 debe leerse, por eso, como un contraste de cobertura frente a un ciclo construido con participación de la fuente comparada, y no como una evaluación externa e independiente.

3.4 Requisitos funcionales

La Tabla 10 resume los requisitos funcionales (RF) que el sistema debe satisfacer. Cabe anticipar una limitación que el Capítulo 6 desarrolla: los umbrales y las ponderaciones que aplica el RF3 provienen del criterio experto descrito en §3.3 y no de datos de conversión, de modo que su validez es de diseño y no empírica.

Tabla 10. Requisitos funcionales del sistema

ID	Requisito funcional
RF1	Permitir a un interesado enviar sus datos mediante un formulario público con validación.
RF2	Normalizar y validar los datos recibidos y rechazar los que no cumplan las reglas mínimas.
RF3	Calcular un puntaje (score) y clasificar cada lead en un nivel HOT, WARM o COLD.
RF4	Generar y enviar por correo una propuesta a los leads HOT y WARM, con los términos comerciales —precio, plazo y alcance— fijados por el profesional.
RF5	Permitir la aceptación de la propuesta mediante un enlace con token de un solo uso.
RF6	Generar la factura en PDF y enviarla por correo al aceptar la propuesta.
RF7	Registrar el cobro de la factura y el cierre del proyecto.
RF8	Ejecutar seguimientos automáticos de las propuestas sin respuesta.
RF9	Enviar recordatorios de las facturas próximas a vencer o vencidas.
RF10	Notificar al profesional los eventos relevantes por Telegram y emitir un reporte diario de métricas.
RF11	Ofrecer un tablero de control autenticado con métricas, embudo y facturas pendientes en tiempo real.

Nota. Elaboración propia.

3.5 Requisitos no funcionales

La Tabla 11 presenta los requisitos no funcionales (RNF) organizados según las características de calidad de ISO/IEC 25010:2023, segunda edición del modelo vigente, que reemplaza a la de 2011 y renombra dos características: «Usabilidad» pasó a «Capacidad de interacción» y «Portabilidad», a «Flexibilidad». Cada requisito se enuncia con un umbral, una unidad y un método de verificación, de modo que pueda declararse cumplido o incumplido sin apelar a un juicio cualitativo. Esa exigencia obliga a que cada requisito sea atómico. Dos de los enunciados originales reunían dos atributos de calidad distintos bajo una sola etiqueta: la detención ante datos inválidos y la degradación defensiva del frontend, por un lado; la ausencia de desborde horizontal y la traducción de los códigos de error, por el otro. Un cumplimiento parcial no podía expresarse, así, sin ambigüedad. Se desagregaron por eso en RNF3a/RNF3b y RNF5a/RNF5b, conservando la numeración para no romper las remisiones; el resultado de su verificación se reporta en el Capítulo 5.

Tabla 11. Requisitos no funcionales del sistema (ISO/IEC 25010:2023)

ID	Característica (ISO/IEC 25010:2023)	Requisito no funcional verificable	Método y criterio de verificación
RNF1	Seguridad	El tablero solo es accesible para usuarios autenticados cuyo perfil tenga <code>role = 'admin'</code> . Todas las tablas de negocio tienen RLS habilitada y ninguna política de escritura para el rol <code>authenticated</code> .	Tres intentos de lectura de <code>leads</code> : sin sesión, con sesión de rol <code>user</code> y con sesión de rol <code>admin</code> . Criterio: 0, 0 y n filas respectivamente. Un <code>INSERT</code> con la clave pública debe ser rechazado.
RNF2	Seguridad	La aceptación de una propuesta exige un token UUID válido y no reutilizable: ninguna invocación posterior a la primera puede alterar el estado ni emitir una segunda factura.	Dos invocaciones secuenciales y dos concurrentes de <code>/lead-acepta</code> con el mismo token. Criterio: exactamente 1 fila en <code>facturas</code> para ese <code>lead_id</code> .
RNF3a	Fiabilidad	Ante datos inválidos, el flujo se detiene sin persistir el lead y deja un registro en logs.	Cuatro entradas inválidas (tres condiciones por separado más su combinación). Criterio: 0 filas nuevas en <code>leads</code> y al menos 3 filas nuevas en <code>logs</code> .
RNF3b	Fiabilidad	Ante la ausencia de las variables de entorno de la base de datos, el frontend se construye y despliega sin error, con degradación defensiva de los tres clientes de Supabase.	Construcción del frontend sin variables de entorno definidas. Criterio: código de salida 0.
RNF4	Mantenibilidad	Ningún componente del frontend escribe en las tablas de negocio: la totalidad de las escrituras se origina en el motor de orquestación.	Inspección estática del directorio <code>src/</code> . Criterio: 0 llamadas a <code>insert</code> , <code>update</code> , <code>upsert</code> o <code>delete</code> sobre tablas de negocio.
RNF5a	Capacidad de interacción (antes «Usabilidad»)	La interfaz se presenta sin desborde horizontal en anchos de 360 a 1920 px.	Revisión en cuatro anchos (360, 768, 1440 y 1920 px). Criterio: ausencia de desborde horizontal en los cuatro.
RNF5b	Capacidad de interacción (antes «Usabilidad»)	El 100 % de los códigos de error de autenticación que la plataforma puede devolver se muestra traducido al español.	Recuento automatizado de los códigos cubiertos por <code>translateAuthError</code> sobre el conjunto de códigos que la plataforma puede devolver en los tres flujos que la aplicación invoca (<code>signUp</code> , <code>signInWithPassword</code> y <code>verifyOtp</code>), declarados uno por uno en <code>tests/auth_errores.js</code> . Criterio: 100 % de cobertura.
RNF6	Eficiencia de desempeño	El evento de alta de un lead se propaga desde su confirmación en la base de datos hasta el cliente del tablero en menos de 3 segundos, sin recarga manual, en el percentil 95. El requisito acota el intervalo de propagación del evento entre el backend y el frontend, no el tiempo de repintado de la interfaz.	Veinte escrituras consecutivas sobre leads con una suscripción abierta al canal <code>postgres_changes</code> , midiendo el intervalo entre la escritura y la recepción del evento en el cliente. Criterio: los veinte eventos recibidos, con el peor caso por debajo de 3 s. El repintado de la fila añade a ese intervalo el tiempo de render del cliente, de orden de milisegundos, que esta medición no instrumenta.
RNF7	Flexibilidad (antes «Portabilidad»)	El motor de orquestación y el conversor de documentos se levantan en un entorno nuevo mediante <code>docker compose up -d</code> , sin editar código fuente.	Puesta en marcha desde cero en una máquina limpia con Docker instalado. Criterio: primer webhook operativo sin modificar archivos versionados.

Nota. Elaboración propia sobre las características de calidad de ISO/IEC 25010:2023.

3.6 Herramientas y tecnologías

La Tabla 12 detalla las herramientas empleadas, su versión y su función. Las versiones del frontend se tomaron de package.json; las de la capa de orquestación y del conversor de documentos, de docker-compose.yml; la de la API de Notion, de la cabecera Notion-Version que envían los nodos HTTP. Consignar el origen de cada dato permite que un tercero verifique la reproducibilidad sin depender de la memoria de los autores. La del motor de la base de datos merece una nota aparte, porque una versión previa de esta tabla la declaraba mal: se la había tomado del campo info.version del endpoint /rest/v1/, que informa la versión de PostgREST y no la de PostgreSQL. El error se detectó al ejecutar la verificación de seguridad a nivel de fila sobre una imagen de la versión mayor que la tabla declaraba, en la que el esquema no llegó a aplicarse. La versión que ahora consigna se deriva de una restricción verificable del propio artefacto, según detalla su última columna.

Tabla 12. Herramientas y tecnologías utilizadas, con su versión

Herramienta	Versión	Origen del dato	Función en el proyecto
Next.js	16.2.2	package.json	Framework del frontend (App Router, React Server Components).
React	19.2.0	package.json	Biblioteca de interfaz de usuario.
TypeScript	5.9.3	package.json	Lenguaje tipado del frontend.
Tailwind CSS	4.1.16	package.json	Framework de estilos utilitario.
@supabase/ssr	0.12.0	package.json	Clientes de Supabase para renderizado en servidor y gestión de sesión por cookies.
@supabase/supabase-js	2.108.2	package.json	Cliente de datos, autenticación y suscripciones en tiempo real.
n8n	n8n 2.34.6 (imagen fijada el 17 de agosto de 2026)	docker-compose.yml	Motor de orquestación de flujos de trabajo. La comprobación de que el flujo se importa en esta versión y queda activo se reporta en el §5.2.
Gotenberg	8	docker-compose.yml	Conversión de la factura de HTML a PDF.
Supabase	plataforma gestionada	consola del proyecto	Backend como servicio: base de datos, autenticación y tiempo real.
PostgreSQL	15 o superior	Restricción del propio esquema, comprobada con <code>npm run test:rls</code> sobre imágenes de distinta versión mayor (docs/evidencia-rls.md)	Motor de la base de datos gestionada por Supabase. La versión mayor se acredita por una restricción del propio esquema y no por la ficha del proveedor: <code>db/schema.sql</code> declara las dos vistas WITH (<code>security_invoker = true</code>), opción de vista que PostgreSQL incorporó en su versión 15, y ambas vistas existen en la instancia. Una edición 14 rechaza el esquema con «unrecognized parameter "security_invoker"» antes de crear política alguna, según comprueba <code>npm run test:rls --imagen postgres:14-alpine</code> .
API de Notion	2022-06-28	cabecera Notion-Version de los nodos HTTP	Registro visual de las oportunidades en un tablero externo.
Gmail (nodo de n8n)	la del motor de n8n	workflow/crm_postgres.json	Envío de propuestas, facturas y recordatorios por correo.
Telegram (nodo de n8n)	la del motor de n8n	workflow/crm_postgres.json	Notificaciones al profesional y reporte diario de métricas.
Vercel	plataforma gestionada	—	Despliegue del frontend.
Docker Compose	v5.3.1	docker compose version	Puesta en marcha del entorno local (Anexo G).
Git	2.55.0	git --version	Control de versiones del código fuente.

Nota. Elaboración propia a partir de los archivos de configuración del repositorio declarado en §4.1. La documentación técnica de cada herramienta se consigna en las Referencias (n8n, s. f.-a; Supabase, s. f.; Vercel, s. f.; PostgreSQL Global Development Group, 2024; Gotenberg, s. f.; Notion, s. f.; Docker Inc., s. f.).

3.7 Estrategia de validación del artefacto

Dado que este trabajo es un desarrollo tecnológico, la validación no se realiza mediante muestreo estadístico sino mediante la ejecución de escenarios controlados que

ejercitan los caminos principales y alternativos de cada flujo. Se definieron casos de entrada representativos —incluidas entradas inválidas— y se verificó que el sistema produjera los estados, las persistencias y las notificaciones esperadas. Los criterios de aceptación son la correcta normalización y validación de los datos, la asignación del score y del nivel conforme a la Tabla 16, la transición de estados prevista en la Tabla 20 y la actualización del tablero dentro del umbral del RNF6. Los resultados se presentan en el Capítulo 5.

La ausencia de inferencia estadística no exime de aportar datos —los tiempos, los conteos y los valores persistidos se reportan— ni de declarar las amenazas a la validez. Se registran cuatro. Primera, sesgo del evaluador: los autores construyeron y validaron su propio artefacto. Para atenderla, el 1 de septiembre de 2026 una persona ajena al equipo ejecutó el protocolo de la Tabla 14 sobre el repositorio declarado en §4.1; la constancia se reproduce en el Anexo K, de modo que la ejecución es auditable por un tercero. La amenaza queda atendida, no eliminada, y subsisten los dos límites que esa constancia declara: la ausencia de procedimiento ciego y que sólo tres de los seis instrumentos dejaban entonces evidencia fechada en archivo. Segunda, selección de los datos de prueba: los casos fueron contruidos por quienes definieron las reglas, y el §3.7.1 describe un conjunto derivado por partición del espacio de entrada que la mitiga en parte. Tercera, validez externa nula sobre la conversión comercial: los escenarios acreditan el comportamiento del sistema, no que el score prediga la conversión real de un lead (Capítulo 6). Cuarta, apoyo de herramientas de inteligencia artificial generativa, usadas para redactar y para verificar afirmaciones contra el código, siempre bajo revisión de los autores: no sustituyen la validación independiente, que aquí aporta la ejecución externa del Anexo K.

3.7.1 Casos de prueba por partición del espacio de entrada

La segunda amenaza declarada más arriba —los casos de prueba contruidos por quienes definieron las reglas— se mitiga en parte con un conjunto de casos derivado del propio espacio de entrada. Definimos ese conjunto por partición sistemática: clases de equivalencia y valores límite sobre las cinco variables que intervienen en el cálculo del score. Las clases de equivalencia y los valores límite de cada variable, tomados de la Tabla 16, se reúnen en la Tabla 13.

Tabla 13. Clases de equivalencia y valores límite de las cinco variables del score

Variable	Clases de equivalencia	Valores límite probados
Presupuesto (USD)	Cinco clases válidas según los tramos: menor que 300; de 300 a 999; de 1.000 a 1.999; de 2.000 a 4.999; y 5.000 o más.	299 y 300 · 999 y 1.000 · 1.999 y 2.000 · 4.999 y 5.000
Urgencia	Tres clases válidas —alta, media y baja— y una clase inválida: todo valor fuera de ese conjunto, que la normalización asigna a media.	No aplica: la variable es categórica y no tiene fronteras numéricas.

Variable	Clases de equivalencia	Valores límite probados
Servicio	Siete clases: las seis opciones del formulario —«Otro» entre ellas— y una clase para todo valor que la normalización no reconoce; «Otro» y esta última se ponderan como consultoría (§4.3.1).	No aplica: la variable es categórica y no tiene fronteras numéricas.
Teléfono	Dos clases según la longitud de la cadena: seis caracteres o menos, que no suma puntos, y más de seis, que suma cinco.	Seis y siete caracteres.
Descripción	Dos clases según la longitud: cincuenta caracteres o menos, que no suma puntos, y más de cincuenta, que suma cinco.	Cincuenta y cincuenta y un caracteres.

Nota. Elaboración propia.

La generación del conjunto es automatizable —basta combinar una representante de cada clase con los pares de valores límite de cada frontera— mientras que la ejecución es humana e instrumentada. `tests/scoring.js` ya recorre 9.240 combinaciones de las cinco variables y verifica que el resultado reproduzca la Tabla 16 (§5.4). Ese barrido cubre buena parte del espacio de entrada, y la partición agrega el foco explícito en los valores límite y un subconjunto reducido y trazable, en el que cada caso se justifica por la clase o la frontera que ejercita.

La técnica mitiga parcialmente la segunda amenaza: los datos de prueba dejan de depender del criterio de quienes escribieron las reglas y pasan a derivarse de la estructura de la propia tabla de scoring. No corrige por sí sola la primera, el sesgo del evaluador: de ésa se ocupa la ejecución externa del Anexo K, con el doble límite que allí se declara.

Resultado del barrido. Corresponde reportar qué cubre efectivamente el instrumento, porque de otro modo la técnica quedaría enunciada y no ejecutada. `tests/scoring.js` combina catorce valores de presupuesto, cinco de urgencia, once de servicio, cuatro de teléfono y tres de descripción —las 9.240 combinaciones del §5.4— y contrasta cada resultado con la Tabla 16; la corrida termina en diez comprobaciones en verde y sin divergencias. De las cuatro fronteras numéricas del presupuesto que fija la Tabla 13 se ejercitan las cuatro, con sus dos valores contiguos: 299 y 300, 999 y 1.000, 1.999 y 2.000, y 4.999 y 5.000. Las clases de urgencia y de servicio se recorren completas, incluidas la urgencia ajena al conjunto admitido y el servicio que la normalización no reconoce. Quedan cubiertas de un solo lado, en cambio, las dos fronteras de longitud. El barrido prueba cadenas de cinco y de dieciséis caracteres para el teléfono, y de cinco y de cincuenta y uno para la descripción, de modo que no ejercita los valores exactos de seis y de cincuenta que separan una clase de la otra. Completar esos dos pares es una modificación del conjunto de datos del propio instrumento y se registra en el punto 7 del Capítulo 8.

3.8 Protocolo de ejecución y registro de la validación

Para que los escenarios sean repetibles por un tercero, la Tabla 14 fija el protocolo bajo el cual deben ejecutarse y registrarse. La exigencia central es que cada ejecución deje

constancia del valor observado —no de su paráfrasis—: el score numérico, el identificador del lead, el estado persistido y la marca temporal.

Tabla 14. Protocolo de ejecución y registro de la validación

Elemento del protocolo	Especificación
Entorno de ejecución	Frontend en el despliegue de Vercel; n8n y Gotenberg en contenedores locales levantados con docker compose up -d (Anexo G); base de datos en el proyecto gestionado de Supabase. Máquina de ejecución: Microsoft Windows 11 Pro, versión 10.0.26100; Docker 29.6.2 (build dfc4efb) con Docker Compose v5.3.1.
Versiones	Las declaradas en la Tabla 12, que deben consignarse en el informe de cada corrida.
Conjunto de datos de prueba	Un lead por escenario, con datos ficticios y neutros. Queda prohibido el uso de datos personales reales de los autores o de terceros y de identificadores no aptos para un documento institucional. Los nombres siguen el patrón <i>Cliente Demo N</i> y los correos, el dominio <i>ejemplo.com</i> .
Repeticiones por escenario	Tres ejecuciones independientes para los escenarios de camino principal (E1–E7) y una para los de camino alternativo (E8–E10), salvo E6, que por definición exige dos invocaciones consecutivas y dos concurrentes. Los escenarios de procesos programados (E11–E13) se disparan una vez desde el editor de n8n, por no tener camino automatizado de invocación. E14, E15 y E16 se ejecutan una vez por corrida de su instrumento, que es idempotente y repetible: E14 contra el doble de la pasarela, E15 sobre un lead que crea y elimina, y E16 sobre los trece webhooks. Se declara esta diferencia en lugar de presentar todas las corridas como equivalentes.
Registro de la evidencia	Por cada ejecución se registran la fecha y la hora, la entrada exacta y la fila resultante de leads o facturas, tomada de la consola SQL. La evidencia complementaria admite dos formas válidas, según el escenario. (a) En los escenarios con interfaz o con invocación automatizada del webhook: captura de la pantalla correspondiente y fila del registro de auditoría en logs. (b) En los procesos programados sin camino de invocación automatizado (E11 a E13), que se disparan desde el editor de n8n: reconstrucción a partir del registro contemporáneo a la corrida en logs y de la respuesta de las API invocadas —por ejemplo, el <i>message_id</i> que devuelve Telegram—, declarada de forma expresa como reconstrucción. Ambas formas son admisibles siempre que el valor observado quede registrado y sea verificable por un tercero.
Criterio de escenario fallido	Un escenario se declara fallido si alguna de sus repeticiones no alcanza el estado esperado, no persiste la fila esperada o supera el umbral del requisito no funcional asociado. No se admite promediar resultados: basta una repetición fallida.
Reejecución tras corrección	Toda corrección del flujo obliga a reejecutar el escenario completo, no solo el paso corregido.

Nota. Elaboración propia.

Capítulo 4: Diseño y Desarrollo del Sistema

Este capítulo describe la arquitectura de la solución y el detalle de cada componente. Las afirmaciones sobre el frontend se apoyan en el código fuente del repositorio del proyecto (disponible en <https://github.com/Boit-6/Tesis>); las relativas al backend de automatización, en el flujo de trabajo exportado desde n8n. Todas las cifras y descripciones de este capítulo corresponden a un estado preciso del repositorio: el commit d1abe08, del 31 de agosto de 2026, que es la versión sobre la que deben verificarse y la que ejecutó de forma independiente el evaluador externo del Anexo K. El repositorio siguió recibiendo trabajo después de esa fecha; el §5.6 declara qué cambió, con qué efecto sobre estas cifras y por qué esos cambios quedan fuera de lo que este documento acredita.

4.1 Arquitectura general

El sistema adopta una arquitectura en tres capas coordinada por eventos (Fowler, 2002), que la Tabla 15 describe.

Tabla 15. Capas de la arquitectura y su responsabilidad

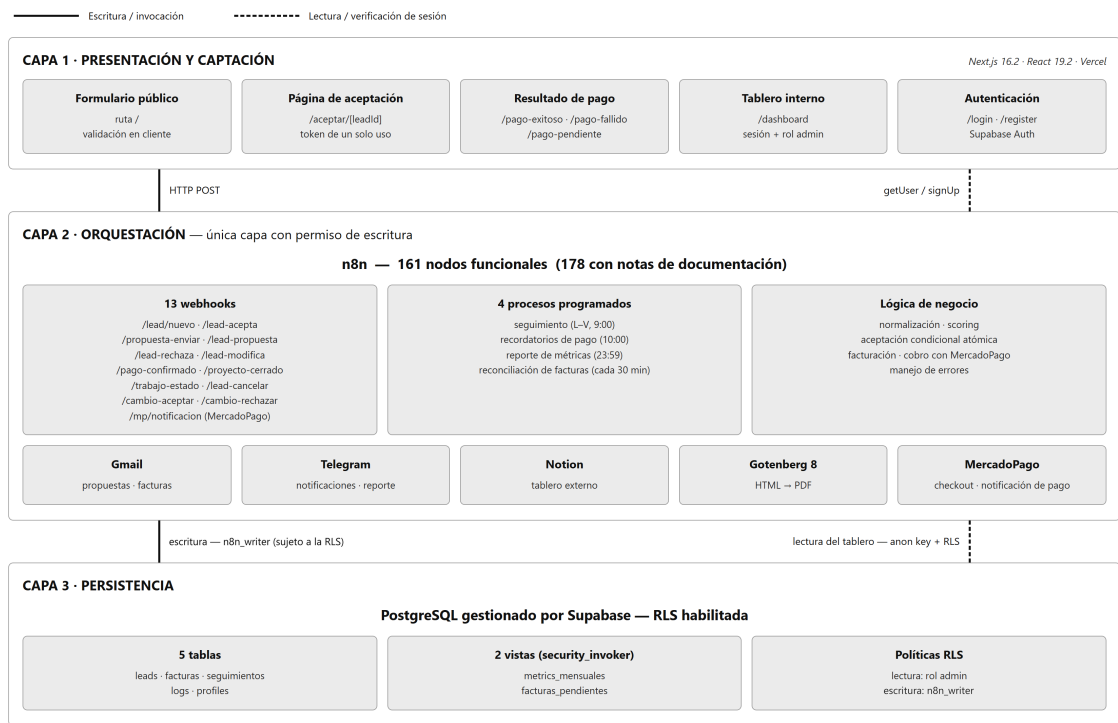
Capa	Componentes y responsabilidad
Capa de presentación y captación (frontend).	Aplicación Next.js desplegada en Vercel. Expone el formulario público de captación, las pantallas de autenticación, el tablero de control y la página de aceptación de propuestas.
Capa de orquestación (backend de automatización).	Instancia de n8n que recibe los eventos por webhook, ejecuta la lógica de negocio y coordina la persistencia y las notificaciones. El flujo implementado consta de 161 nodos funcionales —178 en total, incluidas 17 notas de documentación— organizados en trece webhooks y cuatro procesos programados.
Capa de persistencia (datos).	Base de datos PostgreSQL gestionada por Supabase, con las tablas leads, facturas, seguimientos, logs y perfiles, las vistas metrics_mensuales y facturas_pendientes, y políticas de seguridad a nivel de fila.

Nota. Elaboración propia. La Figura 1 representa las mismas tres capas y el sentido de las comunicaciones entre ellas.

El frontend nunca escribe directamente en la base de datos de negocio: se comunica con n8n mediante webhooks, y n8n es el único componente que escribe en las tablas de negocio. El tablero de control, en cambio, lee directamente de Supabase en modo solo lectura, amparado por las políticas de RLS. Esta separación concentra la lógica de integración en el motor de orquestación (RNF4) y mantiene el frontend delgado.

La Figura 1 representa la arquitectura y el sentido de las comunicaciones entre capas; el detalle de cada una se desarrolla a continuación.

Figura 1. Arquitectura en tres capas del sistema, con los puntos de entrada de la capa de orquestación y el sentido de las escrituras y lecturas.



Nota. Elaboración propia a partir del flujo exportado `workflow/crm_postgres.json` y del esquema `db/schema.sql` del repositorio declarado en §4.1.

4.2 Frontend (Next.js)

El frontend se organiza con el App Router de Next.js; las rutas se enumeran en el Anexo B. Las principales son el formulario público (/), las pantallas de autenticación (/login, /register), el tablero protegido (/dashboard) y la página de aceptación (/aceptar/[leadId]).

4.2.1 Formulario público de captación

El componente `lead-form.tsx` presenta el formulario con los campos nombre, correo, teléfono (opcional), tipo de servicio, urgencia, presupuesto —un control deslizante entre USD 100 y USD 20.000— y descripción. El desplegable de servicio ofrece seis opciones: «Desarrollo Web», «Diseño UX/UI», «Marketing Digital», «SEO / Posicionamiento», «Consultoría» y «Otro». La validación se realiza en el cliente y, al enviar, el componente emite una petición POST al webhook de captación de n8n, cuya dirección base configura la variable `NEXT_PUBLIC_N8N_BASE`; ante una respuesta satisfactoria muestra una confirmación y ante un error, una alerta. Con esto se satisface el OE1 (Figura 5). El §4.3.1 analiza tres consecuencias de este diseño: el desajuste entre el rango del deslizante y los tramos de puntaje, los cuatro valores ponderados que no pueden originarse en este formulario

y el sesgo con que la opción «Otro» penalizaba a los leads que el sistema no supo clasificar. La Figura 17 documenta el comportamiento de la validación ante una entrada inválida.

4.2.2 Autenticación con Supabase Auth

El acceso al tablero se resuelve con Supabase Auth por correo y contraseña. El registro (`register-form.tsx`) invoca `signUp` —crea la cuenta y dispara el correo de confirmación— con redirección a `/auth/confirm`, y el inicio de sesión (`login-form.tsx`) invoca `signInWithPassword`, que valida las credenciales y abre la sesión. El manejador `/auth/confirm` verifica el código de un solo uso (`verifyOtp`, que canjea el enlace del correo por una sesión válida) y el manejador `/auth/signout` cierra la sesión. Los códigos de error de Supabase se traducen a mensajes en español mediante la función `translateAuthError` del módulo `auth-errors.ts`, en cumplimiento del RNF5b.

4.2.3 Protección de rutas

La protección de rutas se resuelve en dos niveles, en un esquema de defensa en profundidad. El primero es la capa de proxy —la convención vigente en Next.js 16, que reemplaza al histórico `middleware.ts`—. En cada petición comprueba si la ruta pertenece al conjunto de rutas protegidas, que incluye `/dashboard`, y valida la sesión contra Supabase. Si no hay usuario, redirige a `/login` conservando el destino original.

Conviene precisar la naturaleza de esa capa: no se ejecuta en el borde ni implementa por sí misma la protección de las rutas. `proxy.ts` corre en el entorno de Node.js. El renombre de `middleware.ts` a `proxy.ts` se introdujo precisamente para señalar que esa capa es infraestructura de enrutamiento y no un mecanismo de seguridad: una vulnerabilidad de alta severidad —CVE-2025-29927, con puntaje CVSS 9.1— había demostrado que las verificaciones de autenticación alojadas allí podían eludirse falsificando la cabecera `x-middleware-subrequest` (NVD, 2025). Documentar la capa de proxy como control efectivo invertiría el orden de importancia entre la comprobación indicativa y la real.

El segundo nivel —y el único que constituye un control de acceso efectivo— es el componente de servidor de la ruta protegida. `dashboard/page.tsx` vuelve a resolver `getUser` y, además, consulta la tabla `profiles` y redirige a la raíz a todo usuario cuyo `role` no sea `admin`. No alcanza, por tanto, con estar autenticado: el tablero exige rol de administrador. La misma exigencia se replica en la capa de datos mediante las políticas de seguridad a nivel de fila (§4.6 y Anexo C), de modo que el control se aplica tres veces: enrutamiento, componente de servidor y base de datos.

4.2.4 Cliente de Supabase y degradación defensiva

La conexión con Supabase usa tres clientes especializados —navegador, servidor y capa de proxy—, todos con degradación defensiva ante variables de entorno ausentes. Si faltan, devuelven un valor nulo en lugar de interrumpir la construcción de la aplicación, lo que permite desplegar sin configurar y evita caídas del despliegue (RNF3b). El detalle de los tres archivos se reproduce en el Anexo B.

4.2.5 Tablero de control en tiempo real

El tablero (dashboard-client.tsx) consulta siete conjuntos de datos: los indicadores del mes desde metrics_mensuales, el embudo a partir del estado de los leads, los leads recientes, las facturas pendientes desde facturas_pendientes, los trabajos activos, los pedidos de cambio pendientes y las propuestas por enviar. Encabeza el tablero esta última bandeja, desde la cual el profesional fija el precio, el plazo y el alcance de cada propuesta antes de enviarla. Presenta además los indicadores clave, un gráfico de embudo por estado y tablas de leads y de facturas con resaltado de vencimientos. La definición operacional de cada indicador está en la Tabla 19, y su lectura es imprescindible porque los indicadores del mes y el embudo por estado no comparten alcance temporal. Para mantener la información actualizada sin recarga manual, el tablero se suscribe a los cambios de leads por el canal postgres_changes y vuelve a consultar los datos ante cada alta, modificación o baja (RNF6). Los eventos se agrupan en una sola recarga porque cada uno obliga a repetir las siete consultas: sin agrupar, un proceso programado que actualiza varios leads a la vez disparaba una ráfaga proporcional a las filas afectadas, posiblemente mayor que el sondeo periódico que la suscripción viene a evitar. Estas prestaciones satisfacen el OE4b y, junto con el modelo de datos, el OE4a (Figura 6).

4.2.6 Página de aceptación de propuestas

La ruta pública /aceptar/[leadId] recibe, además del identificador del lead, un token en el parámetro de consulta, porque así llega el enlace enviado por correo. Desde esa página el cliente dispone de tres acciones —aceptar la propuesta, solicitar cambios o rechazarla— y el componente envía el identificador y el token en el cuerpo de una petición POST al webhook correspondiente. En el caso de la aceptación se informa uno de tres resultados: aceptación satisfactoria, enlace ya utilizado o enlace inválido o vencido. Al no requerir cuenta, la ruta se protege exclusivamente con el token y no con Supabase Auth (Figura 7). La Figura 18 reúne los cuatro desenlaces posibles.

4.3 Backend de automatización (n8n)

El flujo de trabajo exportado desde n8n contiene 161 nodos funcionales, 178 en total incluidas 17 notas de documentación. Organiza la lógica de negocio en varios procesos: captación y calificación de leads; envío de la propuesta; aceptación, con sus variantes de rechazo y pedido de cambios; facturación; pago; cierre de proyecto; gestión del estado del trabajo; cancelación del lead; y un conjunto de procesos programados. Cada proceso se describe a continuación por lo que hace, el orden lógico de sus pasos y los riesgos que plantea; el detalle de sus nodos principales, con el disparador de cada proceso, se reúne en la Tabla 34 del Anexo B.

4.3.1 Captación y calificación (webhook de nuevo lead)

El proceso se dispara con el webhook POST /lead/nuevo. Primero se normalizan los campos: se recortan los espacios, el correo se pasa a minúsculas y el presupuesto se interpreta como entero; la urgencia se restringe al conjunto {alta, media, baja}, con media como valor por defecto, y el servicio se traduce a una lista blanca de nueve valores mediante un mapeo explícito. Se aplican tres validaciones mínimas —correo con arroba, nombre de al menos dos caracteres y presupuesto mayor que cero— y, si alguna falla, el flujo se detiene con un error. En caso contrario se genera un identificador único con el formato LD-<timestamp>-<aleatorio> y se trunca la descripción a 2000 caracteres.

A continuación se calcula el puntaje de prioridad sumando cinco componentes ponderados (Tabla 16) y se deriva el nivel: HOT si el puntaje es mayor o igual a 70, WARM si es mayor o igual a 40, y COLD en caso contrario. Recién entonces el lead, ya calificado, se persiste en la tabla leads: el cálculo del puntaje precede a la persistencia y no al revés (§4.8). Un condicional bifurca el flujo. Los leads HOT y WARM quedan a la espera de que el profesional fije los términos —precio, plazo y alcance— desde el tablero interno, y sólo entonces se genera y se envía la propuesta (§4.3.2). Los COLD no reciben propuesta, para no invertir esfuerzo comercial en oportunidades de baja prioridad (OE2).

Una versión anterior sólo generaba en la rama fría una notificación interna por Telegram, de modo que el lead COLD no recibía ningún correo pese a que el formulario anuncia una respuesta dentro de las 24 horas. El sistema pensado para evitar una imagen poco profesional (§1.1) producía justamente ese caso, y le devolvía al profesional la tarea manual que el §1.2 señala como debilidad. Se incorporó por eso un acuse de recibo por correo, que explica qué sigue sin comprometer una propuesta y que cuelga en paralelo de la

notificación interna, para que una falla del aviso al profesional no impida el correo al cliente (Anexo I). La Figura 16 muestra ejemplos de leads clasificados en los tres niveles.

Tabla 16. Reglas de cálculo del score

Criterio	Condición	Puntos
Presupuesto (USD)	≥ 5.000	40
	2.000 – 4.999	30
	1.000 – 1.999	20
	300 – 999	10
	< 300	0
Urgencia	alta	30
	media	15
	baja	5
Servicio	desarrollo_web · ecommerce	20
	app_movil · automatizacion	18
	diseno_ui · marketing · seo	15
	consultoria (incluye la opción «Otro» del formulario y todo valor que la normalización no reconoce)	12
	soporte	5
	valor no contemplado en la lista (respaldo del nodo de scoring; inalcanzable en el flujo real, véase la nota)	8
Teléfono	longitud mayor que 6 caracteres	+5
Descripción	longitud mayor que 50 caracteres	+5

Nota. Elaboración propia a partir del nodo Code - Scoring del flujo exportado. Clasificación en niveles: $\text{score} \geq 70 \rightarrow \text{HOT}$; $40 \leq \text{score} < 70 \rightarrow \text{WARM}$; $\text{score} < 40 \rightarrow \text{COLD}$. El puntaje máximo alcanzable es 100. La fila de respaldo (8 puntos) es inalcanzable en el flujo real, porque el nodo Normalizar Lead traduce todo valor a la lista blanca antes de puntuar: un servicio no reconocido recibe la ponderación de consultoria, la misma que la opción «Otro» (§4.3.1). Se la documenta por completitud del nodo leído de forma aislada.

Tras el envío de la propuesta, el lead pasa al estado PROPUESTA_ENVIADA y se registra la fecha; queda constancia en el registro de auditoría y se notifica al profesional por Telegram (Figura 8, Anexo A).

Dos riesgos de este proceso quedan por consignar; el primero se resolvió en el curso del trabajo y el segundo sigue abierto. El primero era la ausencia de idempotencia en la captación: el webhook no declaraba clave de idempotencia, ni deduplicación por correo, ni bloqueo del botón de envío, de modo que un doble clic generaba dos leads con identificadores distintos, dos cálculos de score y dos avisos por el mismo interesado. La garantía se había aplicado donde el riesgo era la doble facturación (§4.3.2 y §4.3.3) y no donde el riesgo era el duplicado comercial. Se cerró en las dos puntas que el diagnóstico proponía: en el formulario, el envío queda bloqueado mientras la petición está en vuelo (Anexo I).

En el flujo, la inserción del lead pasó a ser condicional: no inserta si en los últimos DEDUP_VENTANA_MINUTOS —diez por defecto— ya entró un lead con el mismo correo, y sin fila el proceso termina sin segundo puntaje ni segundo aviso. Dos precisiones

importan. El webhook responde en el momento en que recibe la petición, de modo que quien hizo doble clic ve éxito y no un error. Y la comparación por correo, sola, no alcanzaba: dos peticiones solapadas abren transacciones distintas y, bajo el nivel de aislamiento por defecto de PostgreSQL, ninguna ve la fila que la otra no confirmó todavía; se comprobó ejecutando dos envíos simultáneos, que insertaban dos leads igual. La inserción se serializa por eso con un cerrojo consultivo sobre el correo, que sólo una de las dos transacciones obtiene.

El segundo riesgo es que un lead calificado como HOT o WARM quede sin que el profesional envíe la propuesta. Desde que esa decisión pasó a depender de una acción manual desde el tablero (§4.3.2), el estado NUEVO ya no es sólo el punto de partida del ciclo sino también la sala de espera de toda propuesta pendiente. El sistema avisa por Telegram en el momento en que el lead queda a la espera y el tablero lo muestra en primer lugar con su recuento, pero si el profesional no actúa nadie vuelve a insistir. Un proceso programado que detecte leads HOT o WARM sin propuesta enviada con antigüedad superior a un umbral resolvería el caso, y se registra en el Capítulo 8.

El mapeo de servicios merece una precisión, porque son dos reglas diferentes que conviene no confundir:

Mapeo explícito de las opciones del formulario. «Desarrollo Web» → `desarrollo_web`; «Diseño UX/UI» → `diseño_ui`; «Marketing Digital» → `marketing`; «SEO / Posicionamiento» → `seo`; «Consultoría» → `consultoria`; y «Otro» → `consultoria`. Es decir, «Otro» no cae en un valor por omisión: tiene una traducción declarada, que es `consultoria`.

Valor de respaldo de la normalización. Cuando el campo llega vacío, ausente o con un valor que no figura en el mapeo —caso posible si el webhook se invoca desde fuera del formulario, ya que no verifica el origen (§4.6)—, el nodo asigna `consultoria`; este valor no se aplica nunca a una selección hecha en el formulario. En una versión anterior el respaldo era `desarrollo_web`, es decir la ponderación máxima de la tabla: un servicio que el nodo no lograba leer puntuaba como el mejor de la lista. Se lo alineó con la traducción de «Otro» (Anexo I) porque ambos describen el mismo estado de conocimiento —el sistema no sabe qué pidió el cliente— y no hay criterio que justifique resolverlos con pesos opuestos.

De este modo, toda selección del formulario recibe una ponderación de servicio determinada y trazable. Tres consecuencias del diseño acotan, no obstante, el alcance del modelo de puntaje:

Cuatro valores ponderados son inalcanzables desde el formulario. La lista blanca contempla nueve servicios, pero las seis opciones del desplegable se traducen a sólo cinco, porque «Consultoría» y «Otro» comparten destino. Los valores ecommerce (20 puntos), app_movil (18) y automatizacion (18) —los de ponderación alta e intermedia— y también soporte (5), el más bajo, sólo pueden originarse si el webhook se invoca desde fuera del formulario; es por esa vía que el escenario E3 ejercita soporte. En la práctica, casi la mitad del modelo de ponderación no se ejercita por la única fuente de entrada declarada. La decisión de conservarlos responde a la previsión de ampliar el formulario; mientras eso no ocurra, la Tabla 16 documenta un espacio de valores mayor que el efectivamente alcanzable.

La opción «Otro» recibía la ponderación mínima; se corrigió. Al traducirse a soporte, todo proyecto atípico obtenía 5 puntos por tipo de servicio, de modo que el sesgo empujaba hacia COLD precisamente las oportunidades que el sistema no supo clasificar. Un lead de USD 1.000 con urgencia baja, sin teléfono ni descripción extensa, quedaba en 30 puntos (COLD), mientras que el mismo lead clasificado como desarrollo_web alcanzaba 45 (WARM). No es un problema de calibración —ningún dato de conversión puede justificar que «no sé cuánto vale» equivalga a «vale poco»— sino de lógica de negocio. La traducción se reemplazó por consultoria (12 puntos), una asignación neutra, y se alineó con ella el valor de respaldo de la normalización (Anexo I). La calibración de esa ponderación con datos de conversión reales sigue pendiente (Capítulo 8).

El control de entrada y los tramos de puntaje no fueron alineados. El deslizante abarca de USD 100 a USD 20.000 y la Tabla 16 asigna 0 puntos por debajo de 300, de modo que el tramo inferior del propio control no puntúa. En el extremo opuesto había un problema mayor, ya corregido (Anexo I): el tope era 5.000, exactamente el valor en que empieza el tramo más alto. Un proyecto de USD 20.000 debía declararse por eso como uno de 5.000, y ambos quedaban indistinguibles en la base, en el tablero y en la propuesta. El sistema descartaba la información justo en la franja que su propio criterio de priorización considera más valiosa. Subir el tope no altera la Tabla 16 —de 5.000 en adelante se suman los mismos 40 puntos y el máximo sigue siendo 100— pero deja de perder el dato. El corte en 300 proviene del criterio experto del §3.3 y su revisión conjunta con el rango del control se plantea en el Capítulo 8.

4.3.2 Propuesta, aceptación y facturación (webhooks de propuesta enviar y lead acepta)

Los términos de la propuesta los fija el profesional. Un diseño alternativo, descartado, consistía en generar y enviar la propuesta automáticamente en cuanto el scoring clasificara al lead como HOT o WARM, tomando el importe de la columna presupuesto: el valor que el propio interesado elige en el control deslizante. Ese importe se habría convertido después en el monto de la factura, del comprobante en PDF y de la preferencia de cobro. El sistema habría comprometido así al profesional con un precio que él no fija en ningún punto del recorrido, en contra de la condición de la pregunta de investigación de automatizar «sin sacrificar el control del profesional sobre el proceso» (§1.3).

El diseño adoptado lo evita mediante el webhook POST /propuesta-enviar, invocado desde el tablero interno con la misma protección que el resto de las acciones internas. El lead calificado permanece en estado NUEVO y aparece en la sección «Propuestas por enviar», donde el profesional carga precio, plazo y alcance. Una actualización condicional sobre el estado NUEVO y el nivel HOT o WARM impide que repetir la petición reabra una propuesta ya enviada; desde ahí el flujo continúa por la cadena que ya existía. El presupuesto declarado se conserva intacto: sigue siendo la entrada del scoring y el registro de lo que el cliente dijo estar dispuesto a invertir, y el tablero lo muestra como referencia, no como valor por omisión.

Enviada la propuesta, el webhook /lead-acepta recibe el identificador del lead y el token, comprueba la presencia de ambos y consulta leads por identificador y por token de aceptación, interpretado como valor UUID. Si el lead es válido se lo actualiza al estado ACEPTADO y se registra la fecha; en caso contrario se responde que el enlace ya fue procesado o es inválido, lo que impide reutilizar el token (OE3a, RNF2). A partir de la aceptación, el flujo genera la página de confirmación que ve el cliente y, en paralelo, produce la factura en HTML, la convierte a PDF mediante Gotenberg (Figura 15) y la envía por correo, persiste la factura en la tabla facturas y actualiza el lead a FACTURADO. El evento se notifica por Telegram y se registra en Notion (Figura 9).

La verificación del estado y la actualización a ACEPTADO se apoyan en una única sentencia de actualización condicional atómica: la transición sólo se aplica si el lead sigue en un estado válido —PROPUESTA_ENVIADA o EN_SEGUIMIENTO—. El flujo ramifica según la cantidad de filas afectadas. La primera solicitud produce la factura y las notificaciones; cualquier solicitud posterior —o verdaderamente concurrente, como dos pestañas abiertas o un doble clic que dispare dos peticiones casi simultáneas— no afecta ninguna fila y se deriva a una rama de operación nula.

El resultado es que, aunque la verificación y la actualización se implementen como dos nodos, la transición a ACEPTADO es atómica e idempotente a nivel de base de datos. En consecuencia, eso descarta la doble facturación tanto en solicitudes secuenciales como concurrentes sobre el mismo enlace.

Frontera transaccional. Esa atomicidad se limita a un único paso. Entre la transición a ACEPTADO y la persistencia de la factura median cuatro operaciones sobre servicios externos —la generación del HTML, la conversión a PDF por Gotenberg, el envío por correo y la inserción en facturas— y ninguna participa de la transacción. Una falla intermedia deja el lead en ACEPTADO sin factura emitida y, por construcción del propio mecanismo de idempotencia, un reintento del cliente se deriva a la rama de operación nula: el resultado es una factura perdida sin recuperación automática. El sistema no implementa saga, compensación, reintento con retroceso exponencial ni cola de mensajes muertos.

La mitigación mínima —un proceso programado que detecte leads en ACEPTADO sin factura asociada y reintente la facturación de forma idempotente— se describe en §4.3.5. Repara después de la falla en lugar de evitarla: la frontera transaccional sigue sin existir, la inconsistencia dura hasta media hora y el comprobante se reemite sin reenviar el correo con el PDF, que queda a cargo del proceso de recordatorios o del profesional, avisado por Telegram. La Figura 2 representa la secuencia completa y delimita el alcance de la transacción; las Figuras 14 y 15 reproducen el correo de la factura y el comprobante en PDF.

Ordenamiento del tramo como saga. El tramo que va de la aceptación a la emisión de la factura ilustra el problema que la saga resuelve: la operación se descompone en pasos que actúan sobre servicios distintos y ninguna transacción los abarca. La Tabla 17 lo ordena como una saga —una secuencia de pasos locales, cada uno con la transacción de compensación que repara lo hecho si un paso posterior falla—, derivada del flujo que este mismo apartado describe y sin introducir comportamiento nuevo. La compensación está condicionada por un hecho: una vez cumplido el paso 1 el token queda consumido y el lead no puede volver a PROPUESTA_ENVIADA, de modo que no puede ser revertir la aceptación sino reparar hacia adelante.

Tabla 17. El tramo aceptación→facturación ordenado como saga, con la compensación de cada paso

Paso	Operación local	Compensación	¿Implementada?
1	Transición del lead a ACEPTADO mediante una actualización condicional atómica que sólo se aplica si el lead sigue en un estado válido.	No admite reversión: cumplido el paso, el token queda consumido y el lead no puede volver a PROPUESTA_ENVIADA. La reparación sólo puede ser hacia adelante.	No aplica
2	Generación del HTML de la factura con los términos que fijó el profesional.	Registrar la falla y reintentar; revertir el paso 1 no es viable.	No
3	Conversión de ese HTML a PDF mediante el servicio Gotenberg.	Registrar la falla y reintentar; revertir el paso 1 no es viable.	No

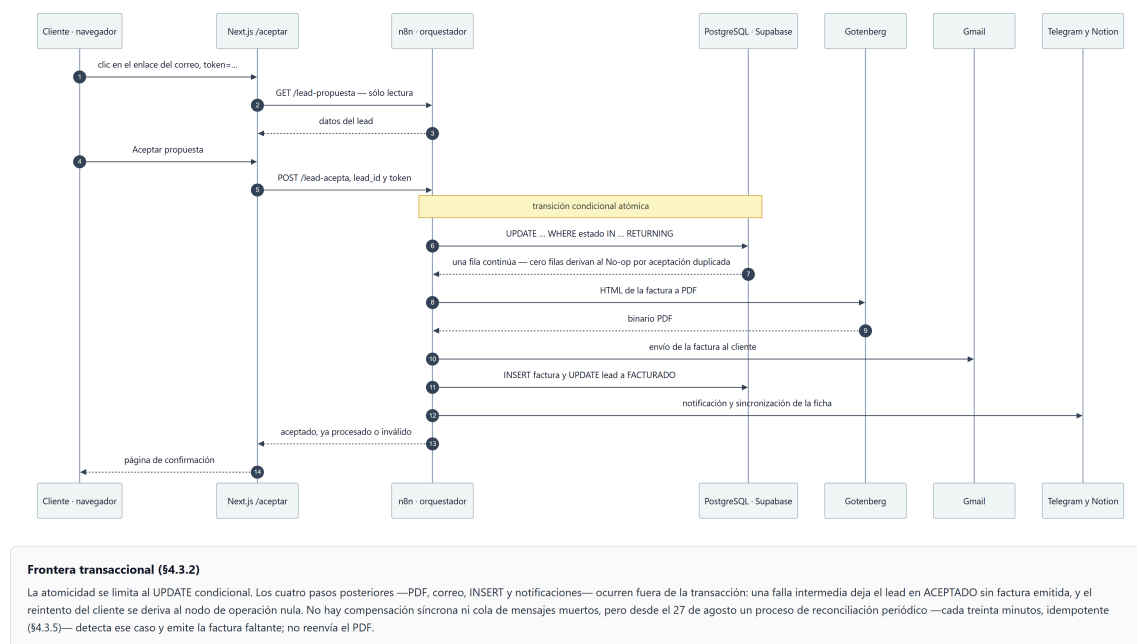
Paso	Operación local	Compensación	¿ Implementada?
4	Envío del PDF al cliente por correo.	Reintentar el envío; el paso no modifica el estado persistido.	No
5	Inserción de la factura en la tabla facturas y transición del lead a FACTURADO.	Reintentar el INSERT de forma idempotente — condicionado a que no exista ya una factura para ese lead— o marcar el lead para revisión manual.	Sí: es la reconciliación programada del §4.3.5
6	Notificación del evento por Telegram y registro en Notion.	Reintentar; su omisión no deja la operación de negocio en un estado inconsistente.	No

Nota. Elaboración propia a partir del flujo exportado workflow/crm_postgres.json. El artefacto implementa una sola de las seis compensaciones.

El artefacto implementa una sola de las seis compensaciones: el proceso programado de reconciliación (§4.3.5) busca leads en ACEPTADO sin factura asociada y reemite el comprobante de forma idempotente, que es exactamente la compensación del paso 5. No hay reintento con retroceso exponencial para los pasos 2 a 4, ni cola de mensajes muertos, ni una unidad de trabajo única que abarque la aceptación y la facturación (§4.9). El trabajo diagnostica el tramo con el vocabulario de la saga pero no la implementa. Lo que existe es una compensación parcial que repara después de la falla y deja una ventana de hasta media hora, y el Capítulo 8 retoma el cierre de esa frontera remitiendo a este diseño en lugar de proponerlo sin desarrollar.

Semántica de los desenlaces. La página pública devuelve un único mensaje, «enlace ya utilizado», para cuatro situaciones de causas distintas, y sólo la primera se corresponde con lo que el mensaje afirma. Es el enlace ya procesado, en el que el token es válido pero la propuesta ya fue aceptada o rechazada: la condición sobre el estado previo no se cumple y el flujo deriva al nodo de operación nula (es el desenlace que ejercita E6). Las otras tres tienen causas distintas. La primera es el enlace vencido, cuyo token superó los TOKEN_VIGENCIA_DIAS aunque nunca se lo haya usado. La segunda es la oportunidad expirada: la condición sobre el estado excluye a un lead marcado PERDIDO por agotamiento de seguimientos, de modo que un clic en su enlace todavía vigente obtiene la misma respuesta. La tercera es el enlace reemplazado, que introdujo la rotación del token en cada reenvío (S3): un cliente que hace clic en un correo de propuesta anterior recibe el mismo mensaje, porque lo que cambió fue el token y no su estado frente a él. A ellas se suma el token inexistente o mal formado, que hoy termina en el mismo lugar. Las cuatro deberían comunicarse por separado, con una acción sugerida distinta. La corrección es de interfaz y no de lógica —el desenlace del flujo es correcto en los cuatro casos: ninguno duplica la factura ni altera el estado—, y por eso se registra como trabajo pendiente y no como defecto funcional.

Figura 2. Diagrama de secuencia del flujo de aceptación y facturación, con los siete participantes y el alcance efectivo de la transacción.



Nota. Elaboración propia a partir del flujo exportado `workflow/crm_postgres.json`.

El token viaja, además, en dos lugares de la cadena de consulta: el enlace de correo que abre la página `/aceptar/[leadId]?token=...` y la petición GET de sólo lectura que esa página emite al cargar para consultar el estado del lead (webhook `/lead-propuesta`). La confirmación en sí se envía por POST con el token en el cuerpo (§4.2.6). Esa exposición implica que el token puede quedar registrado en el historial del navegador y en los registros de acceso del servidor. El riesgo queda acotado por tres vías en lugar de una: el carácter de un solo uso del token una vez aceptado, su vencimiento configurable (`token_expira_en`, `TOKEN_VIGENCIA_DIAS=14` por defecto) y, desde el 31 de agosto de 2026, su rotación en cada reenvío de la propuesta (S3). Un token capturado en un registro de acceso deja de servir, entonces, en cuanto la propuesta se reenvía. No queda eliminado —el token sigue almacenándose en claro en `accept_token`— y evitar que la cadena de consulta completa quede registrada en los logs de producción sigue siendo una línea de código pendiente.

4.3.3 Pago (webhook de confirmación de MercadoPago)

Al aceptar la propuesta se solicita a la API de MercadoPago una preferencia de checkout real (MercadoPago, s. f.-a); el botón «Pagar ahora» de la factura apunta a ese checkout y no a un enlace del propio sistema. Cuando el estado del pago cambia, MercadoPago —no el navegador del cliente— invoca el webhook POST `/webhook/mp/notificacion`. El flujo no confía en el cuerpo de esa notificación: consulta el

pago contra la API del proveedor, que es la fuente de verdad (MercadoPago, s. f.-b), y sólo lo deja pasar si su estado es approved. La factura pasa a COBRADO mediante una actualización condicional sobre el estado PENDIENTE, del mismo tipo que la de la aceptación, de modo que una notificación repetida —MercadoPago reintenta hasta recibir 2xx— no genera un segundo cobro ni una segunda notificación. Si MP_ACCESS_TOKEN no está configurada, el sistema cae a un modo de desarrollo declarado en el que el enlace de la factura es GET /webhook/pago-confirmado (Figura 10).

Modelo de confianza del modo de desarrollo. El endpoint GET /webhook/pago-confirmado —vigente sólo cuando no hay credenciales reales configuradas— conserva un defecto conocido: el enlace viaja en el PDF que recibe el propio cliente deudor. En el modo de producción, con MP_ACCESS_TOKEN configurado, ese modelo de confianza ya no aplica: el estado COBRADO sólo puede originarse en una notificación verificada contra la API de MercadoPago, nunca en una acción del cliente.

El modo de desarrollo hereda además un problema de semántica HTTP: GET es un método seguro e idempotente por definición, y usarlo para mutar estado lo expone al prefetching de navegadores y al escaneo automático de enlaces. Ese defecto queda circunscripto al modo de desarrollo; en producción la mutación ocurre por POST. La validación de firma del webhook de MercadoPago (x-signature, mediante MP_WEBHOOK_SECRET; MercadoPago, s. f.-c) era opcional y estaba deshabilitada por defecto. El endpoint aceptaba así notificaciones sin verificar su origen, aunque el flujo igual las contrastara contra la API; esa deuda se cerró y la firma es obligatoria por defecto (Tabla 22, S8).

4.3.4 Cierre de proyecto (webhook de proyecto cerrado)

El webhook POST /proyecto-cerrado cierra el ciclo: el lead pasa a CERRADO, se registra la fecha y se calcula la duración total del ciclo en días. La factura se concilia mediante una actualización condicional sobre el estado PENDIENTE, de modo que el cierre no reescribe la fecha de cobro de una factura ya cobrada ni da por cobrada una anulada. Sin esa condición, cerrar el proyecto marcaba la factura como COBRADO cualquiera fuese su estado previo, lo que contradecía la regla del §4.8 y podía inflar la tasa de cobro del tablero con dinero que no ingresó. A continuación se notifica el cierre por Telegram y se solicita al cliente un testimonio por correo (Figura 11).

En una versión anterior este webhook no se invocaba desde ninguna ruta del frontend: lo disparaba el profesional con una petición directa. La consecuencia no era sólo

de comodidad. CERRADO es el único estado sobre el que se calculan la tasa de conversión y el tiempo promedio de ciclo (Tabla 19), de modo que sin una acción en la interfaz que lo produjera ambos indicadores permanecían en cero y el reporte diario los informaba así todas las noches. Tampoco se enviaba el correo de testimonio. El tablero ofrece ahora la acción «Cerrar proyecto» sobre los trabajos entregados, por la misma vía autenticada que el resto de las acciones internas (Anexo I).

4.3.5 Procesos programados

Cuatro disparadores por calendario ejecutan tareas desatendidas: los tres primeros responden a OE5 y el cuarto cierra la deuda S5 de la Tabla 22. La Tabla 18 los reúne. Dos de ellos derivan su desenlace a una condición: el de seguimiento decide si el intento en curso es el último y, en ese caso, da el lead por perdido; el de recordatorios decide si la factura está lo bastante vencida como para notificarla también por mensajería. Ambas condiciones operan sobre lotes, porque cada corrida atiende a todos los casos que cumplen su criterio temporal. Esa combinación impone una restricción cuyo incumplimiento costó un defecto (§5.2). El nodo que precede a la condición tiene que devolver una fila por caso, lo que exige que su sentencia de actualización lleve cláusula de retorno. Y la condición tiene que resolverse sobre el ítem que está tratando, y no indexando por posición la salida de un nodo anterior.

Tabla 18. Procesos programados: disparador y comportamiento

Proceso	Disparador	Qué hace
Seguimiento de propuestas	lunes a viernes, 9:00	Selecciona los leads en estado PROPUESTA_ENVIADA o EN_SEGUIMIENTO con menos de tres seguimientos cuya última interacción ocurrió hace al menos $3 \times (\text{seguimientos} + 1)$ días —un espaciado creciente—. Les envía uno de tres mensajes según el número de seguimiento, cada uno con el enlace a la propuesta, y registra el seguimiento. Al alcanzar el tercer seguimiento sin respuesta, el lead se marca como PERDIDO (Figura 12, Anexo A).
Recordatorios de pago	diario, 10:00	Lee las facturas pendientes y clasifica cada una según los días al vencimiento —recordatorio de uno a tres días antes, aviso el día del vencimiento, aviso de vencida hasta tres días y aviso urgente a partir de allí—. Envía el correo correspondiente, con el enlace de pago guardado con la factura; en el caso urgente notifica además por Telegram.
Reporte diario de métricas	diario, 23:59	Envía por Telegram un resumen de los indicadores del mes en curso, encabezado por la cantidad de propuestas calificadas que esperan que el profesional fije sus términos (§4.3.2): es el único dato del reporte sobre el que hay que actuar. La consulta parte de una fila sintética del mes en curso, de modo que el reporte se emite aunque el mes no registre actividad (Anexo I). Su verificación se reporta en el escenario E13; no cuenta con evidencia visual en el Anexo A.
Reconciliación de facturas	cada 30 minutos	Cierra la deuda S5 de la Tabla 22. Busca el lead más antiguo en estado ACEPTADO sin factura asociada cuya aceptación date de hace más de RECONCILIACION_GRACIA_MINUTOS —quince por defecto, para no pisar una aceptación en curso— y emite el comprobante que la cadena de facturación no llegó a persistir, de a un lead por corrida. La emisión es idempotente por dos vías independientes: la inserción está condicionada a que no exista ya una factura para ese lead, y el identificador de la factura se deriva del propio lead. La factura recuperada nace PENDIENTE, lleva el precio que el profesional comprometió y el mismo enlace de pago que habría tenido de emitirse a tiempo, y entra al proceso de recordatorios. El proceso no depende del correo: persiste, marca el lead como FACTURADO y avisa al profesional por Telegram.

4.3.6 Manejo de errores

El flujo incorpora una rama de manejo de errores que formatea la falla, la registra en la base de datos y emite una notificación de error crítico por Telegram, de modo que las anomalías no pasen inadvertidas (RNF3a).

4.3.7 Gestión interactiva del ciclo posterior a la propuesta

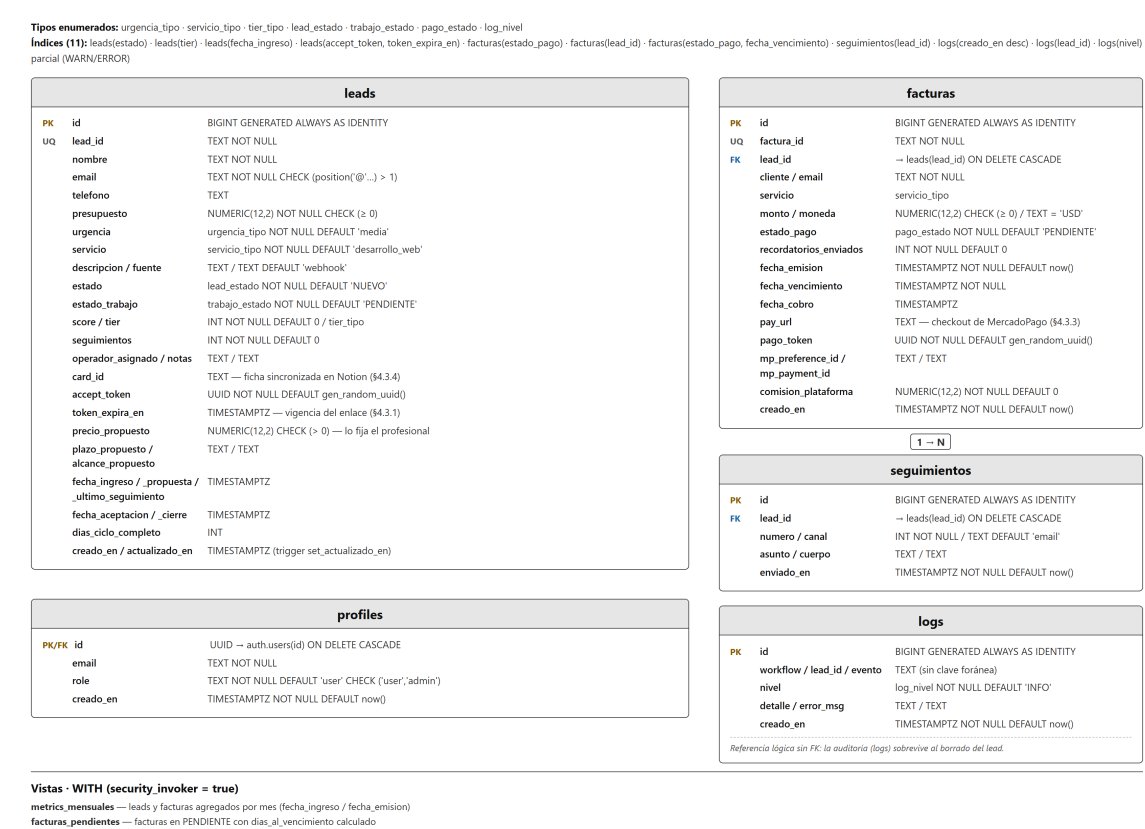
Más allá del camino principal, el flujo gestiona de forma interactiva lo que ocurre después de enviar la propuesta. Desde la página pública de aceptación el cliente puede aceptar, solicitar cambios o rechazar: el rechazo marca el lead como PERDIDO y el pedido de cambios lo devuelve a EN_SEGUIMIENTO, guarda el mensaje del cliente y notifica al profesional. Una vez aceptado el proyecto, su ejecución se sigue mediante una máquina de estados propia —la columna estado_trabajo, con los valores PENDIENTE, EN_PROGRESO, EN_REVISION y ENTREGADO—, que se actualiza desde el tablero interno y se sincroniza con Notion. Desde ese mismo tablero el equipo puede cancelar un lead y resolver los pedidos de cambio: al aceptarlos se reenvía la propuesta y al rechazarlos se mantiene la original. Los webhooks de cada acción figuran en la Tabla 21 y en el Anexo B.

4.4 Modelo de datos

El modelo relacional se compone de cinco tablas y dos vistas: cuatro tablas de negocio —leads, facturas, seguimientos y logs— y profiles, que asigna a cada usuario un rol de aplicación (§4.6). La Figura 3 presenta el diagrama entidad-relación completo, con tipos, claves, restricciones y valores por defecto tomados de db/schema.sql. El esquema declara siete tipos enumerados, integridad referencial con borrado en cascada, once índices y un disparador que mantiene la marca temporal de modificación, y se reproduce una sola vez, en el Anexo F. De él conviene retener tres decisiones de modelado que el resto del documento invoca. La tabla leads distingue dos estados con enumerados propios: estado, que sigue el ciclo comercial, y estado_trabajo, que sigue la ejecución del proyecto (Tabla 20). Separa además dos importes: presupuesto, declarado por el interesado y única entrada monetaria del scoring, y precio_propuesto, que fija el profesional al enviar la propuesta y que es el importe efectivamente facturado (§4.3.2). El control de la aceptación descansa en accept_token —UUID almacenado en claro— y token_expira_en, con el índice compuesto que sostiene su revalidación. En facturas, factura_id sigue el formato FAC-<año>-<aleatorio de cuatro

dígitos>, no correlativo, lo que el §2.7 retoma al discutir la validez fiscal, y el enumerado pago_estado prevé los valores VENCIDA y ANULADA que ningún nodo asigna (regla R7). Y profiles asigna el rol cuya verificación exige el §4.6, mientras que logs concentra el registro de auditoría que sostiene la trazabilidad de E15.

Figura 3. Diagrama entidad-relación del modelo de datos, con tipos, claves, restricciones y valores por defecto.



Nota. Elaboración propia a partir de db/schema.sql. El DDL completo se reproduce en el Anexo F.

Las vistas metrics_mensuales y facturas_pendientes computan, respectivamente, los indicadores mensuales que alimentan el tablero y el listado de facturas en estado PENDIENTE con sus días al vencimiento. Ambas se declaran con la opción security_invoker, de modo que respetan las políticas de las tablas subyacentes (§4.6). La Tabla 19 define operacionalmente cada indicador; la Tabla 20 enumera los estados por los que transita un lead.

Tabla 19. Definición operacional de los indicadores del tablero

Indicador (rótulo en el tablero)	Definición operacional	Alcance temporal
Leads	Cantidad de filas de Leads cuya fecha_ingreso pertenece al mes considerado.	Mes en curso
Conversión	$100 \times (\text{leads en estado CERRADO del mes}) \div (\text{total de leads del mes})$. Se redondea a un decimal y devuelve 0 si el denominador es nulo.	Mes en curso
Tasa de cobro	$100 \times (\text{suma de monto de las facturas emitidas en el mes con estado_pago = 'COBRADO'}) \div (\text{suma de monto de todas las facturas emitidas en el mes})$.	Mes en curso
Facturación	Suma de monto de las facturas cuya fecha_emision pertenece al mes.	Mes en curso
Cobrado / Pendiente	Suma de monto de las facturas del mes con estado_pago = 'COBRADO' y con estado distinto de COBRADO, respectivamente.	Mes en curso

Indicador (rótulo en el tablero)	Definición operacional	Alcance temporal
Facturas vencidas	Cantidad de facturas con estado_pago = 'PENDIENTE' y fecha_vencimiento < now().	Mes en curso
Tiempo promedio de ciclo	Promedio de días_ciclo_completo sobre los leads del mes en estado CERRADO.	Mes en curso
Embudo de leads (por estado)	Cantidad de filas de leads agrupadas por la columna estado. No se filtra por mes : el embudo se calcula sobre la totalidad histórica de los registros.	Histórico acumulado
Propuestas por enviar	Cantidad de filas de leads en estado NUEVO cuyo tier es HOT o WARM y que todavía no tienen propuesta enviada: son las que esperan que el profesional fije el precio, el plazo y el alcance (§4.3.2). No se filtra por mes : encabeza el tablero y el reporte diario de métricas (§4.3.5), y se cuenta sobre la totalidad histórica de los registros.	Histórico acumulado

Nota. Elaboración propia a partir de la vista metrics_mensuales de db/schema.sql y del componente dashboard-client.tsx. La diferencia de alcance temporal entre el indicador «Leads» (mes en curso) y el embudo por estado (histórico acumulado) explica que ambos valores no coincidan en la Figura 6 —el indicador declara 9 y la suma del embudo, 23—: no es una inconsistencia de datos sino dos criterios de agregación distintos, y por la misma razón «Propuestas por enviar» tampoco es comparable con «Leads». Dos indicadores tienen, además, un desajuste de cohorte: «Conversión» y «Tiempo promedio de ciclo» se computan sobre el mes calendario, de modo que un lead ingresado sobre el final del mes no alcanza a cerrarse dentro de él y la conversión mensual subestima de forma sistemática. Calcularlos por cohorte de ingreso corrige ambos sesgos y se registra en el punto 7 del Capítulo 8.

La columna estado_pago de facturas declara cuatro valores: PENDIENTE (factura emitida, aún sin cobrar), COBRADO (pago registrado), VENCIDA (se superó la fecha de vencimiento sin cobro) y ANULADA (factura anulada o cancelada). Sólo los dos primeros se asignan efectivamente: los otros dos están previstos en el enumerado y ningún componente los escribe, según se declara en §4.8. Las otras dos máquinas de estados del sistema —el ciclo comercial del lead y el ciclo de ejecución del trabajo— se representan en la Figura 4.

Figura 4. Máquinas de estados del sistema: ciclo comercial del lead (A) y ciclo de ejecución del trabajo (B).



Nota. Elaboración propia a partir de db/schema.sql y del flujo exportado.

Tabla 20. Estados del ciclo de vida del lead y del trabajo

Máquina	Estado	Significado	Transición de entrada
Comercial (leads.estado)	NUEVO	Lead ingresado y normalizado. Tras el scoring conserva este estado: si es HOT o WARM, esperando que el profesional fije los términos de la propuesta; si es COLD, descartado para el envío automático. La columna tier distingue ambos casos.	Alta por POST /lead/nuevo.
	PROPUESTA_ENVIADA	Se generó y envió la propuesta al cliente.	POST /propuesta-enviar desde el tablero interno, sobre un lead HOT o WARM.

Máquina	Estado	Significado	Transición de entrada
	EN_SEGUIMIENTO	Se realizó al menos un seguimiento sin respuesta, o el cliente pidió cambios.	Proceso programado de seguimiento; POST /lead-modifica.
	ACEPTADO	El cliente aceptó la propuesta mediante el enlace con token.	POST /lead-acepta con actualización condicional atómica.
	FACTURADO	Se generó, envió y persistió la factura.	Continuación del flujo de aceptación.
	CERRADO	El proyecto se cerró y se registró la duración del ciclo.	POST /proyecto-cerrado.
	PERDIDO	Oportunidad descartada. Agrupa tres causas distintas: rechazo del cliente, agotamiento de los tres seguimientos sin respuesta y cancelación desde el tablero interno .	POST /lead-rechaza; tercer seguimiento sin respuesta; POST /lead-cancelar.
Ejecución (leads.estado_trabajo)	PENDIENTE	Trabajo aceptado y aún no iniciado.	Valor por omisión de la columna.
	EN_PROGRESO	Trabajo en ejecución.	POST /trabajo-estado desde el tablero interno.
	EN_REVISION	Entregable en revisión del cliente.	POST /trabajo-estado.
	ENTREGADO	Trabajo entregado y aceptado.	POST /trabajo-estado.

Nota. Elaboración propia a partir de los tipos enumerados `lead_estado` y `trabajo_estado` de `db/schema.sql`. Sobre la cancelación: el webhook `/lead-cancelar` no introduce un estado propio, sino que ejecuta `UPDATE leads SET estado = 'PERDIDO' ... WHERE estado NOT IN ('CERRADO','PERDIDO')`, de modo que la distinción entre «perdido» y «cancelado» sólo existe en el tablero externo de Notion y no en la base; se documenta como limitación del modelo de estados. Invariante entre ambas máquinas: `estado_trabajo` sólo avanza más allá de `PENDIENTE` cuando `estado` es `ACEPTADO`, `FACTURADO` o `CERRADO`; el esquema no lo impone mediante una restricción y depende de la lógica del flujo.

4.5 Contratos de integración

La comunicación entre el frontend y n8n se realiza, en su mayor parte, mediante los webhooks de la Tabla 21. Once se originan en una acción del usuario en el frontend o en la página pública de aceptación, y los dos restantes —`mp/notificacion` y `pago-confirmado`— en el servidor de MercadoPago y en el PDF de la factura. El frontend envía el lead como un objeto JSON con los campos del formulario más metadatos de origen y marca temporal, y consume la respuesta de la aceptación para informar el desenlace al cliente; el contrato completo se reproduce en el Anexo D. La tabla incorpora, junto al propósito de cada webhook, el origen real de cada invocación y el control de origen aplicado, columna que hace visible la principal deuda de seguridad del sistema, analizada en §4.6.

Se observa además una inconsistencia de nomenclatura que conviene declarar: de los trece webhooks, dos emplean barra como separador —`/lead/nuevo` y `/mp/notificacion`— y los once restantes emplean guion. La ruta de captación diverge por motivos históricos, por haber sido la primera en construirse; la de MercadoPago, porque su formato lo impone el proveedor. Ninguna tiene efecto funcional, pero rompen la uniformidad del contrato de integración; unificar el criterio de las rutas propias exige actualizar la variable de entorno del frontend y el nodo correspondiente del flujo, y se registra en el punto 7 del Capítulo 8.

Tabla 21. Contratos de integración (webhooks de n8n)

Webhook	Método	Origen de la invocación	Control de origen y autenticación	Propósito
<code>/lead/nuevo</code>	POST	Formulario público (Lead-form.tsx)	Lista blanca de orígenes (CORS). Sin autenticación . Sin clave de idempotencia.	Ingreso de un nuevo lead.

Webhook	Método	Origen de la invocación	Control de origen y autenticación	Propósito
/lead-propuesta	GET	Página /aceptar/[leadId] al cargar	Token en la cadena de consulta. Solo lectura.	Lectura de la propuesta para la página de aceptación.
/lead-acepta	POST	Página /aceptar/[leadId]	Token de un solo uso en el cuerpo, validado contra <code>accept_token</code> .	Aceptación de la propuesta y disparo de la facturación.
/lead-rechaza	POST	Página /aceptar/[leadId]	Token de un solo uso.	Rechazo de la propuesta (el lead pasa a PERDIDO).
/lead-modifica	POST	Página /aceptar/[leadId]	Token de un solo uso.	Pedido de cambios (vuelve a EN_SEGUIMIENTO y guarda el mensaje).
/pago-confirmado	GET	Botón «Pagar ahora» del PDF de la factura, en poder del cliente deudor	Token por factura: pago_token, mismo mecanismo que accept_token, generado al emitir la factura y exigido junto al identificador en la cadena de consulta (§4.3.3, S1 de la Tabla 22, cerrada de forma parcial el 31-ago-2026).	Confirmación de pago — modo de desarrollo (sin MercadoPago).
/mp/notificacion	POST	Notificación de MercadoPago (servidor a servidor), cuando cambia el estado de un pago.	Firma obligatoria (S8). Valida HMAC sobre x-signature con MP_WEBHOOK_SECRET; sin firma válida la notificación se ignora sin aplicar efecto alguno, y el pago igual se reconfirma contra la API de MercadoPago antes de marcar la factura COBRADO.	Confirmación de pago real, verificada contra la API de MercadoPago antes de marcar la factura COBRADO.
/proyecto-cerrado	POST	Tablero interno, vía /api/crm/[accion], sobre los trabajos en estado ENTREGADO (§4.3.4)	Header Auth verificado por el webhook; el route handler revalida sesión y rol admin del lado del servidor antes de reenviar.	Cierre del proyecto y solicitud de testimonio.
/trabajo-estado	POST	Tablero interno, vía el route handler /api/crm/[accion].	Header Auth, ídem fila anterior.	Actualización de estado_trabajo, con sincronización a Notion.
/lead-cancelar	POST	Tablero interno, vía el route handler /api/crm/[accion].	Header Auth, ídem fila anterior.	Cancelación del lead (lo marca como PERDIDO).
/cambio-aceptar	POST	Tablero interno, vía el route handler /api/crm/[accion].	Header Auth, ídem fila anterior.	Aceptación de un pedido de cambios (reenvía la propuesta).
/cambio-rechazar	POST	Tablero interno, vía el route handler /api/crm/[accion].	Header Auth, ídem fila anterior.	Rechazo de un pedido de cambios (mantiene la propuesta original).
/propuesta-enviar	POST	Tablero interno, vía /api/crm/[accion], sobre un lead HOT o WARM en estado NUEVO (§4.3.2)	Header Auth, ídem fila anterior.	Fija precio, plazo y alcance, y dispara la generación y el envío de la propuesta.

Nota. Elaboración propia a partir de los nodos de tipo Webhook del flujo exportado y de las llamadas fetch del frontend. La lista blanca de orígenes es un control del navegador (CORS) y no una autenticación: no impide la invocación desde un cliente que no sea un navegador. Sobre la inconsistencia de nomenclatura entre /lead/nuevo y /mp/notificacion (con barra) y el resto de las rutas (con guion), véase §4.5.

4.6 Seguridad

El acceso al tablero requiere autenticación con Supabase Auth y, además, rol de aplicación admin. Sobre las cinco tablas se habilita la seguridad a nivel de fila, y las políticas de lectura de leads, facturas y seguimientos no se conforman con que el usuario esté autenticado: exigen que exista una fila en perfiles con `role = 'admin'` para el identificador de la sesión. La tabla perfiles admite que cada usuario lea únicamente su propia fila, y logs no tiene ninguna política para authenticated, de modo que el registro de auditoría no es accesible desde el tablero. Las vistas se declaran con `security_invoker` para que respeten las políticas de las tablas subyacentes, y se revocan los privilegios del rol público. La escritura la realiza n8n con un rol de Postgres propio, `n8n_writer`, sujeto a políticas de fila que autorizan sólo las operaciones que el flujo ejecuta y sin el atributo `BYPASSRLS` que sí tiene `service_role`. La aceptación de propuestas se protege con el token de un solo uso, sin exigir cuenta. Este conjunto materializa los RNF1 y RNF2; el detalle de las políticas está en el Anexo C.

De esa descripción se sigue una consecuencia que, hasta el 31 de agosto de 2026, correspondía explicitar sin atenuantes: la seguridad a nivel de fila protegía la lectura, no la

integridad, porque toda inserción o actualización se realizaba con `service_role`, que evade la RLS por diseño (S4). Ese día se agregó `n8n_writer`, un rol sin `BYPASSRLS`, con `GRANT` de `SELECT`, `INSERT` y `UPDATE` —nunca `DELETE`— sobre las cuatro tablas que el flujo escribe (`leads`, `facturas`, `seguimientos` y `logs`; nunca `profiles`), y con políticas de fila propias. Si esa credencial se filtra —el §6.3 documenta que ya ocurrió una vez, con `service_role`— el radio de daño queda acotado a esas cuatro tablas y a esas tres operaciones, y no a la base entera con privilegios de superusuario de facto. La mitigación está implementada y verificada con evidencia ejecutable (`tests/rls/casos.sql`, `tests/idempotencia.mjs`), y ese mismo día se aplicó al proyecto de Supabase real, con la conexión de `n8n` usando las credenciales del nuevo rol y verificada con un alta de lead contra el sistema levantado (S4, cerrada).

Deudas de seguridad declaradas. El apartado quedaría incompleto si describiera los mecanismos implementados y omitiera los que faltan; la Tabla 22 los enumera. La deuda más relevante es que cinco de los trece webhooks no verifican la identidad de quien los invoca: no hay firma HMAC, ni secreto compartido, ni lista blanca de direcciones, ni mTLS. Un sexto —pago-confirmado— tenía el mismo problema y se cerró el 31 de agosto de 2026 con un token por factura (S1, parcial). Los seis invocados desde el tablero interno exigen `Header Auth` con revalidación de sesión y rol, y el de MercadoPago contrasta cada notificación contra la API del proveedor y, desde esa misma fecha, exige además una firma válida (S8, cerrada). La lista de orígenes permitidos que declaran algunos webhooks es un control del navegador —CORS— y no una autenticación: no impide que un cliente que no sea un navegador invoque el endpoint.

En consecuencia, cualquiera que conozca las URL sin protección puede inyectar leads, leer una propuesta o aceptarla en nombre del cliente. Que un producto mínimo viable tenga deudas de seguridad es admisible; no declararlas no lo es. Tampoco alcanza con declararlas: el escenario E16 las mide, ejecutado por primera vez el 30 de agosto de 2026 y reejecutado el 31 tras cerrar S1 y S8, todas las veces contra el sistema levantado (`docs/evidencia-E15-E16.md`).

Conviene fijar qué cuenta ese escenario, porque de la unidad de medida depende cómo leer su resultado. E16 mide una sola cosa: si el punto de entrada acepta y procesa una petición que no presenta credencial de origen, es decir si no la rechaza en el borde con un 403. No mide si el efecto de negocio termina aplicándose: un webhook puede atender la petición y rechazar después la operación por faltarle el token que la autoriza, que es el caso de pago-confirmado desde que S1 se cerró de forma parcial. Con esa unidad de medida, e invocados los trece webhooks desde un cliente que no es un navegador y sin credencial, seis

atienden la petición, seis responden 403 y el restante —la pasarela de MercadoPago— ignora sin aplicar cambios la notificación sin firma válida. De esos seis, cinco no admiten credencial de origen alguna por diseño y el sexto ya no aplica la confirmación del pago sin su token por factura. El alta de un lead prospera y se persiste por la vía de los cinco primeros, lo que comprueba sobre el propio artefacto que la lista blanca de orígenes no es una autenticación.

Ese resultado exige una precisión sobre la naturaleza de los cinco, porque «sin autenticación de origen» no equivale a «sin ningún control de acceso» y leerlo así sobreestima la exposición. Sólo uno —lead/nuevo, el formulario público— es alcanzable conociendo únicamente su URL, y es un punto de entrada abierto por definición, porque debe atender a interesados que todavía no son nadie para el sistema. Los otros cuatro exigen un accept_token válido: un UUID versión 4, de 122 bits de entropía, que no se publica en ninguna parte, que sólo viaja en el correo dirigido al interesado, que vence a los TOKEN_VIGENCIA_DIAS configurados y que rota en cada reenvío (S3). Conocer su URL no alcanza para producir efecto alguno: sin el token, la consulta no encuentra fila y el flujo termina en el nodo de operación nula. Lo que falta en los cuatro es autenticación del invocante, y de ahí que la deuda se declare como tal; pero la superficie realmente abierta a cualquiera que conozca la URL se reduce al alta de leads, que es también la que E16 comprueba que prospera.

Tabla 22. Deudas de seguridad identificadas, con su estado de cierre

#	Deuda de seguridad	Consecuencia	Mitigación propuesta	Estado
S1	Cinco de los trece webhooks no autentican el origen de la invocación (§4.5): lead/nuevo, lead-acepta, lead-propuesta, lead-rechaza y lead-modifica. Los seis invocados desde el tablero interno exigen Header Auth con revalidación de sesión y rol admin en /api/crm/[accion]; el de MercadoPago se trata aparte en S8.	Cualquiera que conozca una de esas URL puede inyectar leads o aceptar una propuesta sin autorización. Mientras pago-confirmado estuvo abierto, marcar un pago era posible adivinando el factura_id (10.000 combinaciones por año).	Implementada el 31-ago-2026 para pago-confirmado: un token por factura (pago_token), exigido en la consulta de confirmación. Para los otros cinco un secreto compartido no es viable —los ejecuta el navegador de un tercero y quedaría expuesto—: les corresponde limitar el abuso con rate limiting o captcha (Capítulo 8).	Parcial — cerrada para pago-confirmado; los otros cinco quedan abiertos por diseño.
S2	El webhook de pago es un GET cuyo enlace viaja en el PDF que recibe el deudor (§4.3.3).	Mientras no hubo control, el propio cliente —o cualquier reenvío o rastreador de enlaces— podía marcar una factura como COBRADO sin haber pagado. En el modo de desarrollo sigue siendo el comportamiento esperado.	Implementada el 23-ago-2026: el nodo Code - Validar Pago rechaza la petición si MP_ACCESS_TOKEN está configurado, de modo que con MercadoPago activo el estado COBRADO sólo puede originarse en la notificación del proveedor.	Parcial — cerrada para el cobro real; el modo de desarrollo sigue alcanzable por diseño.
S3	El token de aceptación tiene vencimiento (token_expira_en, revalidado en las cuatro consultas que lo aceptan; TOKEN_VIGENCIA_DIAS = 14 por defecto) y rota en cada reenvío de la propuesta, pero sigue almacenándose en claro y viajando en la cadena de consulta (§4.3.2).	Posible registro del token en historiales y en bitácoras mientras esté vigente. La ventana queda acotada por el vencimiento y se acorta en cada reenvío: un token capturado deja de servir en cuanto rota, no sólo cuando vence.	Implementada el 31-ago-2026: rotación en cada reenvío, en lugar del resumen criptográfico propuesto originalmente — inviable porque los recordatorios releen el token en claro días después para reconstruir el enlace—. Sigue pendiente no registrar la cadena de consulta en los logs.	Parcial — cerrada la reutilización del valor en claro; sigue abierto el registro de la cadena de consulta.
S4	Toda la escritura se ampara en una clave de servicio que evade la RLS.	Mientras estuvo abierta, la RLS no protegía la integridad de los datos y la protección de escritura dependía por completo de la custodia de la clave de service_role.	Implementada el 31-ago-2026: n8n_writer, un rol sin BYPASSRLS con GRANT de SELECT/INSERT/UPDATE —nunca DELETE ni profiles— sobre las cuatro tablas que toca el flujo, con políticas propias. La cobertura se agregó primero: nueve casos en tests/rls/casos.sql y las cuatro consultas de S5 y S6 con SET ROLE en tests/idempotencia.mjs.	Cerrada — 31-ago-2026, verificada contra el proyecto real: un alta de lead disparada contra el webhook se persistió sin error. Con 33 casos de RLS y 22 aserciones de idempotencia bajo el rol acotado,

#	Deuda de seguridad	Consecuencia	Mitigación propuesta	Estado
				n8n_writer reemplaza a service_role en la escritura.
S5	Ausencia de frontera transaccional entre la aceptación y la facturación (§4.3.2).	Una falla intermedia deja el lead en ACEPTADO sin factura emitida. Mientras no hubo reparación automática, la factura se perdía salvo que alguien advirtiera el hueco y la rehiciera a mano.	Implementada el 27-ago-2026: un cuarto proceso programado, cada 30 minutos, busca leads en ACEPTADO sin factura más antiguos que RECONCILIACION_GRACIA_MINUTOS y emite el comprobante faltante. Es idempotente por partida doble —WHERE NOT EXISTS sobre lead_id y el identificador derivado del lead—, de modo que dos corridas no producen dos facturas.	Parcial — 27-ago-2026: la mitigación está implementada y verificada, pero la frontera transaccional sigue sin existir. Es un proceso compensatorio que repara después de la falla, con una ventana de hasta media hora, y no reenvía el PDF.
S6	Ausencia de idempotencia en la captación (§4.3.1).	Mientras estuvo abierta, un doble envío generaba dos leads con identificadores distintos, dos cálculos de score y dos avisos al profesional por el mismo interesado.	Implementada el 27-ago-2026 en las dos puntas: en el formulario, un cerrojo por referencia descarta el segundo envío mientras el primero está en vuelo; en el flujo, la inserción pasa a ser condicional y no inserta si en los últimos DEDUP_VENTANA_MINUTOS ya entró un lead con el mismo correo. Como el webhook responde al recibir, quien hizo doble clic ve éxito.	Cerrada — 27-ago-2026: verificada con dos peticiones simultáneas contra el sistema levantado —un solo lead y un solo aviso— y con tests/idempotencia.mjs, que ejecuta la consulta del propio nodo sobre el esquema real.
S7	Credenciales sensibles del proyecto de Supabase compartidas por un canal no seguro durante la configuración inicial.	Mientras estuvo abierto: la confidencialidad y la integridad de los datos no estaban garantizadas. Riesgo cerrado con la rotación de las claves.	Rotación de la secret key y de la service_role key. Véase el §6.3.	Cerrado — 27-ago-2026: rotación de la service_role key y deshabilitación de las claves JWT legacy.
S8	La validación de firma del webhook de MercadoPago (x-signature, MP_WEBHOOK_SECRET) era opcional y estaba deshabilitada por defecto (§4.3.3).	Mientras estuvo abierta, el endpoint aceptaba notificaciones sin verificar que provinieran de MercadoPago. El nodo Obtener Pago MP mitigaba el impacto al contrastar el pago contra la API antes de aplicar cambios, pero la superficie quedaba abierta a notificaciones falsas.	Implementada el 31-ago-2026: el nodo Code - Leer Notificación MP pasó a rechazar por defecto toda notificación sin firma válida, y MP_WEBHOOK_SECRET pasó a tener valor obligatorio también en desarrollo.	Cerrada
S9	El motor colapsa a un único ítem la salida de una actualización sin retorno, de modo que la condición se evaluaba siempre sobre el primer caso del lote (§4.3.5, §5.2).	Con más de un caso por lote, la transición a PERDIDO del tercer seguimiento era inalcanzable y la rama de avisos urgentes notificaba una sola factura de las varias que correspondían. No compromete la confidencialidad ni la integridad: es una pérdida de completitud del proceso.	Implementada el 2-sep-2026: se agregó RETURNING a las tres sentencias que alimentan una rama y se reemplazó el indexado por posición por la resolución por ítem emparejado en los cinco nodos que lo usaban.	Cerrada — 2-sep-2026, commit c09008d, con posterioridad al corte documental (§5.6): reejecución de E11 y E12 con trece comprobaciones en verde, y conteos de ítems por nodo del registro de ejecución que confirman la propagación restituida.

Nota. Elaboración propia. De las nueve deudas, cinco quedaron cerradas (S4, S6, S7, S8 y S9) y cuatro, cerradas de forma parcial (S1, S2, S3 y S5); el estado y la fecha de cada una constan en la última columna. La única deuda sin mitigación al cierre del trabajo es el remanente de S1: los cinco webhooks que quedan sin autenticación de origen por diseño.

4.7 Decisiones de diseño

Se destacan tres decisiones. Primera, el patrón de tres clientes de Supabase, que separa las responsabilidades de navegador, servidor y capa de proxy —esta última también en el entorno de Node.js, no en el borde (§4.2.3)—. Esto está en línea con el modelo de renderizado híbrido de Next.js. Segunda, la degradación defensiva ante la ausencia de

variables de entorno, que privilegia la resiliencia del despliegue. Tercera, la elección de tiempo real por suscripción frente al sondeo periódico para el tablero, que reduce la latencia de actualización y el consumo innecesario de recursos.

4.8 Reglas de negocio

El sistema aplica, de forma consistente, las reglas de negocio que enumera la Tabla 23.

Tabla 23. Reglas de negocio que aplica el sistema

Nº	Regla de negocio
R1	Solo se procesan los leads que superan la validación mínima: correo con arroba, nombre de al menos dos caracteres y presupuesto mayor que cero.
R2	Todo lead se inserta ya calificado: el cálculo del puntaje precede a la persistencia, no al revés. Permanece en estado NUEVO hasta que el profesional envía la propuesta (§4.3.1).
R3	Solo los leads clasificados como HOT o WARM reciben propuesta; los COLD no la reciben y dan lugar a dos avisos en paralelo: una notificación interna por Telegram y un acuse de recibo por correo al interesado (§4.3.1).
R4	La aceptación es idempotente respecto del token: la actualización condicional atómica descrita en §4.3.2 impide que una segunda solicitud —secuencial o concurrente— produzca una segunda factura.
R5	Los seguimientos se espacian de forma creciente — $3 \times (\text{seguimientos} + 1)$ días— y se limitan a tres; agotado el tercero sin respuesta, el lead pasa a PERDIDO.
R6	Una factura solo puede pasar a COBRADO desde el estado PENDIENTE.
R7	Los valores VENCIDA y ANULADA del enumerado pago_estado están previstos en el modelo de datos (§4.4 y Anexo F) pero ningún componente los asigna: una factura impaga permanece en PENDIENTE aunque supere su fecha de vencimiento. El vencimiento se comunica por la vía de los recordatorios (§4.3.5), que lo registran como nivel de auditoría y no como estado de la factura, y el tablero lo computa contando las facturas PENDIENTE con fecha de vencimiento pasada (Tabla 19). Se declara aquí para no presentar como regla vigente una transición que el artefacto no ejecuta; su implementación se registra en el Capítulo 8.
R8	La cancelación de un lead desde el tablero interno lo lleva a PERDIDO y solo se aplica si el lead no está ya en CERRADO o PERDIDO (véase la nota de la Tabla 20).

Nota. Elaboración propia a partir del flujo exportado y del esquema de datos. Dos de estas reglas las impone el flujo y no el esquema, y corresponde declararlo con el mismo criterio con que la nota de la Tabla 20 declara el invariante entre las dos máquinas de estados. La validación de R1 exige un presupuesto mayor que cero, mientras la columna admite el valor cero, porque declara CHECK (presupuesto ≥ 0); y el valor de respaldo que el §4.3.1 alineó en consultoría rige en el nodo de normalización, mientras la columna conserva DEFAULT 'desarrollo_web', que es la ponderación máxima de la Tabla 16. Ninguna de las dos diferencias se manifiesta cuando el alta llega por el formulario, porque el flujo normaliza y valida antes de persistir; ambas quedarían expuestas ante una inserción que no pasara por él, caso posible porque la captación no verifica el origen (§4.6). Su alineación en el esquema se registra en el punto 7 del Capítulo 8.

4.9 Síntesis de los riesgos de diseño

El desarrollo de este capítulo identificó diez riesgos de diseño, que conviene reunir para que la discusión (Capítulo 6) y los trabajos futuros (Capítulo 8) los retomen sin dispersión. Ese conjunto no coincide con las nueve deudas de seguridad S1–S9 ni en cantidad ni en composición, y la Tabla 24 establece la correspondencia y su estado de cierre. Cinco de los diez riesgos son de seguridad y cuatro son de consistencia funcional; de estos últimos, S5, S6 y S9 figuran también en aquella tabla. S7, que sí la integra, es un incidente puntual y no un riesgo de diseño. El detalle de cada mitigación vive en la Tabla 22 para los que son deudas de seguridad, y en §4.3.1, §4.3.2 y §4.3.5 para los de consistencia funcional.

Tabla 24. Correspondencia entre los riesgos de diseño y las deudas de seguridad S1–S9

Riesgo de diseño	¿En la Tabla 22?	Naturaleza	Estado y fecha de cierre
Modo de desarrollo del webhook de pago alcanzable con credenciales de MercadoPago reales	Sí (S2)	Seguridad	Cerrado — 23-ago-2026
Credenciales de Supabase compartidas por un canal no seguro	Sí (S7)	Incidente puntual, no un riesgo de diseño	Cerrado — 27-ago-2026
Ausencia de idempotencia en la captación	Sí (S6)	Consistencia funcional	Cerrado — 27-ago-2026
Ausencia de frontera transaccional entre la aceptación y la facturación	Sí (S5)	Consistencia funcional	Mitigado — 27-ago-2026; la frontera transaccional propiamente dicha sigue sin existir
Firma del webhook de MercadoPago opcional y deshabilitada por defecto	Sí (S8)	Seguridad	Cerrado — 31-ago-2026
Webhooks que no autentican el origen de la invocación	Sí (S1)	Seguridad	Parcial — 31-ago-2026 para pago-confirmado (token por factura); los otros cinco quedan abiertos por diseño
Token de aceptación almacenado en claro y transportado en la cadena de consulta	Sí (S3)	Seguridad	Parcial — 31-ago-2026 (rotación en cada reenvío); queda evitar registrar la cadena de consulta completa en los logs
Escritura amparada en una clave de servicio que evade la seguridad a nivel de fila	Sí (S4)	Seguridad	Cerrado — 31-ago-2026 (rol acotado n8n_writer)
Nodos de los procesos programados que actualizan sin cláusula de retorno y resuelven su condición indexando por posición	Sí (S9)	Consistencia funcional	Cerrado — 2-sep-2026, con posterioridad al corte documental (§5.6)
Ausencia de un proceso desatendido que reclame al lead calificado sin propuesta enviada	No	Consistencia funcional	Abierto; hoy sólo se mitiga con la notificación por Telegram y la visibilidad en el tablero (Capítulo 8)

Nota. Elaboración propia; las filas se ordenan por fecha de cierre. «Seguridad» indica además que el riesgo figura en la Tabla 22. S5, S6 y S9 se clasifican aquí como de consistencia funcional, aunque la Tabla 22 las trate junto con las deudas de seguridad: la consecuencia declarada de las tres es una operación que queda incompleta —una factura que no se emite, un lead que se duplica, un lote que se procesa sólo en su primer caso— y no un acceso indebido.

Ninguno de estos riesgos invalida la demostración de viabilidad técnica que persigue el objetivo general: todos los flujos ejecutan correctamente su camino principal, y el análisis de concurrencia del §4.3.2 resuelve con precisión el problema más difícil del ciclo. Lo que delimitan es la distancia entre un producto mínimo viable que funciona en un entorno controlado y un sistema apto para operar con clientes reales; el Capítulo 8 la cuantifica en términos de trabajo pendiente.

Capítulo 5: Resultados y Validación

Los resultados de este capítulo corresponden a la ejecución de escenarios de prueba controlados, concebidos para verificar el comportamiento del sistema, y no a métricas de operación en producción, que exceden el alcance del trabajo (§1.9). Los escenarios se ejecutaron conforme al protocolo de la Tabla 14, que fija el entorno, las repeticiones, el conjunto de datos de prueba y el procedimiento de registro de la evidencia.

5.1 Escenarios de validación

Una precisión sobre el origen de los datos. La columna «Resultado esperado» de la Tabla 25 consigna valores derivados aritméticamente de las reglas de la Tabla 16, de modo que puede verificarse sin ejecutar el sistema. La columna «Resultado observado» recoge lo que produjo una corrida real: los escenarios E1, E1b, E2 a E6 y E8 a E10 se ejecutan de forma automatizada con `tests/escenarios.mjs`, que dispara los webhooks reales de n8n y consulta el estado resultante en la base. Cada corrida escribe `docs/evidencia-validacion.md`, y la Tabla 32 del Anexo A reproduce íntegra la última. Las entradas de la suite son exactamente las declaradas en la Tabla 25, y las aserciones verifican el desglose del score y no sólo su total: un resultado observado que reformulara al esperado no constituiría evidencia. E11 a E13 se dispararon manualmente desde el editor de n8n, vía que el protocolo contempla para los procesos programados, y E14 se ejecutó contra un doble de prueba y contra un adaptador sobre una pasarela real (§5.3).

Alcance de la cobertura. E1 a E4 verifican la normalización, la validación y el scoring (OE1, OE2); E1b, la fijación de términos y el envío de la propuesta (OE3a); E5 y E6, la aceptación, la facturación y la protección por token (OE3a). E7 cubre la actualización en tiempo real del tablero (OE4b); E8 a E10, los caminos alternativos posteriores a la propuesta (OE3a); E11 a E13, los tres procesos programados (OE5); y E14, la rama de cobro (OE3b). E15 ejercita la tercera condición de refutación del §1.3, reconstruyendo el ciclo de una oportunidad a partir de leads y de logs sin volver a consultar al motor de orquestación; E16 mide la deuda S1 invocando los trece webhooks desde un cliente que no es un navegador. Ambos se ejecutaron el 30 de agosto de 2026 —E16 se reejecutó el 31 tras cerrar S1 y S8—, con la salida en `docs/evidencia-E15-E16.md`. Los diecisiete escenarios se trazan en el Anexo E.

Tabla 25. Escenarios de validación, resultado esperado y evidencia

#	Escenario y entrada	Resultado esperado	Resultado observado (valores de la corrida)	Fecha de la corrida acreditante	Evidencia
E1	Lead válido de alto valor: desarrollo_web, USD 5000, urgencia alta, con teléfono de más de 6 caracteres y descripción de más de 50.	score = 40 + 30 + 20 + 5 + 5 = 100; nivel HOT; una fila en leads; el lead permanece en NUEVO a la espera de que el profesional fije los términos (§4.3.2).	score = 100 (40 + 30 + 20 + 5 + 5) · tier = HOT · estado = NUEVO · precio_propuesto = nulo · 1 fila en leads.	31-ago-2026 (suite automatizada, commit d1abe08).	Figura 16 (clasificación por score y nivel) y Figura 8 (flujo de captación).
E1b	Sobre el lead de E1, el profesional envía la propuesta desde el tablero fijando un precio distinto del presupuesto declarado: USD 7200, plazo «3 semanas» y un alcance de una línea.	El lead pasa a PROPUESTA_ENVIADA con token de aceptación y vencimiento; precio_propuesto = 7200; una segunda invocación de /propuesta-enviar no reabre la propuesta.	estado = PROPUESTA_ENVIADA · precio_propuesto = 7200 ≠ presupuesto = 5000 · plazo_propuesto = «3 semanas» · token generado y con vencimiento · la petición repetida respondió status = invalido.	31-ago-2026 (suite automatizada, commit d1abe08).	Suite automatizada tests/escenarios.mjs (docs/evidencia-validacion.md).
E2	Lead válido de valor medio: consultoria, USD 1000, urgencia media, sin teléfono y descripción de menos de 50 caracteres.	score = 20 + 15 + 12 + 0 + 0 = 47; nivel WARM; queda esperando los términos, igual que el HOT: estado NUEVO.	score = 47 (20 + 15 + 12 + 0 + 0) · tier = WARM · estado = NUEVO (queda esperando los términos, igual que el HOT).	31-ago-2026 (suite automatizada, commit d1abe08).	Suite automatizada tests/escenarios.mjs (docs/evidencia-validacion.md). La Figura 16 reproduce la fila que este escenario persiste en leads.
E3	Lead válido de bajo valor: soporte, USD 300, urgencia baja, sin teléfono y descripción breve.	score = 10 + 5 + 5 = 20; nivel COLD; sin propuesta; los dos avisos en paralelo de la rama fría —notificación interna por Telegram y acuse de recibo por correo al interesado (§4.3.1, §4.8)—; el lead permanece en NUEVO.	score = 20 (10 + 5 + 5 + 0 + 0) · tier = COLD · estado = NUEVO (no se envió propuesta).	31-ago-2026 (suite automatizada, commit d1abe08).	Suite automatizada tests/escenarios.mjs (docs/evidencia-validacion.md). La Figura 16 reproduce la fila que este escenario persiste en leads.
E4	Lead inválido, evaluado en cuatro invocaciones: nombre de un carácter, correo sin arroba y presupuesto cero por separado, y las tres condiciones juntas.	Las tres validaciones fallan en cada invocación; el flujo se detiene; 0 filas nuevas en leads; cuatro filas en logs con nivel ERROR.	filas nuevas en leads = 0 · el webhook respondió sin caerse en las cuatro invocaciones · 4 filas nuevas en logs con nivel ERROR.	31-ago-2026 (suite automatizada, commit d1abe08).	Figura 17 y suite automatizada, que envía las tres condiciones inválidas por separado y combinadas. La figura ilustra una sola: el formulario valida de a un campo por vez.
E5	Aceptación con token válido de una propuesta HOT.	Estado ACEPTADO; generación de la factura en PDF; envío por correo; una fila en facturas con fecha_vencimiento = emisión + 15 días; estado FACTURADO; notificación.	estado = FACTURADO · 1 fila en facturas · monto facturado = 7200, el precio que fijó el profesional y no el presupuesto declarado por el cliente · fecha_vencimiento – fecha_emision = 15 días.	31-ago-2026 (suite automatizada, commit d1abe08).	Figura 18(a) (aceptación satisfactoria), Figura 13 (correo de propuesta), Figura 14 (correo de la factura) y Figura 15 (comprobante en PDF).
E6	Reutilización del mismo enlace de aceptación: dos invocaciones consecutivas y dos concurrentes.	La primera aplica; las tres restantes no afectan ninguna fila y se derivan al nodo de operación nula; exactamente 1 fila en facturas para ese lead_id.	4 invocaciones (2 concurrentes + 2 consecutivas) · ninguna devolvió error de servidor · filas en facturas = 1.	31-ago-2026 (suite automatizada, commit d1abe08).	Figura 18(b) (enlace ya utilizado) y consulta SQL de recuento sobre facturas.
E7	Alta de un lead con el tablero abierto.	El tablero refleja el nuevo lead sin recarga manual, en menos de 3 s en el percentil 95 sobre 20 altas (RNF6).	20 de 20 eventos recibidos · p95 0,56 s · peor caso 0,81 s, por debajo del umbral de 3 s. La dispersión completa, en la Tabla	3-sep-2026, commit c09008d, posterior al corte (§5.6). El 26-ago hubo corridas exploratorias de cinco	Salida de scripts/medir-realtime.mjs, que informa cada latencia y el p95 y

#	Escenario y entrada	Resultado esperado	Resultado observado (valores de la corrida)	Fecha de la corrida acreditante	Evidencia
			26; instrumento scripts/medir-realtime.mjs.	repeticiones, por debajo de la muestra que fija el método.	termina con código distinto de cero si no se cumple el umbral: la medición es repetible por un tercero.
E8	Rechazo de la propuesta por el cliente desde la página de aceptación.	Estado PERDIDO; sin factura; notificación al profesional.	estado = PERDIDO · filas en facturas = 0.	31-ago-2026 (suite automatizada, commit d1abe08).	Figura 18(c).
E9	Pedido de cambios del cliente sobre la propuesta.	Estado EN_SEGUIMIENTO; mensaje del cliente persistido; notificación por Telegram.	estado = EN_SEGUIMIENTO · el mensaje del cliente quedó persistido en notas.	31-ago-2026 (suite automatizada, commit d1abe08).	Figura 18(d).
E10	Cambio del estado del trabajo desde el tablero (sección «Trabajos activos»).	estado_trabajo pasa de PENDIENTE a EN_PROGRESO y se sincroniza con la ficha de Notion.	estado_trabajo pasó a EN_PROGRESO · el lead tiene card_id, es decir que la sincronización con Notion se aplicó.	31-ago-2026 (suite automatizada, commit d1abe08).	Suite automatizada (Tabla 32). La sincronización con Notion se acredita por el card_id que devuelve su API y queda persistido en la fila, no por una captura de la ficha.
E11	Proceso programado de seguimiento: lote con dos leads vencidos a la vez —uno en su tercer y último intento, otro en el primero—, disparado a mano. El lote múltiple es parte de la entrada: la rama de «último intento» sólo falla con más de un caso.	Correo a los dos, una fila en seguimientos por cada uno, incremento de leads.seguimientos y de fecha_ultimo_seguimiento. El que agota el tercer intento pasa a PERDIDO; el otro sigue en EN_SEGUIMIENTO.	13 de 13 comprobaciones en verde: el del tercer intento quedó en PERDIDO con auditoría; el otro avanzó a seguimientos = 1 en EN_SEGUIMIENTO. El escenario estuvo fallando hasta esta corrida y las versiones previas de este documento lo daban por corregido sin estarlo (§5.2).	2-sep-2026, commit c09008d, posterior al corte documental (§5.6), con el defecto corregido y sobre un lote de dos leads. Corridas anteriores: 23-ago-2026 y 2-sep-2026 previa a la corrección, ambas fallidas en la transición a PERDIDO.	Corrida manual desde el editor de n8n: los disparadores programados no se pueden ejecutar por la API ni por la línea de comandos. Evidencia en docs/evidencia-E11-E12.md; instrumento tests/procesos-programados.mjs.
E12	Proceso programado de recordatorios de pago: lote con dos facturas PENDIENTE de vencimiento distinto —una a dos días de vencer y otra vencida hace diez—, disparador ejecutado manualmente.	La primera se clasifica con nivel RECORDATORIO y la segunda con nivel URGENTE; cada una recibe su correo y se le incrementa recordatorios_enviados. La vencida dispara además la notificación por Telegram.	Ambos escalones en la misma corrida. RECORDATORIO —nunca antes ejercitado— sobre una factura sembrada a dos días del vencimiento; URGENTE sobre una vencida hace diez. La corrida reveló que la rama urgente arrastraba el defecto de propagación de E11 (§5.2): notificaba una sola factura por lote. Corregido, notifica las cuatro.	2-sep-2026, commit c09008d, posterior al corte documental (§5.6): corrida acreditante de ambos escalones, con el defecto ya corregido. Corridas anteriores: 23 y 27-ago-2026, que acreditaron el escalón URGENTE pero no el escalón RECORDATORIO.	Corrida manual desde el editor de n8n, según prevé la Tabla 14. Evidencia en docs/evidencia-E11-E12.md; instrumento tests/procesos-programados.mjs.
E13	Proceso programado de reporte diario de métricas, disparador ejecutado manualmente.	Se envía por Telegram un resumen con los indicadores de metrics_mensuales del mes en curso, coincidentes con los del tablero.	La API de Telegram devolvió ok = true y message_id = 594, y el texto reproduce los indicadores del mes (2026-08, 6 leads, facturado 11.700 USD, cobrado 7.200, tasa 61,5 %, 8 propuestas esperando términos), coincidentes con metrics_mensuales. El recuento de propuestas no procede de esa vista y es histórico acumulado (Tabla 19), lo que explica que supere al total del mes.	28-ago-2026 (ejecución contra el sistema real; causa corregida el 24-ago-2026).	Respuesta de la API de Telegram (ok = true, message_id = 594), que consigna el texto entregado y permite contrastarlo con la vista. Disparo desde el editor de n8n sobre los nodos reales.
E14	Pago real con tarjeta de prueba de MercadoPago sobre una factura PENDIENTE.	El webhook POST /webhook/mp/notificación consulta la API de MercadoPago, confirma	Contra el doble: estado_pago PENDIENTE → COBRADO, mp_payment_id = 900000001, notificación duplicada aplicada	27-ago-2026 (doble de prueba); 31-ago-2026 (adaptador de Stripe).	Dos instrumentos (§5.3): un doble del contrato documentado de

#	Escenario y entrada	Resultado esperado	Resultado observado (valores de la corrida)	Fecha de la corrida acreditante	Evidencia
		status = approved y marca la factura COBRADO solo entonces; una segunda notificación para el mismo pago no repite el efecto ni la notificación por Telegram (idempotencia).	una sola vez. Contra el adaptador de Stripe en modo de prueba: mismo ciclo, con PaymentIntent real (pi_3UAXFV48ekm6gWbu2scTyQzZ) confirmado con la tarjeta pm_card_visa y mp_payment_id = 900000002, identificador sintético que exige el nodo real.		MercadoPago (tests/e14-cobro-mp.mjs) y un adaptador contra Stripe en modo de prueba (tests/adaptador-stripe-e14.mjs). Ninguno verifica el servicio real de MercadoPago: la activación de credenciales de prueba falla por causa del proveedor (Capítulo 8, punto 6).
E15	Trazabilidad: una oportunidad recorre la captación y el envío de la propuesta, y luego se reconstruye su historia leyendo únicamente la base.	La línea de tiempo se reconstruye a partir de las marcas temporales de leads y de las filas de logs; arranca en el alta, es cronológicamente consistente y el estado actual se explica por la última transición fechada.	Línea reconstruida NUEVO → PROPUESTA_ENVIADA, con la fila de auditoría propuesta_enviada intercalada · score = 100 y tier = HOT recuperables · precio_propuesto = 8400 ≠ presupuesto = 6000 · 10 de 10 comprobaciones en verde.	30-ago-2026 (contra el sistema levantado).	Corrida de tests/trazabilidad.mjs (npm run test:trazabilidad). El escenario elimina el lead de prueba al terminar, conforme al protocolo de la Tabla 14. Salida completa en docs/evidencia-E15-E16.md.
E16	Exposición: los trece webhooks se invocan desde un cliente que no es un navegador y sin credencial alguna (deuda S1 de la Tabla 22).	Los seis del panel responden 403; los seis sin autenticación de origen atienden la petición —atender es no rechazarla en el borde (§4.6), con independencia de que el efecto se aplique—; el de la pasarela la contrasta contra la API del proveedor.	6 de 13 atendieron la petición sin credencial, en el sentido del §4.6: cinco no admiten credencial por diseño y pago-confirmado atiende pero no aplica la confirmación sin su token por factura. Los seis del panel respondieron 403 y mp/notificacion ignoró la notificación sin firma (S8). El alta prosperó desde un cliente que no es un navegador, lo que mide que la lista blanca de orígenes no es una autenticación.	30-ago-2026 (contra el sistema levantado); reejecutado el 31-ago-2026 tras cerrar S1 y S8.	Corrida de tests/exposicion-webhooks.mjs (npm run test:exposicion). El escenario falla si el reparto deja de coincidir con la Tabla 22, en cualquiera de las dos direcciones. Salida completa en docs/evidencia-E15-E16.md.

Nota. Elaboración propia. En E1 a E3 los valores de «Resultado esperado» se derivan aritméticamente de la Tabla 16 y son verificables por cálculo sin ejecutar el sistema. En los demás proceden del requisito, del contrato de integración o de la garantía que cada escenario ejercita: E4 del RNF3a, E5 y E6 del RNF2, E7 del RNF6, E8 a E10 de las transiciones de la Tabla 20, E11 a E13 de la definición de los procesos programados (§4.3.5), E14 del contrato documentado de la API del proveedor (§4.3.3), E15 de la tercera condición de refutación del §1.3 y E16 del reparto de la Tabla 22. Todos los escenarios cuentan con evidencia registrada; la de E11 y E12 proviene de una corrida manual y la de E14, de un doble de prueba y un adaptador, circunstancias que se declaran en cada celda en lugar de presentarse como equivalentes a una corrida automatizada contra el servicio real.

5.2 Alcance de la validación

Conviene precisar qué acredita la validación y qué no. Como comprobación de reproducibilidad del entorno, el 24 de agosto de 2026 el flujo se importó en una instancia de n8n de la versión declarada en la Tabla 12 y quedó activo sin errores, y los escenarios que la suite tenía entonces se ejecutaron en verde tres veces seguidas. La suite cubre diez

escenarios de camino principal y alternativo —E1, E1b, E2 a E6 y E8 a E10—, y cada corrida verifica el desglose del resultado y no sólo su total. El protocolo de la Tabla 14 exige tres repeticiones independientes para los escenarios de camino principal, y toda corrección sustantiva obliga a reejecutarlos completos; ambas exigencias se cumplieron, con el detalle cronológico en el Anexo I. E7, el único escenario de camino principal ajeno a la suite, se acredita con la corrida de veinte repeticiones que su método exige, cuya estadística completa reporta la Tabla 26 por ser esta la única medición cuantitativa del trabajo.

Tabla 26. Latencia de propagación al tablero (RNF6), veinte repeticiones

Estadístico	Valor (s)
n (repeticiones)	20
Mínimo	0,52
Media	0,55
Mediana	0,54
Percentil 95	0,56
Máximo	0,81
Desvío estándar	0,06
Umbral (RNF6)	3,00
Resultado	Cumple: los veinte eventos se recibieron y el peor caso, 0,81 s, queda muy por debajo del umbral de 3 s

Nota. Elaboración propia a partir de la salida de scripts/medir-realtime.mjs, corrida acreditante del 3 de septiembre de 2026 (00:22:04 UTC) sobre el commit c09008d con el árbol de trabajo limpio, archivada en docs/evidencia-realtime.md. Los siete estadísticos los calcula el propio script sobre las veinte latencias individuales. Se informa la dispersión completa por tratarse de la única medición cuantitativa del trabajo. Dos precisiones de alcance: la magnitud medida es el intervalo entre la confirmación de la escritura y la recepción del evento en el cliente, y no incluye el repintado de la interfaz; y con $n = 20$ el percentil 95 se resuelve en el penúltimo valor ordenado, de modo que el p95 informado (0,56 s) y el peor caso (0,81 s) son contiguos y aquél no aporta información de cola independiente de éste. La muestra es la que fija el método del §3.7 y no habilita inferencia bajo carga concurrente ni en redes distintas de la del ensayo.

El ejercicio manual de los tres procesos programados (E11–E13) resultó más revelador que confirmatorio, y su desenlace corrige una afirmación que este mismo documento sostuvo hasta el 2 de septiembre de 2026. La corrida del 23 de agosto puso al descubierto que la rama de «último intento» de E11 —la que marca el lead como PERDIDO— no se completaba cuando el proceso manejaba más de un caso a la vez. El diagnóstico fue correcto y se aplicó una corrección, pero no se reejecutó el escenario: la corrección no se verificó y el documento pasó a declarar el defecto cerrado. La reejecución del 2 de septiembre, sobre un lote sembrado a propósito con dos leads vencidos a la vez, mostró que el defecto seguía vivo.

La causa es una interacción entre dos decisiones del flujo, y se estableció leyendo los conteos de ítems por nodo del registro de ejecución del motor. Los nodos que cierran la rama ejecutaban una sentencia de actualización sin cláusula de retorno, y el nodo de base de datos de $n8n$ colapsa a un único ítem la salida de una sentencia que no devuelve filas, cualquiera sea la cantidad de entradas procesadas: tres ítems entraron y salió uno. Aguas abajo, el nodo

condicional resolvía su condición indexando por posición la salida de un nodo anterior; con el flujo ya colapsado, esa posición era siempre la primera. La transición a PERDIDO resultaba por eso inalcanzable siempre que el proceso atendiera más de un lead, y sólo ocurría por casualidad cuando el único lead vencido era además el que agotaba su tercer intento.

El mismo defecto afectaba a la rama de avisos urgentes de E12, que comparte esa forma: notificaba una sola factura de las cuatro que el lote traía en esa condición. La corrección fue doble y alcanzó a los dos procesos. Se agregó la cláusula de retorno a las tres sentencias de actualización que alimentan una rama, con lo que se restituye el flujo de un ítem por fila. Y se reemplazó el indexado por posición por la resolución por ítem emparejado que el propio motor provee, en los cinco nodos que lo usaban. La segunda mitad se apoya en evidencia de la misma corrida: el nodo que actualiza el contador de seguimientos ya usaba esa resolución y actualizó correctamente los tres leads, en la misma ejecución en que el condicional fallaba. Reejecutados ambos procesos sobre lotes múltiples, las trece comprobaciones terminaron en verde.

De este episodio conviene retener una lección de método más que de programación. El defecto sobrevivió nueve días a su propio diagnóstico porque se corrigió sin reejecutar, y el documento lo declaró cerrado sobre la base de esa corrección y no de una verificación. Es la misma deriva entre documento y artefacto que el §7.4 identifica como el error más costoso del trabajo. Es también la razón por la que el instrumento que ahora ejercita estos dos procesos siembra deliberadamente un segundo caso en cada lote: con un solo lead la prueba pasaría incluso con el defecto presente.

Queda un único escenario que no pudo ejercitarse contra el proveedor real: E14, la rama de cobro con MercadoPago. Se ejecutó contra un doble que reproduce el contrato documentado de su API y, además, contra un adaptador que llama a una pasarela real en modo de prueba; ninguno de los dos prueba el servicio real de MercadoPago (§5.3).

5.3 Rama de cobro y doble de prueba

El escenario E14 merece una explicación aparte. El 27 de agosto de 2026 se intentó activar credenciales de prueba en el panel de desarrolladores de MercadoPago para la aplicación registrada del proyecto, y la activación falló de forma consistente con un error 422 del propio backend del proveedor (endpoint interno `testUserAutomation/create`), sin motivo específico. Se descartó que fuera un problema de la cuenta o de la aplicación: el mismo error se reprodujo creando una cuenta de prueba vendedor nueva y probando desde

la cuenta de un tercero independiente. El patrón —una falla intermitente de la activación de usuarios de prueba en el entorno de sandbox— está documentado en reportes de otros desarrolladores en foros públicos. La causa es del proveedor y no del artefacto.

Ante esa indisponibilidad, el escenario se ejecutó contra un doble que reproduce el contrato documentado de los dos únicos endpoints que el flujo consume: la creación de la preferencia de Checkout Pro y la consulta del pago. Para poder hacerlo se parametrizó la base de la API como variable de entorno (`MP_API_BASE`), con el servicio real como valor por omisión, de modo que el comportamiento de producción no cambia y el artefacto gana testabilidad; los nodos que intervienen son los reales del flujo, sin modificar. La corrida verificó, con los valores de la Tabla 25, que la preferencia se crea y su identificador se persiste en la factura. Verificó también que la notificación dispara la consulta del pago contra la API —el flujo no confía en el cuerpo notificado— y que la factura pasa de PENDIENTE a COBRADO con su `mp_payment_id`. Y verificó que una notificación repetida no vuelve a aplicar el efecto. El instrumento es `tests/e14-cobro-mp.mjs`, con el doble en `tests/mp-doble.mjs` y el andamiaje en `tests/fixtures-e14/`, todo versionado, de modo que la corrida es repetible por un tercero.

El 31 de agosto de 2026 se sumó un segundo instrumento con un alcance distinto: `tests/adaptador-stripe-e14.mjs` habla el mismo contrato hacia los nodos reales de n8n, pero por dentro llama a la API real de Stripe en modo de prueba (Stripe, s. f.) en lugar de fabricar el desenlace del pago. La corrida de esa fecha confirmó un `PaymentIntent` real (`pi_3UAXFV48ekm6gWbu2scTyQzZ`) con la tarjeta de prueba oficial `pm_card_visa` y verificó sobre los nodos reales el mismo ciclo que exige la Tabla 25.

Las divergencias declaradas respecto de producción son tres. La primera es de moneda (USD en el adaptador de prueba, `MP_CURRENCY = ARS` en producción). La segunda, de mecanismo de confirmación: el adaptador confirma el pago directamente contra la API, sin el checkout hospedado que usaría un cliente real. La tercera es del identificador del pago y merece detalle porque afecta al dato persistido. El nodo real Code - Procesar Pago MP exige que el `payment_id` sea numérico puro y los identificadores nativos de Stripe no lo son, de modo que el adaptador mantiene —igual que el doble— un mapa interno de un identificador numérico sintético hacia el real. En la corrida acreditada, `mp_payment_id` almacenó ese valor sintético (9000000002) y el identificador real del `PaymentIntent` quedó en `mp_preference_id`, columna TEXT que no impone esa restricción. Las dos primeras divergencias no alteran la lógica de negocio; la tercera tampoco, pero implica que el

identificador de pago almacenado no es el que devolvería la pasarela en producción, y por eso se declara aquí y no en una nota al pie.

Alcance de lo que estas corridas acreditan. Contra el doble, la corrida verifica el comportamiento del artefacto frente al contrato documentado del proveedor y no el del proveedor mismo: un cambio no anunciado en la API de MercadoPago no sería detectado. Contra el adaptador de Stripe, acredita además que la misma lógica de cobro sostiene el ciclo de vida de un pago real, no fabricado, de una pasarela real. Ninguna de las dos prueba el servicio de MercadoPago en particular: esa validación sigue pendiente (Capítulo 8, punto 6). Una segunda precisión de método vale para ambas. La factura PENDIENTE sobre la que opera el escenario se sembró directamente en la base como precondition, en lugar de producirse recorriendo el flujo de aceptación. La razón es que los nodos de envío de correo detienen la cadena cuando la credencial OAuth del servicio está vencida, como ocurría en el entorno de ambas corridas. La emisión de la factura por el camino completo es lo que verifica E5, ya ejecutado; lo que E14 ejercita de punta a punta es la rama de cobro.

5.4 Pruebas automatizadas

De forma complementaria a la validación por escenarios, el repositorio incluye una prueba de humo automatizada (`tests/smoke_code_nodes.js`), escrita en Node.js sin dependencias externas. Lee la definición del flujo, extrae los nodos de tipo Code y ejecuta el código de cada uno con un conjunto de datos representativo y con sustitutos de las variables del entorno de n8n, sin levantar el motor. Para cada nodo verifica que su ejecución no arroje errores y que la salida respete la forma esperada, informando el resultado nodo por nodo y devolviendo un código de salida distinto de cero ante cualquier fallo. Es, por tanto, una prueba de humo que protege la lógica de los nodos Code frente a regresiones estructurales, y no una verificación de la corrección funcional de las reglas de negocio.

A esa prueba se suma una batería más amplia, que se ejecuta sin levantar el sistema y que cubre precisamente las reglas que constituyen el aporte declarado. `tests/scoring.js` ejecuta el código real del nodo de calificación sobre 9.240 combinaciones de presupuesto, urgencia, servicio, teléfono y descripción, y verifica que el resultado reproduzca la Tabla 16 y que los umbrales sigan siendo configurables. Es también el instrumento que ejecuta la partición del espacio de entrada del §3.7.1, cuyo alcance ese apartado reporta frontera por frontera. La prueba de humo cubre los 41 nodos Code de ambos flujos versionados. Esa cifra incluye los del módulo de tickets, excluido del alcance en §1.9, de modo que no debe leerse como cobertura del artefacto validado: el flujo del CRM aporta 31 de esos nodos (Tabla 33).

tests/verificar_sql.mjs compila con PREPARE las consultas de los nodos Postgres contra el esquema real, de modo que una columna mal escrita se detecta antes de producción. tests/verificar_qls.mjs levanta un PostgreSQL desechable de la misma versión mayor que la instancia real, aplica db/schema.sql tal cual está versionado y ejecuta 33 casos de seguridad a nivel de fila rol por rol, nueve de ellos bajo n8n_writer (S4). tests/idempotencia.mjs hace lo mismo con las dos garantías que la Tabla 22 da por resueltas. Sobre ese mismo PostgreSQL ejecuta la inserción condicional del lead y las tres consultas de la reconciliación, leídas del propio flujo exportado y no copiadas a mano. Comprueba en veintidós aserciones que un envío repetido no cree un segundo lead, que dos envíos simultáneos tampoco lo hagan y que dos corridas de la reconciliación no emitan dos facturas. tests/auth_errores.js cierra el RNF5b: declara los veintidós códigos que la plataforma de autenticación puede devolver en los tres flujos que la aplicación invoca y comprueba que todos tengan mensaje en español.

Tres de estos verificadores —el de seguridad a nivel de fila, el de idempotencia y el de propagación al tablero— informaban hasta el 2 de septiembre de 2026 únicamente por consola. Sus resultados no quedaban auditables, por eso, sin volver a ejecutarlos: era la primera observación de método del evaluador externo (Anexo K). La instrumentación se resolvió el 2 de septiembre de 2026, con posterioridad al corte documental que fija el §5.6, de modo que en el estado que este documento describe esa observación seguía abierta. Desde esa fecha los tres escriben su salida completa en docs/, cada uno con la marca temporal de la corrida, el commit, el estado del árbol de trabajo y el código de salida, igual que ya hacían los otros tres instrumentos del protocolo.

Y tests/verificar_afirmaciones.js recalcula desde el repositorio los números que este documento afirma —nodos, webhooks, tablas, vistas, umbrales y las ponderaciones de la Tabla 16— y falla si dejaron de ser ciertos. Es la contención contra la deriva entre el documento y el artefacto que el §7.4 señala como el error más costoso del trabajo. Verificaba al principio sólo cantidades, y eso resultó insuficiente, porque tres afirmaciones del documento habían dejado de describir el flujo sin que nada fallara: ninguna era un número. Se le incorporaron por eso diez comprobaciones de forma, que se verifican contra el flujo exportado. Cinco recaen sobre el orden y las transiciones: que el cálculo del puntaje precede a la persistencia, que la rama fría acusa recibo al interesado, que una factura sólo pasa a COBRADO desde PENDIENTE, que ningún nodo escribe VENCIDA ni ANULADA y que la transición a ACEPTADO es condicional sobre el estado previo. Las otras cinco recaen sobre los controles de acceso y la separación de responsabilidades. Comprueban que los seis webhooks del panel —y sólo éstos— exigen Header Auth, que la captación sigue sin

autenticación de origen, que pago-confirmado exige su token por factura, que el token de aceptación rota en cada reenvío y que ningún componente del frontend escribe en las tablas de negocio. A ellas se suman tests/trazabilidad.mjs y tests/exposicion-webhooks.mjs, que instrumentan E15 y E16.

5.5 Cobertura de requisitos y evidencia registrada

Los once requisitos funcionales y los nueve no funcionales se trazan individualmente en el Anexo E, con el resultado de su verificación. Los que quedaron sin verificar del todo se identifican como tales: además del cierre de proyecto (RF7), verificado manualmente y sin escenario propio, son los tres que dependen de los procesos programados. El seguimiento (RF8) y los recordatorios de pago (RF9) quedaron verificados por completo el 2 de septiembre de 2026, cuando se sembraron las precondiciones que sus tramos finales exigían: un lead en su tercer y último intento y una factura dentro de la ventana de uno a tres días. Esa corrida obligó, además, a corregir el defecto de propagación que relata el §5.2. El reporte diario (RF10) quedó pendiente hasta el 28 de agosto de 2026, cuando se lo ejecutó contra el sistema real: la API de Telegram confirmó la entrega y los indicadores del mensaje coinciden con los de metrics_mensuales.

Como evidencia complementaria, el Anexo A reúne las capturas del formulario, del tablero, de los flujos configurados en n8n y de los desenlaces de la página pública de aceptación, y reproduce en su Tabla 32 la salida de la corrida completa de la suite. Cada captura se numera y se titula de forma individual, de modo que toda remisión del texto pueda resolverse sin ambigüedad.

5.5.1 Cobertura de la suite sobre el flujo

La prueba de humo del §5.4 recorre los 41 nodos Code de los dos flujos versionados. Resta estimar qué proporción del flujo del CRM —161 nodos funcionales, Tabla 33— ejercitan de extremo a extremo los diecisiete escenarios, y qué partes no toca ninguno. E1 a E10 recorren la captación, la calificación, el envío de la propuesta, la aceptación, la facturación, el rechazo, el pedido de cambios y el estado del trabajo; E11 a E13, los tres procesos programados; E14, la rama de cobro; y E15 y E16, la trazabilidad por lectura de la base y la exposición de los trece webhooks. La Tabla 27 cruza ese recorrido con la composición por tipo de nodo de la Tabla 33.

Tabla 27. Cobertura estimada de los diecisiete escenarios sobre los nodos del flujo del CRM

Tipo de nodo (Tabla 33)	Total	Ejercitados por los 17 escenarios (est.)	No ejercitados (est.)	Observación
n8n-nodes-base.postgres	35	≈ 27	≈ 8	Camino principal y procesos programados. Sin ejercitar: parte de la reconciliación fuera del camino feliz y las transiciones de estado_pago a VENCIDA y ANULADA, que ningún componente ejecuta (§4.8).
n8n-nodes-base.code	31	≈ 24	≈ 7	Normalización, scoring, validación de token y generación de documentos, cubiertos por E1–E6 y E15. Sin ejercitar: ramas de la integración con MercadoPago que sólo corren contra el servicio real y el formateo de la rama de errores.
n8n-nodes-base.respondToWebhook	20	≈ 17	≈ 3	E16 invoca los trece webhooks; se ejercitan la respuesta de cada entrada y sus desenlaces principales. Sin ejercitar: respuestas de ramas de error poco frecuentes.
n8n-nodes-base.if	16	≈ 12	≈ 4	Se recorren ambas salidas de las bifurcaciones del camino principal y las alternativas de E6, E8 y E9. Sin ejercitar: guardas que sólo se activan ante fallos concurrentes.
n8n-nodes-base.telegram	15	≈ 10	≈ 5	E12 confirmó la vía de notificación tras corregirla y E4 alcanza el aviso de error crítico por la rama global de errores. Sin ejercitar: avisos de ramas infrecuentes.
n8n-nodes-base.webhook	13	13	0	E16 invoca los trece puntos de entrada desde un cliente que no es un navegador.
n8n-nodes-base.httpRequest	13	≈ 9	≈ 4	Gotenberg (E5), API de Notion (E1b, E10) y API de MercadoPago mediante el doble y el adaptador (E14). Sin ejercitar: la creación de preferencia de la reconciliación y sincronizaciones de ramas secundarias.
n8n-nodes-base.gmail	8	≈ 5	≈ 3	Propuesta (E1b), factura (E5), recordatorio (E12) y acuse de la rama fría (E3). Sin ejercitar: el correo de testimonio del cierre (RF7, verificado manualmente).
n8n-nodes-base.noOp	5	≈ 2	≈ 3	E6 ejercita la operación nula de la aceptación duplicada. Sin ejercitar: las ramas de lead duplicado y de factura ya

Tipo de nodo (Tabla 33)	Total	Ejercitados por los 17 escenarios (est.)	No ejercitados (est.)	Observación
				emitida de la reconciliación.
n8n-nodes-base.scheduleTrigger	4	3	1	Seguimiento (E11), recordatorios (E12) y reporte diario (E13) se dispararon manualmente. Sin ejercitar: el disparador por calendario de la reconciliación.
n8n-nodes-base.errorTrigger	1	1	0	E4 la ejercita de forma indirecta: la detención del flujo ante una entrada inválida dispara esta rama, que registra las cuatro filas de nivel ERROR. Ningún escenario la provoca de forma deliberada.
Total funcional	161	≈ 123	≈ 38	Del orden de tres cuartas partes de los nodos funcionales, entendido como cota inferior. No se expresa como porcentaje con cifras significativas porque el recuento proviene de la inspección del flujo y no de una herramienta de cobertura. Los 17 stickyNote, no funcionales, quedan fuera del recuento.

Nota. Elaboración propia a partir de la inspección de workflow/crm_postgres.json y de la traza de los diecisiete escenarios. Los totales por tipo de nodo son los de la Tabla 33. Las columnas «Ejercitados» y «No ejercitados» son una estimación conservadora por lectura del flujo y no una medición con herramienta de cobertura: n8n no la ofrece de fábrica y el valor real sólo puede ser igual o mayor. Su instrumentación se registra como parte del punto 7 del Capítulo 8.

5.6 Trabajo posterior al corte documental

Las cifras y las descripciones del Capítulo 4 y del Anexo B corresponden a un estado preciso del repositorio: el commit d1abe08, del 31 de agosto de 2026. Es también el que ejecutó el evaluador externo del Anexo K y aquel sobre el que se obtuvo la corrida acreditante que reproduce la Tabla 32. Fijar ese corte obliga a declarar qué ocurrió después, porque el repositorio siguió recibiendo trabajo y quien lo clone hoy encontrará un artefacto mayor que el que este documento describe.

Entre el 1 y el 3 de septiembre de 2026 se incorporaron cuatro conjuntos de cambios y una corrección. El 1 de septiembre se diferenciaron los cuatro desenlaces del enlace de aceptación que el §4.3.2 señala confundidos bajo un único mensaje, y se unificó la nomenclatura de las rutas propias que el §4.5 declara divergente, con lo que /lead/nuevo pasó a /lead-nuevo (commit 0c073de). Se incorporaron después el proceso programado que reclama las propuestas calificadas sin enviar, que es el riesgo que la última fila de la Tabla 24 declara abierto. Se incorporaron también las transiciones a VENCIDA y ANULADA del

enumerado pago_estado que la regla R7 declara previstas y no ejecutadas, con un punto de entrada nuevo, /factura-anular (commit 5992ca7). Se añadió luego limitación de tasa delante de los cinco webhooks públicos y el enmascarado del token en los registros, que son las dos mitigaciones que el punto 1 del Capítulo 8 deja pendientes de S1 y de S3 (commit 43881bf). Y se incorporó al formulario el aviso de privacidad del artículo 6 de la Ley 25.326 que el §2.7 y el punto 5 del Capítulo 8 reclaman (commit 96fd811). El 2 de septiembre se corrigió el defecto de propagación S9 que el §5.2 relata y se instrumentó el archivado en docs/ de los tres verificadores que hasta entonces informaban sólo por consola (commit c09008d).

El efecto sobre las cifras es considerable y conviene anticiparlo. El flujo del CRM pasó de 178 nodos —161 funcionales y 17 notas— a 226, de los cuales 207 son funcionales y 19 son notas. Los puntos de entrada pasaron de trece a catorce; los procesos programados, de cuatro a cinco; las tablas del esquema, de cinco a seis, por la incorporación de rate_limit_log; y los índices, de once a doce. Las cifras que este documento declara son las del corte y no las del repositorio actual. El propio instrumento de contención lo hace explícito: ejecutado sobre el estado posterior, tests/verificar_afirmaciones.js no se limita a fallar sino que informa cada desvío junto a la razón que lo produjo, de modo que la distancia entre el documento y el artefacto queda medida y no supuesta.

De ahí se sigue el alcance de lo que este capítulo acredita. Ninguno de los cambios del 1 de septiembre está validado por los escenarios que aquí se reportan: se los declara implementados, no verificados, y las secciones que los dan por pendientes describen correctamente el estado del corte. Tres corridas acreditantes, en cambio, se ejecutaron sobre el estado posterior y así lo consignan sus celdas. La del RNF6 que reporta la Tabla 26, del 3 de septiembre, no se ve afectada por el cambio de estado, porque mide el intervalo entre la confirmación de la escritura en la base y la recepción del evento en el cliente y no interviene en ella el flujo de orquestación. Las de E11 y E12, del 2 de septiembre, sí acreditan el comportamiento del artefacto ya corregido y no el del corte, en el que el defecto S9 seguía presente. Es la razón por la que los requisitos RF8 y RF9 se declaran verificados sobre el estado posterior y no sobre el que el Capítulo 4 describe.

Declarar este corte es, además, la aplicación del aprendizaje que el §7.4 formula. El documento y el artefacto se desincronizan si no se los versiona juntos, y la contención no puede ser sólo un verificador de cantidades: exige decir a qué versión corresponde cada afirmación. Un tribunal que clone el repositorio encontrará un artefacto más avanzado que el descrito, y esa diferencia no es una omisión sino el intervalo entre la fecha del corte y la de la entrega.

Capítulo 6: Discusión

Los resultados confirman que el ciclo de vida del cliente puede automatizarse de extremo a extremo con una arquitectura orientada a eventos de bajo código, en línea con lo que la literatura anticipa (Hohpe y Woolf, 2003; Järvinen y Taiminen, 2016). La concentración de la lógica de integración en n8n confirma buena parte de la ventaja de mantenibilidad que se atribuye a este estilo: incorporar un nuevo paso del flujo no exige modificar el frontend ni el esquema de datos. Conviene, sin embargo, acotar esa afirmación: Gmail, Telegram, Notion y Gotenberg se invocan directamente desde nodos del flujo, sin una capa de abstracción de notificaciones ni de generación documental, de modo que incorporar un canal nuevo sí exige tocar el flujo en cada punto donde hoy se notifica. La ventaja de mantenibilidad es real entre capas y menor dentro de la capa de orquestación.

El aporte del scoring merece una lectura matizada. La clasificación en niveles ordena efectivamente el esfuerzo comercial, pero los umbrales y las ponderaciones se fijaron a partir de un criterio experto y no de datos históricos de conversión. Su validez es, por lo tanto, de diseño y no empírica: para afirmar que el score predice la conversión real sería necesaria una etapa posterior de calibración con datos de operación.

A esa validez de diseño se suma una limitación del propio modelo, analizada en §4.3.1: cuatro de los nueve valores de servicio ponderados no pueden originarse desde el formulario, y la opción «Otro» recibía la ponderación mínima, lo que sesgaba hacia COLD precisamente las oportunidades que el sistema no supo clasificar. Ese sesgo se corrigió durante el desarrollo (§4.3.1 y Anexo I), aunque la ponderación que lo reemplaza sigue siendo de criterio experto y no de datos de conversión. La clasificación ordena el esfuerzo comercial, pero lo hace sobre un espacio de entrada más estrecho que el que la Tabla 16 sugiere.

El contraste con la literatura arbitrada permite situar mejor lo que este trabajo no midió. Amir y Atif (2026) reportan, sobre un flujo de captación automatizado con n8n en un negocio de pequeña escala, una reducción del tiempo de ejecución de 185,35 a 1,23 segundos y la desaparición de la tasa de error del 5 % que observaban en la ejecución manual. En orden de magnitud, es el resultado que la versión original del objetivo general prometía y que su estrategia de validación no podía acreditar. Dos precisiones impiden trasladar el hallazgo. La fuente es un preprint sin revisión por pares. Y, más de fondo, mide el tiempo de ejecución del flujo —lo que tarda el sistema— y no el tiempo del profesional, que es el factor que el Modelo de Costos Estándar exige y que el §2.8 declara sin medir. La

comparación no valida por transitividad el ahorro de este artefacto: lo que hace es fijar qué debería medir el estudio del punto 4 del Capítulo 8 para que la afirmación fuera propia y no prestada.

La validez de diseño del scoring admite una lectura análoga. Wu et al. (2024) concluyen que la ponderación de atributos por reglas explícitas sigue siendo el método predominante y que las alternativas basadas en aprendizaje automático exigen volúmenes históricos de los que un profesional individual no dispone, lo que respalda la elección metodológica de la Tabla 16. Pero esos mismos autores señalan que el campo carece de un marco de evaluación común y de conjuntos de datos comparables. La literatura tampoco ofrece, entonces, un umbral de referencia contra el cual contrastar los cortes de 70 y de 40 puntos: avala el tipo de modelo elegido, no sus valores. Moreira et al. (2025) refuerzan por otra vía la decisión de alcance y comparten la misma limitación de fondo: validación en una única demostración práctica, sin operación en producción. La coincidencia es informativa, porque muestra que el límite que este trabajo declara no es un descuido propio sino el estado habitual de la evidencia disponible en el campo.

6.1 Comparación con el estado del arte

El Capítulo 2 analizó HubSpot, Pipedrive, Zapier y Make para justificar la decisión de construir. Queda por responder qué obtuvo efectivamente el freelancer que no habría obtenido configurando alguna de esas herramientas: la Tabla 28 estima el costo de propiedad a doce meses y la Tabla 29 compara las prestaciones. Ambas deben leerse con sus supuestos declarados, y la primera con una salvedad que su propio título recoge: es una estimación hipotética. El trabajo no operó en producción ni desplegó el sistema en el proveedor de infraestructura que la tabla cotiza, porque no se ejecutó de forma autoalojada en un equipo local. La fila de infraestructura corresponde, por lo tanto, al despliegue en la nube que el punto 2 del Capítulo 8 propone y no a un gasto incurrido. A doce meses, la solución construida cuesta aproximadamente USD 56 sólo en infraestructura de alojamiento, frente a USD 588 de Pipedrive en su primer nivel con automatización de flujos y USD 360 de Zapier en su plan de entrada. La comparación no es plenamente equivalente —Pipedrive y Zapier incluyen soporte del proveedor y mantenimiento del producto, mientras que la solución construida traslada ese costo al tiempo del propio profesional—, pero el diferencial de costo directo es sustancial.

Tabla 28. Estimación hipotética de costos de propiedad a doce meses para un freelancer individual

Concepto	Solución construida	CRM comercial (HubSpot / Pipedrive)	iPaaS (Zapier / Make)
Modelo de facturación	Costo de la infraestructura que hospeda n8n. Sin costo por ejecución.	Suscripción por asiento y por mes; las funciones de scoring y automatización avanzada residen en los niveles superiores.	Suscripción por volumen de tareas u operaciones ejecutadas por mes.
Costo mensual estimado (USD)	USD 4,70 (Hetzner Cloud CX22; EUR 4,35/mes; fecha de consulta: 17 de agosto de 2026)	USD 49,00 (Pipedrive, plan Growth, facturación mensual; fecha de consulta: 17 de agosto de 2026)	USD 29,99 (Zapier, plan Starter, 750 tareas/mes; fecha de consulta: 17 de agosto de 2026)
Costo de la base de datos	Incluido en el nivel gratuito de Supabase para el volumen previsto; escala con el almacenamiento.	Incluido en la suscripción.	No aplica: no es sistema de registro.
Costo de la generación de PDF	Gotenberg autoalojado, sin costo de licencia.	Integración de terceros, facturada aparte.	Integración de terceros, facturada aparte.
Costo total a 12 meses (USD)	USD 56,40	USD 588,00	USD 359,88
Costo no monetario	Desarrollo inicial y mantenimiento del flujo, la base y el frontend, a cargo del profesional.	Configuración inicial; el mantenimiento del producto es del proveedor.	Configuración de escenarios; mantenimiento del proveedor.

Nota. Elaboración propia. Supuestos: un único usuario; volumen de 20 a 40 leads por mes; retención de datos de 12 meses; sin equipo de soporte. Cada importe consigna el plan concreto y la fecha en que se consultó la tarifa publicada, de modo que la estimación es auditable y su vigencia, acotada. La tarifa de infraestructura se publica en euros (Hetzner Online, s. f.): la celda consigna EUR 4,35 y su conversión a USD 4,70 al tipo de cambio de 1 EUR = 1,08 USD de esa misma fecha. Los totales a doce meses son el producto directo del costo mensual por doce, y el diferencial frente al CRM comercial es de USD 531,60 anuales; el \$7.1 desarrolla por qué el costo de mantenimiento lo neutraliza con pocas horas de trabajo al año. La estimación no incorpora el costo del desarrollo ni el del mantenimiento, consignados por separado por no ser monetizables sin una tarifa horaria declarada, ni el despliegue del frontend en Vercel, cuyo nivel gratuito cubre el volumen declarado. Es hipotética en un sentido preciso: la celda de infraestructura corresponde al despliegue en la nube que el punto 2 del Capítulo 8 propone —una cotización de lista de Hetzner Cloud, proveedor en el que el sistema nunca se desplegó— y no al entorno local en que el trabajo se desarrolló.

Tabla 29. Comparación funcional frente a las alternativas revisadas en §2.10

Dimensión	Solución construida	HubSpot / Pipedrive	Zapier / Make	Open source autoalojable (EspoCRM / Odoo Community)
Sistema de registro propio	Sí: PostgreSQL con esquema, enums e integridad referencial.	Sí, con el modelo de datos del proveedor.	No: operan sobre datos alojados en otras plataformas.	Sí: base propia autoalojada (EspoCRM sobre su motor; Odoo Community sobre PostgreSQL).
Reglas de scoring propias	Sí, definidas en código y auditables (Tabla 16).	Scoring configurable, en niveles pagos; reglas dentro del modelo del proveedor.	No incluyen scoring como capacidad nativa.	Parcial: EspoCRM calcula puntajes con su función de fórmulas, sin módulo de calificación dedicado; Odoo Community no trae scoring de leads en la edición libre.
Ciclo completo hasta la factura	Sí, incluida la generación del comprobante en PDF.	Requiere integración de terceros para facturar.	Requiere integración de terceros.	Parcial: Odoo Community incluye CRM y facturación básica de fábrica; EspoCRM no integra la emisión del comprobante con una pasarela de pago. Ninguna trae el encadenamiento a medida.
Control sobre la lógica de negocio	Total: el flujo es un artefacto versionado y exportable.	Limitado a las opciones de configuración expuestas.	Alto sobre la integración, nulo sobre el modelo de datos.	Alto: código abierto (AGPLv3 / LGPL-3.0) modificable, con motor de automatización propio; acotado por tratarse de software general y no de un flujo a medida.

Dimensión	Solución construida	HubSpot / Pipedrive	Zapier / Make	Open source autoalojable (EspoCRM / Odoo Community)
Custodia del dato	Parcial: sólo la orquestación es autoalojada; la persistencia y los canales son servicios de terceros (§2.11).	En la infraestructura del proveedor.	En la infraestructura del proveedor.	Propia, si se autoaloja: es la dimensión en la que esta categoría supera a los cuatro productos comerciales.
Esfuerzo de puesta en marcha	Alto: desarrollo, despliegue y mantenimiento propios.	Bajo: configuración asistida.	Bajo a medio.	Alto: instalación, despliegue y mantenimiento propios; Odoo agrega la configuración de una suite de gestión general.
Madurez operativa	Baja: producto mínimo viable sin operación real y con deudas de seguridad declaradas (Tabla 22).	Alta: producto comercial con soporte.	Alta.	Media a alta: productos con comunidad y trayectoria, sin soporte comercial garantizado en la edición libre; caracterización basada en documentación del proveedor.

Nota. Elaboración propia. Las siete dimensiones se fijaron antes de la comparación y se derivan del problema del §1.2 (§2.10). Las celdas de la solución construida se resuelven contra el artefacto: db/schema.sql, el flujo exportado y la Tabla 16. Las de HubSpot, Pipedrive, Zapier, Make, EspoCRM y Odoo describen lo que cada proveedor declara en su documentación y en sus páginas de precios, consultadas en las fechas que consigna cada celda; no se dispone de evaluación independiente por producto, y las fuentes de analista del §2.10 no suplen esa falta, porque su contenido es de acceso restringido y no sostiene ninguna celda de esta tabla. Tampoco lo hacen las reseñas agregadas de usuarios de Make y Pipedrive (Gartner, s. f.-a; Gartner, s. f.-b). La caracterización de la categoría sí se apoya en literatura arbitrada (Coronado y Iglesias, 2016; Neifer et al., 2021; Bucaioni et al., 2022; Domański et al., 2023).

La lectura conjunta de ambas tablas permite una respuesta honesta y acotada. Lo que la solución construida aporta frente a las alternativas es control sobre las reglas de negocio y un ciclo comercial cerrado sin integraciones de terceros; a cambio, obliga a asumir el desarrollo y el mantenimiento y a renunciar a la madurez operativa de un producto comercial. El argumento de costo es plausible por su estructura —ausencia de cuota por asiento y por ejecución— pero no está demostrado, y el de propiedad del dato se sostiene sólo parcialmente. Para un freelancer con necesidades estándar y sin interés en construir, configurar una herramienta comercial sigue siendo la opción razonable, o bien, con más trabajo de puesta en marcha, una suite de código abierto autoalojada. El valor de esta solución se concentra en escenarios de personalización alta y en el objetivo formativo del §1.4.

De las limitaciones que el §1.9 declara, una condiciona directamente esta comparación: durante el desarrollo el motor de automatización se ejecutó de forma autoalojada en un equipo local, sin URL pública estable, lo que impide que el navegador de un cliente externo alcance los webhooks en un entorno productivo. Su resolución —mediante un túnel o un alojamiento en la nube— es condición para el uso real y se plantea como trabajo

futuro. Las restantes —la configuración administrativa de la plataforma de autenticación, la ausencia de operación real y la falta de validación de la pasarela contra el servicio del proveedor— se enuncian en el §1.9 y en el §7.2 y no se reiteran aquí.

6.2 Limitaciones de las afirmaciones del trabajo

El Capítulo 7 enumera, como límites de lo acreditado, las limitaciones que afectan directamente a lo que el trabajo puede afirmar; este apartado las interpreta. Ninguna invalida el núcleo demostrado —la viabilidad técnica del ciclo automatizado— y todas se ubican dentro de lo que la literatura considera aceptable en un producto mínimo viable, que verifica comportamiento antes de medir impacto.

La reformulación del objetivo general no debilita el resultado, lo ajusta a la evidencia: el trabajo acredita que los flujos funcionan y que dieciséis de las dieciocho acciones del camino principal quedan delegadas. Ese dato es una medida de cobertura sobre el alcance construido —su denominador se deriva del propio artefacto (§1.7)— y no una medición de cuánto trabajo se ahorra. La arquitectura híbrida es una decisión de alcance coherente con ese estado de madurez: concentrar el autoalojamiento en la capa de orquestación, donde reside la lógica de negocio, y delegar la persistencia, el correo y la mensajería en servicios gestionados es la relación costo-control habitual en un prototipo. La preocupación por la custodia del dato que motiva construir está documentada como determinante de la adopción de estas plataformas (Neifer et al., 2021); el artefacto la satisface de forma parcial y así se declara.

El remanente de S1 fija con precisión la distancia entre el prototipo y un sistema apto para clientes reales: cinco de los trece webhooks siguen sin autenticar el origen de la invocación por diseño (§4.6). Eso no compromete la demostración de viabilidad técnica, pero sí impide el uso con clientes reales. De ahí una precaución sobre la difusión del artefacto: exponer el motor de orquestación en una URL estable no debe preceder al cierre de esa deuda, porque el repositorio público facilita localizar esas rutas (Capítulo 8, puntos 1 y 2). La distancia respecto de los dos marcos normativos aplicables, en cambio, no es una deuda técnica sino una condición del tipo de trabajo, en los términos del §2.7. Lo que no sería admisible es operar en producción en ese estado, y por eso el cumplimiento figura como condición previa y no como mejora opcional. El detalle de cada deuda —consecuencia, mitigación, estado y fecha— está en la Tabla 22 y en la Tabla 24, y no se reproduce aquí.

6.3 Incidente de seguridad: exposición y cierre de credenciales

Durante la configuración inicial del proyecto de Supabase, la `secret key` y la `service_role key` se compartieron por un canal de mensajería no seguro. Ninguna de las dos se utilizó en el código ni se almacenó en el repositorio: el frontend emplea únicamente la clave pública, y el motor de automatización guarda la `service_role key` en sus credenciales internas. Pese a eso, la exposición ocurrió. El 27 de agosto de 2026 se rotó la `service_role key` del proyecto y se deshabilitaron las claves JWT legacy que permanecían activas, cerrando ambos frentes de la exposición.

Mientras estuvo abierto no fue una línea de trabajo futuro sino un incidente de seguridad activo, y así se consignó aquí y en la Tabla 22 (S7). Su relevancia era directa: la `service_role key` evade por diseño la seguridad a nivel de fila y era la única credencial con permiso de escritura sobre las cinco tablas, de modo que mientras no se rotara la integridad de los datos no estaba garantizada. Con la rotación y la deshabilitación de las claves JWT legacy completadas el 27 de agosto de 2026, la integridad vuelve a depender exclusivamente de la custodia normal de credenciales descrita en el §4.6.

Capítulo 7: Conclusiones

Este trabajo se propuso responder si es técnicamente viable automatizar el ciclo de vida completo de la gestión de clientes de un freelancer mediante una arquitectura orientada a eventos de bajo código, de modo que las tareas hoy manuales queden delegadas en el sistema sin sacrificar la trazabilidad ni el control del profesional. La respuesta es afirmativa en cuanto a la viabilidad técnica y a la trazabilidad, y queda fuera del alcance de la evidencia recogida en cuanto a la magnitud del ahorro de trabajo. A continuación se responde punto por punto a los objetivos específicos y, después, al objetivo general y a la pregunta de investigación, reunidos en la Tabla 30.

7.1 Respuesta a los objetivos y a la pregunta de investigación

Tabla 30. Respuesta a los objetivos específicos, con su estado y su evidencia

Objetivo	Estado	Fundamento y evidencia
OE1	Cumplido	Cumplido. Se implementó y validó el formulario público con validación, integrado a n8n por webhook (E1–E4).
OE2	Cumplido con salvedad	Cumplido con salvedad. El mecanismo de normalización y scoring clasifica cada lead en HOT, WARM o COLD según cinco criterios ponderados (E1–E3). La salvedad: la validez del scoring es de diseño y no empírica.
OE3a	Cumplido	Cumplido. Se automatizó el ciclo hasta la emisión del comprobante —propuesta, aceptación con token de un solo uso y facturación en PDF— con notificaciones por correo y Telegram (E1b, E5, E6). La automatización es deliberadamente parcial en un punto: los términos comerciales los fija el profesional antes del envío (§4.3.2), porque automatizarlos habría significado comprometerlo con un importe que no eligió.
OE3b	Cumplido parcialmente	Cumplido parcialmente. Se integró la pasarela y se verificó la rama de cobro contra el contrato documentado de la API de MercadoPago, mediante un doble de prueba y un adaptador sobre una pasarela real en modo de prueba, en ambos casos sobre los nodos reales del flujo (E14). La pasarela no se validó contra el servicio del proveedor (§4.3.3 y Capítulo 8, punto 6).
OE4a	Cumplido	Cumplido. El modelo de datos relacional con seguridad a nivel de fila está verificado por inspección del esquema con <code>npm run test:rls</code> , que ejecuta treinta y tres casos rol por rol sobre un PostgreSQL desechable (Anexo C y Anexo F).
OE4b	Cumplido	Cumplido. El tablero autenticado expone métricas del mes, embudo y facturas pendientes. La actualización en tiempo real (E7) se midió el 3 de septiembre de 2026 sobre el commit c09008d con las veinte repeticiones que exige el método: media de 0,55 s, mediana de 0,54 s, p95 de 0,56 s y peor caso de 0,81 s, por debajo del umbral de 3 s (§5.2 y Tabla 26).
OE5	Cumplido	Cumplido. Los procesos de seguimiento, recordatorio de pago y reporte diario operan sin intervención. E11 y E12 quedaron verificados por completo el 2 de septiembre de 2026, sembradas las precondiciones que sus tramos finales exigían: la reejecución sobre lotes de más de un caso acreditó la transición a PERDIDO y el escalón RECORDATORIO, y reveló el defecto de propagación que la Tabla 22 registra como S9. E13 se ejecutó el 28 de agosto de 2026 contra el sistema real (ok = true, message_id = 594). Queda sin ejercitar el disparador por calendario del cuarto proceso, la reconciliación, cuya lógica se verifica en <code>tests/idempotencia.mjs</code> .
OE6a	Cumplido	Cumplido. Los controles de acceso se especificaron en tres capas —enrutamiento, componente de servidor y seguridad a nivel de fila (§4.2.3 y §4.6)— y se verifican con <code>npm run test:rls</code> , que aplica el esquema versionado sobre un PostgreSQL desechable y ejecuta treinta y tres casos rol por rol, nueve de ellos bajo n8n_writer. La deuda S4 se cerró el 31 de agosto de 2026 y el cierre se verificó también contra el proyecto de Supabase real.
OE6b	Cumplido	Cumplido en cuanto a medición, no a cierre. La superficie sin proteger se declara como nueve deudas (S1 a S9) en la Tabla 22, con su consecuencia, su mitigación y su estado, y se mide con el escenario E16 en lugar de enunciarse: seis de los trece webhooks atienden a un cliente sin credencial y sin navegador, y el alta de un lead prospera por la vía de los cinco que no admiten credencial de origen por diseño. Ninguna deuda impide la demostración de viabilidad técnica; el remanente de S1 sí impide el uso con clientes reales.

Nota. Elaboración propia. La matriz del Anexo E traza cada objetivo con la sección de diseño que lo implementa y el escenario que lo valida.

Respuesta al objetivo general. El objetivo general se alcanza: el artefacto existe, es público y ejecuta de extremo a extremo los flujos de captación, calificación, propuesta, aceptación, facturación, cobro y cierre, con cuatro procesos programados que operan sin intervención. El cobro se verificó contra un doble que reproduce el contrato documentado del proveedor y contra un adaptador sobre una pasarela real en modo de prueba, no contra el servicio de MercadoPago (OE3b). El control del profesional, primera condición de la pregunta de investigación, se materializa en dos puntos y no en uno. El primero es el tablero interno, desde el que cancela leads, resuelve pedidos de cambio, actualiza el estado del trabajo y cierra el proyecto. El segundo es la decisión sobre los términos comerciales de cada propuesta, que el sistema no toma por él. Esta segunda pieza se incorporó al detectarse que hasta entonces el importe de la propuesta y de la factura provenía del formulario que completaba el propio interesado (§4.3.2), de modo que esa condición carecía de respaldo real en el artefacto.

La trazabilidad, segundo término de la pregunta, se acredita por evidencia sobre el tramo que ejerce E15 y por construcción en el resto del ciclo. Cada lead tiene un estado explícito, marcas temporales de cada transición, un registro de auditoría en logs y un tablero que expone el embudo completo. E15 comprueba que eso alcanza para el tramo que ejerce: recorre una oportunidad por la captación y el envío de la propuesta y reconstruye después su historia leyendo únicamente la base, sin volver a preguntarle nada al motor de orquestación. Las diez comprobaciones de esa reconstrucción —que la línea de tiempo arranca en el alta, que es cronológicamente consistente, que el estado actual se explica por la última transición fechada y que los términos que fijó el profesional son recuperables— terminaron en verde.

Cierre de la proposición orientadora. Ninguno de los tres supuestos que el §1.3 fijó como condición de refutación se verificó al término del trabajo, pero no los tres con el mismo alcance probatorio. El primero —que algún flujo no pudiera completarse— no se verificó, porque los diecisiete escenarios muestran que todos completan su recorrido; queda descartado por construcción y no por contraste, según ya declara la Tabla 1. El segundo —que el sistema tomara por el profesional una decisión de negocio reservada a él— no se verifica en la versión final del artefacto, aunque sí llegó a verificarse transitoriamente durante el desarrollo. Lo que se sostiene es, por lo tanto, el artefacto final y no el intermedio. Es el único de los tres que se descarta sobre el ciclo completo, si bien por una decisión de arquitectura y no por un hallazgo empírico. El tercero —que el estado de una oportunidad no pudiera reconstruirse con lo que el sistema persiste— no se verifica sobre el tramo captación → propuesta, que es el que E15 ejerce leyendo únicamente la base. Sobre el resto

del ciclo la reconstrucción se sostiene por construcción, y su verificación íntegra queda pendiente. La proposición se sostiene, en consecuencia, con evidencia plena en el segundo supuesto, parcial en el tercero y por construcción en el primero.

Respuesta a la pregunta de investigación, y su límite. La condición «sin sacrificar la trazabilidad ni el control» se satisface. Lo que la evidencia recogida no acredita es la magnitud de la reducción de la carga administrativa: es una limitación de diseño de la validación y no un hallazgo negativo, porque los escenarios funcionales verifican comportamiento y no miden esfuerzo. Afirmar lo contrario sería exceder lo demostrado, y por eso el objetivo general fue reformulado en §1.5 en lugar de darse por cumplido en su versión original.

Queda sugerida, pero no demostrada, la mejora de indicadores comerciales reales, que dependería de una operación en producción. El menor costo operativo se estima a partir de precios de lista y no se verifica contra una operación propia: la Tabla 28 lo cifra en USD 56,40 a doce meses frente a USD 588,00 de un CRM comercial y USD 359,88 de una plataforma de integración. La salvedad merece aquí el lugar que la nota de esa tabla no le da: la comparación no incorpora el costo del desarrollo ni el del mantenimiento, que recaen sobre el profesional y que ninguna de las alternativas le cobra. La aritmética de esa omisión decide el argumento. El diferencial anual frente al CRM comercial es de USD 531,60, de modo que bastan unas veinte horas de mantenimiento al año para consumirlo a una tarifa de USD 26,58 la hora, o diez horas y media a USD 50. Veinte horas anuales son menos de dos por mes, un umbral que la operación de un sistema propio alcanza sin dificultad. La conclusión honesta es que la ventaja de costo directo no sobrevive a la incorporación del costo de oportunidad del tiempo del profesional, salvo que ese tiempo tenga para él un valor bajo o que el mantenimiento se considere parte del objetivo formativo del §1.4. El costo no es, entonces, la razón que sostiene la decisión de construir; lo son el control sobre las reglas de negocio y el cierre del ciclo (§6.1).

Cierre de la comparación con las alternativas. La pregunta que abrieron el §2.10 y el §2.11 —qué obtuvo el freelancer que no habría obtenido configurando una herramienta— se responde en el §6.1, con la Tabla 28 y la Tabla 29. El arco se cierra con una ventaja real y acotada, no con una superioridad general: control sobre las reglas de negocio y un ciclo comercial cerrado sin integraciones de terceros, a cambio de asumir el desarrollo y el mantenimiento.

7.2 Límites de lo acreditado

La respuesta anterior se sostiene dentro de límites que conviene enunciar sin atenuantes. Son los alcances de lo que la evidencia recogida permite afirmar; el Capítulo 6 interpreta por qué ninguno invalida el núcleo del trabajo.

El objetivo general se reformuló y no se midió. La carga administrativa recibió una definición operacional y su componente contable se midió como cobertura de acciones del ciclo —el sistema ejecuta dieciséis de las dieciocho del camino principal—, pero el tiempo y el esfuerzo que cada acción cuesta carecen de línea de base y de medición posterior. Ese indicador tiene, además, un límite propio: su denominador se deriva del propio artefacto y no de una observación independiente de un ciclo gestionado a mano, de modo que mide la cobertura del sistema construido sobre su propio alcance y no una reducción respecto de una línea de base externa. El trabajo acredita cuántas acciones se delegan, no cuánto trabajo ahorran.

La arquitectura es híbrida, no autoalojada. El sistema depende de cinco servicios gestionados por terceros —Supabase, Vercel, Gmail, Telegram y Notion— y sólo n8n corre de forma autoalojada, en un equipo local. La propiedad total del dato que se invoca como razón para construir no se verifica, por tanto, en el artefacto.

El sistema opera fuera de dos marcos normativos aplicables. El comprobante que emite no cumple el régimen de facturación electrónica argentino, y el tratamiento de datos de contacto no contempla la Ley 25.326 ni el régimen de transferencia internacional de su art. 12, según la Disposición 60-E/2016. Esto acota la validez del trabajo a un entorno de prueba con datos ficticios y excluye la operación con clientes reales hasta que se satisfagan.

A estos tres límites se suma el remanente de la deuda S1, ya detallado en la respuesta al OE6: cinco webhooks sin autenticación de origen que no afectan la demostración de viabilidad técnica, pero sí impiden el uso con clientes reales.

7.3 Aporte

Queda por precisar qué le queda disponible a un tercero que retome este trabajo. El aporte es acotado y de tres clases.

Aporte de artefacto. El flujo exportable (crm_postgres.json), el esquema versionado de la base (db/schema.sql), la suite de verificación (directorio tests/, con tests/verificar_afirmaciones.js) y el instrumento de conteo de acciones

(scripts/medir-carga-administrativa.mjs) son públicos, están versionados y se importan o ejecutan sin modificar. Un tercero obtiene un pipeline de referencia completo y reproducible para el ciclo captación–scoring–propuesta–aceptación–factura–cobro–cierre.

Aporte metodológico. El trabajo deja dos procedimientos transferibles. El primero es el patrón de verificar el documento contra el artefacto: tests/verificar_afirmaciones.js recalcula desde el repositorio los números que el texto afirma y añade diez comprobaciones de forma sobre el flujo exportado (§5.4). El segundo es la definición operacional de la carga administrativa como cobertura de acciones del ciclo (§1.7), con un script de medición que se invalida solo si el artefacto cambia.

Aporte de conocimiento acotado. La comparación de construir frente a configurar, abierta en el §2.10 y en el §2.11, queda cerrada en el §6.1 con una estimación de costo total de propiedad a doce meses (Tabla 28) y una comparación funcional de siete dimensiones (Tabla 29). El resultado es una respuesta con supuestos declarados para un caso concreto, no una recomendación general.

El alcance de este aporte no debe sobreestimarse: el trabajo no operó en producción, el argumento de costo es estructural pero no está medido y la custodia del dato se sostiene sólo en parte. Para un freelancer con necesidades estándar y sin interés en construir, configurar una herramienta comercial sigue siendo la opción razonable; el valor de esta solución se concentra en escenarios de personalización alta y en el objetivo formativo declarado (§1.4 y §6.1).

7.4 Aprendizajes del proceso

Un capítulo de conclusiones aporta algo que ningún otro puede aportar: qué aprendieron los autores. Se registran cuatro aprendizajes, formulados a partir de lo que efectivamente ocurrió durante el desarrollo.

El documento y el artefacto se desincronizan si no se los versiona juntos. La distancia más costosa de este trabajo no fue técnica sino documental. El diagrama de arquitectura quedó anclado a una versión con 71 nodos mientras el sistema llegaba a 128, y a los 161 nodos funcionales que declara la Tabla 33 al cierre. Los anexos quedaron declarativos mientras el repositorio acumulaba el material que debían contener, y el capítulo de seguridad describía menos de lo implementado y omitía lo pendiente. La lección es que la documentación de un artefacto vivo debe generarse

desde el artefacto, no en paralelo a él. El matiz costó descubrirlo: la contención mecánica que se construyó para eso verificaba cantidades, y las afirmaciones que efectivamente derivaron no eran cantidades sino descripciones —el orden de dos nodos, el alcance de una regla, la existencia de una transición—. Una verificación que sólo cuenta lo que es fácil contar da una seguridad que no tiene.

Una promesa en el objetivo general condiciona toda la cadena metodológica.

Incorporar «reduciendo la carga administrativa» al objetivo obligaba a una estrategia de validación capaz de medirlo, que nunca se diseñó. La decisión correcta —tomada tarde— fue ajustar el objetivo a lo que la validación podía acreditar. Si el trabajo se repitiera, las variables y su forma de medición se definirían antes de escribir el objetivo, no después.

La atomicidad se resolvió bien; la frontera transaccional, no. El análisis de concurrencia de §4.3.2 protege correctamente contra la doble facturación, y fue el problema técnico más difícil del ciclo. Sin embargo, concentrar el esfuerzo en ese punto ocultó que los cuatro pasos siguientes quedaban fuera de toda garantía. Una decisión de diseño correcta en un punto no cubre la operación completa: la pregunta útil no es «¿este nodo es atómico?» sino «¿qué ocurre si falla el paso siguiente?».

El método HTTP no es un detalle de forma. Resolver la confirmación de pago con un GET alcanzable desde el PDF del deudor parecía, durante el desarrollo, una simplificación admisible para un pago simulado. No lo era: el modelo de confianza que introduce se habría trasladado intacto a una pasarela real. La lección es que las decisiones etiquetadas como «provisorias» deben revisarse con el mismo criterio que las definitivas, porque rara vez se revisan después.

Capítulo 8: Trabajos Futuros y Recomendaciones

A partir de las limitaciones y de los riesgos identificados se proponen las siguientes líneas, ordenadas de mayor a menor prioridad. Las tres primeras son condición previa a cualquier uso con clientes reales; las restantes amplían el alcance del sistema o del estudio. Son de dos naturalezas distintas: los puntos 1, 2, 3, 5, 6, 7, 9 y 11 son trabajo técnico pendiente para la puesta en producción, mientras que los puntos 4, 8 y 10 son líneas de investigación que exigen datos de operación o de usuarios que este trabajo no recogió. Cada línea declara su relación de dependencia y el criterio que permitirá considerarla cerrada; la Tabla 31 reúne el mapa completo. Corresponde declarar una consecuencia de conjunto: al cierre de este trabajo el artefacto no está en condiciones de operar con clientes reales. Lo impiden los tres hechos que las líneas 1, 2 y 5 se proponen remover: los cinco puntos de entrada sin autenticación de origen, la ausencia de una URL pública estable para el motor de orquestación y la distancia respecto de los dos marcos normativos del §2.7. Ninguno de ellos es una mejora deseable sino una condición previa.

1. Cerrar las deudas de seguridad declaradas en la Tabla 22. La firma del webhook de MercadoPago, el token de pago-confirmado y la rotación del token de aceptación se implementaron el 31 de agosto de 2026. Queda evitar registrar la cadena de consulta completa en los logs de producción (S3) e implementar rate limiting o captcha en el formulario público y en los cuatro webhooks protegidos por accept_token, que es la mitigación que les corresponde en lugar de un secreto compartido (S1). Ninguna de las dos exige rediseñar el sistema: se resuelven dentro del flujo y del formulario, sin tocar el modelo de datos ni la arquitectura. Ninguna otra línea la condiciona, y el punto 2 no debe emprenderse hasta que esté cerrada. Se considerará cerrada cuando E16 informe que ningún webhook atiende una petición sin la credencial que le corresponde, salvo los cinco públicos por diseño, y los logs no registren la cadena de consulta. A esta línea se suma una tarea de validación. Lo verificado externamente y lo que el documento acredita ya coinciden: ambos son el commit d1abe08 (§5.6). Lo que queda pendiente no es repetir la verificación sobre otro estado, sino cubrir el límite que la propia constancia declara: una verificación con procedimiento ciego, y su extensión al estado posterior al corte cuando ese estado pase a ser el descrito.

2. Exponer el backend de automatización. Publicar la instancia de n8n mediante un túnel seguro o un alojamiento en la nube, con URL estable y HTTPS. Es condición previa a cualquier uso real y depende del punto 1: debe hacerse después de resolverlo, porque exponer

públicamente webhooks sin autenticación de origen agrava el riesgo. Se considerará cerrada cuando el sistema atienda un webhook desde el navegador de un cliente externo sobre una URL pública estable con certificado válido, y el punto 1 esté cerrado.

3. Completar la frontera transaccional y la reconciliación que queda. De los tres encargos que esta línea reunía, dos se resolvieron el 27 de agosto de 2026: la reconciliación idempotente de los leads ACEPTADO sin factura (§4.3.5) y la deduplicación por correo en la captación (§4.3.1). Queda el proceso que detecte, entre los leads en NUEVO, la propuesta calificada que el profesional dejó sin enviar, hoy sólo mitigada con la notificación inmediata y la visibilidad en el tablero. Y queda la frontera transaccional propiamente dicha: la reconciliación repara después de la falla en lugar de evitarla, deja una ventana de hasta media hora y no reenvía el comprobante. Cerrarla exige llevar la aceptación y la facturación a una única unidad de trabajo, o adoptar la saga con compensación explícita que el §4.3.2 diseña y de la que hoy sólo está implementada la compensación del paso de persistencia. Se considerará cerrada cuando ambas operaciones compartan una unidad de trabajo o una saga con compensación, sin ventana de inconsistencia, y un proceso desatendido reclame las propuestas calificadas sin enviar.

4. Medir la carga administrativa. Este punto tiene un requisito previo que no es técnico: por tratarse de una investigación con participantes humanos, no puede iniciarse sin el dictamen favorable de un comité de ética institucional. Corresponde presentar el protocolo del estudio, los instrumentos del Anexo J y el formulario de consentimiento del apartado J.3 ante el comité que la Facultad Regional Mendoza designe, o, en su defecto, ante el comité de ética en investigación de la Universidad Tecnológica Nacional. Su aprobación debe obtenerse antes de reclutar al primer participante.

Cumplido ese requisito, corresponde aplicar la definición operacional del §1.7 — número de acciones manuales por lead, del ingreso al cobro— sobre dos muestras comparables: leads gestionados manualmente y leads gestionados por el sistema. Un diseño posible parte de una muestra no probabilística de entre ocho y doce freelancers que presten servicios comparables a los del §1.1, reclutados por conveniencia. Cada participante gestiona un lote equivalente de leads ficticios con su flujo manual habitual y, en un segundo bloque, con el sistema; el orden se contrapesa entre participantes para atenuar el efecto de aprendizaje. El instrumento de registro es una planilla de acciones por lead con marca de tiempo de inicio y fin, y cada bloque se acota a dos semanas. Se controla como variable interviniente principal la familiaridad previa con herramientas de automatización.

El estudio exige consentimiento informado y por escrito de cada participante y un tratamiento de sus datos conforme a la Ley 25.326, en los mismos términos que el §2.7 reclama para los datos de los clientes, de modo que no puede iniciarse hasta que el punto 5 esté cerrado. Exige además que el instrumento de registro se acompañe de una definición operacional de «acción manual» compartida por los observadores y de una comprobación de acuerdo entre observadores sobre una submuestra, sin la cual el conteo no sería comparable. Es la medición que permitiría afirmar lo que la versión original del objetivo general prometía; los tres instrumentos se especifican, listos para usar, en el Anexo J. Conviene anticipar su alcance: una muestra no probabilística de ocho a doce participantes no sostiene inferencia sobre la población de freelancers, de modo que su resultado sería una medición situada y no una estimación con validez externa. Se considerará cerrada cuando exista un informe con las dos mediciones, el acuerdo entre observadores calculado y un análisis que compare ambas distribuciones.

5. Completar el cumplimiento normativo. Es condición previa a operar con clientes reales, no una mejora opcional, y también condición del punto 4, cuyo estudio trata datos personales de participantes (§2.7). En materia de datos personales corresponde incorporar al formulario el aviso del art. 6 de la Ley 25.326: finalidad, destinatarios, identidad del responsable, carácter de cada respuesta y derechos de acceso, rectificación y supresión. Ese aviso debe acompañarse de consentimiento expreso y separado para la cesión a terceros (art. 11) y para la transferencia internacional (art. 12), o bien de las cláusulas contractuales tipo de la Disposición 60-E/2016, suscritas con cada encargado domiciliado fuera de los países de protección adecuada. Falta además definir una política de retención que opere la supresión del art. 4.7 e inscribir la base ante la Agencia de Acceso a la Información Pública (art. 21). En materia fiscal, integrar el servicio web de ARCA para solicitar el CAE de cada comprobante y completar CUIT, punto de venta, tipo de comprobante y numeración correlativa. Se considerará cerrada, en datos personales, cuando el formulario cumpla el art. 6 y el régimen de consentimiento de los arts. 11 y 12, la base esté inscrita y una política de retención opere la supresión. En materia fiscal, cuando cada comprobante obtenga el CAE con CUIT, punto de venta y numeración correlativa.

6. Cerrar lo que queda de la integración de pagos. La pasarela real ya está integrada (§4.3.3) y su rama de cobro está verificada contra dos instrumentos: un doble de prueba y un adaptador contra una pasarela real en modo de prueba (E14, §5.3). Queda repetir esa verificación contra el servicio real de MercadoPago cuando la activación de credenciales de prueba del proveedor vuelva a estar disponible, que es lo único que ninguna de las dos

corridas actuales puede acreditar; es una dependencia externa y no de otra línea. Queda también evaluar el split de pagos vía la API de Marketplace cuando el sistema pase a operar con más de un profesional cobrando a cuentas distintas (§4.5). Se considerará cerrada cuando E14 se ejecute contra el servicio real de MercadoPago, y no contra un doble ni un adaptador, y verifique el cobro de una factura PENDIENTE y la idempotencia de la notificación repetida.

7. Ampliar la cobertura de pruebas automatizadas y la observabilidad. Extender las pruebas desde la prueba de humo existente hacia la verificación funcional de las reglas de negocio —aserciones sobre el score de la Tabla 16 y sobre las transiciones de la Tabla 20—, hacia los componentes del frontend y hacia los flujos de integración de extremo a extremo. Incorporar además métricas de ejecución aprovechando la tabla logs, que hoy existe y no alimenta ningún tablero de salud. Incorporar además el proceso que aplique las transiciones a VENCIDA y ANULADA de estado_pago, hoy previstas en el modelo y no ejecutadas por ningún componente (§4.8); al hacerlo habrá que rehacer el indicador de facturas vencidas de la Tabla 19, que hoy depende de que esa transición no ocurra. Queda también recalcular por cohorte de ingreso los indicadores «Conversión» y «Tiempo promedio de ciclo», hoy computados sobre el mes calendario con el desajuste que declara la nota de la Tabla 19. Y queda unificar la nomenclatura de las rutas propias, hoy repartida entre barra y guion salvo /mp/notificacion, cuyo formato impone el proveedor (§4.5). Queda corregir dos faltas de acentuación de la plantilla del correo de propuesta —«interes» e «Inversion», visibles en la Figura 13—, que llegan al cliente tal como se envían y comprometen la imagen profesional que el propio §1.1 invoca. Queda también completar en tests/scoring.js los dos pares de valores límite que el barrido cubre de un solo lado —seis y siete caracteres de teléfono, cincuenta y cincuenta y uno de descripción (§3.7.1)—. Y queda alinear el esquema con las dos reglas que hoy sólo impone el flujo: el CHECK de presupuesto, que admite el valor cero mientras R1 exige uno mayor, y el valor por omisión de servicio, que conserva desarrollo_web mientras la normalización asigna consultoria (nota de la Tabla 23).

El punto 11 se apoya en la observabilidad que esta línea incorpora, y el punto 3 la aprovecha para detectar la ventana de inconsistencia, aunque no la necesita para cerrarla. Se considerará cerrada cuando se cumplan cuatro condiciones. La suite debe verificar el score de la Tabla 16 y las transiciones de la Tabla 20, y cubrir el frontend y un recorrido de integración completo. Un tablero de salud debe consumir la tabla logs, y un proceso debe aplicar las transiciones a VENCIDA y ANULADA con el indicador de la Tabla 19 rehecho. Ambos indicadores deben computarse por cohorte de ingreso. Y las rutas propias deben usar

un separador único, con el esquema imponiendo por sí mismo las dos reglas que hoy sólo garantiza el flujo.

8. Calibrar el scoring con datos históricos. Registrar la conversión efectiva de los leads y ajustar umbrales y ponderaciones a partir de esos datos, transformando la validez de diseño del modelo en validez empírica. Revisar en esa instancia las tres decisiones del §4.3.1: la alineación entre el rango del control deslizante y los tramos de puntaje, la ponderación que reemplazó al sesgo de la opción «Otro» y los cuatro valores de servicio hoy inalcanzables desde el formulario. Depende de los datos de conversión que produce el punto 10. Se considerará cerrada cuando se registre la conversión efectiva de una cohorte de leads reales y los umbrales y ponderaciones de la Tabla 16 se reajusten con esos datos.

9. Completar la configuración de autenticación y roles. Finalizar la configuración administrativa de Supabase (URL de redirección y confirmación de correo) y ampliar el modelo de roles para escenarios con más de un profesional. No depende de ninguna otra línea. Se considerará cerrada cuando el alta de un usuario se complete sin confirmación manual —URL de redirección y correo ya configurados— y el modelo de roles admita más de un profesional con permisos diferenciados.

10. Realizar un relevamiento primario con usuarios reales. Como el punto 4, y por la misma razón, exige el dictamen favorable previo de un comité de ética institucional. Cumplido ese requisito, corresponde encuestar o entrevistar a una muestra de freelancers para validar empíricamente las prioridades del ciclo comercial y calibrar los umbrales del scoring (§3.3 y Capítulo 6). Corresponde también contrastar la comparación del §2.10 contra evaluaciones de analistas independientes o datos agregados de usuarios, de modo que las afirmaciones funcionales sobre las cuatro alternativas dejen de apoyarse únicamente en lo que cada proveedor declara de sí mismo. No depende de ninguna otra línea, y sus datos de conversión habilitan el punto 8. Se considerará cerrada cuando un informe recoja ese relevamiento y ese contraste.

11. Abstraer los canales de notificación y de generación documental. Introducir una capa que desacople el flujo de Gmail, Telegram, Notion y Gutenberg, de modo que incorporar o sustituir un canal no exija modificar cada punto de notificación (Capítulo 6). Se coordina con el punto 7, cuya observabilidad cubre los puntos de integración que esta capa encapsula. Se considerará cerrada cuando sustituir o agregar un canal se resuelva en un único módulo de adaptación, sin tocar los puntos de notificación del flujo.

Tabla 31. Dependencias entre las líneas de trabajo futuro

Línea	Depende de	Tipo de dependencia
1. Cerrar las deudas de seguridad de la Tabla 22	—	—
2. Exponer el backend de automatización	Línea 1	Técnica
3. Completar la frontera transaccional y la reconciliación	— (la línea 7 aporta la observabilidad que permite detectar la ventana de inconsistencia, no la capacidad de cerrarla)	—
4. Medir la carga administrativa	Línea 5 (el estudio trata datos personales) y dictamen favorable de un comité de ética institucional	Normativa y ética
5. Completar el cumplimiento normativo	—	—
6. Cerrar lo que queda de la integración de pagos	Reactivación de las credenciales de prueba de MercadoPago	Externa
7. Ampliar pruebas y observabilidad; unificar rutas	—	—
8. Calibrar el scoring con datos históricos	Línea 10 (datos de conversión reales)	De datos
9. Completar la configuración de autenticación y roles	—	—
10. Relevamiento primario con usuarios reales	Dictamen favorable de un comité de ética institucional	Ética
11. Abstractar los canales de notificación y documentales	Línea 7 (observabilidad de los puntos de integración)	Técnica

Nota. Elaboración propia. «Técnica» indica dependencia de una capacidad que otra línea construye; «de datos», de información que sólo la operación o un estudio generan; «normativa», de una condición legal previa; «ética», del dictamen favorable de un comité de ética institucional, exigible a toda línea que involucre participantes humanos; «externa», de un tercero ajeno al equipo. Cinco líneas no dependen de ninguna otra —la 1, la 3, la 5, la 7 y la 9— y cuatro de ellas habilitan a las restantes: la 1 a la 2, la 5 a la 4, la 7 a la 11 y la 10 a la 8, de modo que la 3 y la 9 son las únicas que no participan de ninguna dependencia en ninguno de los dos sentidos.

Referencias

- Agencia de Recaudación y Control Aduanero [ARCA]. (2026). *Facturación electrónica*. Recuperado el 27 de agosto de 2026, de <https://www.arca.gob.ar>
- American Psychological Association. (2020). *Publication manual of the American Psychological Association* (7.^a ed.). <https://doi.org/10.1037/0000165-000>
- Amir, A. R., y Atif, S. M. (2026). Evaluating workflow automation efficiency using n8n: A small-scale business case study [Preprint]. ScienceOpen. <https://doi.org/10.14293/PR2199.002882.v1>
- Bucaioni, A., Cicchetti, A., y Ciccozzi, F. (2022). Modelling in low-code development: A multi-vocal systematic review. *Software and Systems Modeling*, 21(5), 1959–1981. <https://doi.org/10.1007/s10270-021-00964-0>
- Césari, M. I., Stefanoni, M. E., Alfonso, E., y Castellano, C. (2026). Automatización no code con Make y n8n aplicada a PYMEs: arquitectura, costo total de propiedad y potencial educativo. En XXVIII Workshop de Investigadores en Ciencias de la Computación (WICC 2026) (pp. 61–65). Red de Universidades con Carreras en Informática. <http://sedici.unlp.edu.ar/handle/10915/196034>
- Coronado, M., y Iglesias, C. A. (2016). Task automation services: Automation for the masses. *IEEE Internet Computing*, 20(1), 52–58. <https://doi.org/10.1109/MIC.2015.73>
- Disposición 60-E/2016 de 2016 [Dirección Nacional de Protección de Datos Personales]. Aprueba las cláusulas contractuales tipo y las condiciones para la transferencia internacional de datos personales. 18 de noviembre de 2016. Boletín Oficial de la República Argentina. <https://servicios.infoleg.gob.ar/infolegInternet/anexos/265000-269999/267922/norma.htm>
- Docker Inc. (s. f.). *Docker Compose documentation*. Recuperado el 17 de agosto de 2026, de <https://docs.docker.com/compose/>
- Domański, R., Wojciechowski, H., Lewandowicz, J., y Hadaś, Ł. (2023). Digitalization of management processes in small and medium-sized enterprises—An overview of low-code and no-code platforms. *Applied Sciences*, 13(24), Article 13078. <https://doi.org/10.3390/app132413078>

- EspoCRM. (s. f.). *EspoCRM — Free open source CRM*. Recuperado el 30 de agosto de 2026, de <https://www.espocrm.com/>
- Fielding, R. T. (2000). *Architectural styles and the design of network-based software architectures* [Tesis doctoral, University of California, Irvine]. eScholarship, University of California. <https://escholarship.org/uc/item/9jd6t6zk>
- Fowler, M. (2002). *Patterns of enterprise application architecture*. Addison-Wesley.
- Garcia-Molina, H., y Salem, K. (1987). Sagas. En Proceedings of the 1987 ACM SIGMOD International Conference on Management of Data (pp. 249–259). Association for Computing Machinery. <https://doi.org/10.1145/38713.38742>
- Gartner. (s. f.-a). *Make reviews*. Gartner Peer Insights. Recuperado el 31 de agosto de 2026, de <https://www.gartner.com/reviews/market/integration-platform-as-a-service/vendor/make/product/make>
- Gartner. (s. f.-b). *Pipedrive reviews*. Gartner Peer Insights. Recuperado el 31 de agosto de 2026, de <https://www.gartner.com/reviews/product/pipedrive>
- Gotenberg. (s. f.). *Gotenberg documentation*. Recuperado el 17 de agosto de 2026, de <https://gotenberg.dev/docs>
- Hetzner Online. (s. f.). *Cloud — precios*. Recuperado el 17 de agosto de 2026, de <https://www.hetzner.com/cloud/>
- Hevner, A. R., March, S. T., Park, J., y Ram, S. (2004). Design science in information systems research. *MIS Quarterly*, 28(1), 75–105. <https://doi.org/10.2307/25148625>
- Hohpe, G., y Woolf, B. (2003). *Enterprise integration patterns: Designing, building, and deploying messaging solutions*. Addison-Wesley.
- HubSpot. (s. f.). *HubSpot pricing*. Recuperado el 17 de agosto de 2026, de <https://www.hubspot.com/pricing>
- International Organization for Standardization e International Electrotechnical Commission [ISO/IEC]. (2023). ISO/IEC 25010:2023. Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — Product quality model (2.^a ed.). ISO.
- Järvinen, J., y Taiminen, H. (2016). Harnessing marketing automation for B2B content marketing. *Industrial Marketing Management*, 54, 164–175. <https://doi.org/10.1016/j.indmarman.2015.07.002>

- Kässi, O., y Lehdonvirta, V. (2018). Online labour index: Measuring the online gig economy for policy and research. *Technological Forecasting and Social Change*, 137, 241–248. <https://doi.org/10.1016/j.techfore.2018.07.056>
- Kleppmann, M. (2017). Designing data-intensive applications: The big ideas behind reliable, scalable, and maintainable systems. O'Reilly Media.
- Kotler, P., y Armstrong, G. (2017). *Fundamentos de marketing* (13.^a ed.). Pearson.
- Lamanna, A. (2025). *A structured evaluation framework for low-code platform selection: A multi-criteria decision model for enterprise digital transformation* [Preprint]. arXiv. <https://doi.org/10.48550/arXiv.2510.18590>
- Ley 25.326 de 2000. Protección de los Datos Personales. 4 de octubre de 2000. Boletín Oficial de la República Argentina, 2 de noviembre de 2000. <https://servicios.infoleg.gob.ar/infolegInternet/anexos/60000-64999/64790/norma.htm>
- Make. (s. f.). *Make help center*. Recuperado el 17 de agosto de 2026, de <https://help.make.com/>
- MercadoPago. (s. f.-a). Checkout Pro: integración. Documentación para desarrolladores. Recuperado el 27 de agosto de 2026, de <https://www.mercadopago.com.ar/developers/es/docs/checkout-pro/landing>
- MercadoPago. (s. f.-b). API de pagos: obtener un pago. Documentación para desarrolladores. Recuperado el 27 de agosto de 2026, de https://www.mercadopago.com.ar/developers/es/reference/payments/_payments_id/get
- MercadoPago. (s. f.-c). Notificaciones webhooks: validación de la firma x-signature. Documentación para desarrolladores. Recuperado el 27 de agosto de 2026, de <https://www.mercadopago.com.ar/developers/es/docs/your-integrations/notifications/webhooks>
- Mero, J., Tarkiainen, A., y Tobon, J. (2020). Effectual and causal reasoning in the adoption of marketing automation. *Industrial Marketing Management*, 86, 212–222. <https://doi.org/10.1016/j.indmarman.2019.12.008>
- Moreira, S., Mamede, H. S., y Santos, A. (2025). Methodology for business process automation in SMEs: From requirements analysis to practical demonstration.

- Emerging Science Journal*, 9(3), 1561–1590. <https://doi.org/10.28991/ESJ-2025-09-03-022>
- Moynihan, D. P., Herd, P., y Harvey, H. (2015). Administrative burden: Learning, psychological, and compliance costs in citizen-state interactions. *Journal of Public Administration Research and Theory*, 25(1), 43–69. <https://doi.org/10.1093/jopart/muu009>
- Muhammad, S., Prybutok, V., y Sinha, V. (2025). Unlocking citizen developer potential: A systematic review and model for digital transformation. *Encyclopedia*, 5(1), Article 36. <https://doi.org/10.3390/encyclopedia5010036>
- n8n. (s. f.-a). n8n documentation. Recuperado el 17 de agosto de 2026, de <https://docs.n8n.io>
- n8n. (s. f.-b). Sustainable Use License. Recuperado el 2 de septiembre de 2026, de <https://docs.n8n.io/sustainable-use-license/>
- National Vulnerability Database [NVD]. (2025). *CVE-2025-29927 detail*. Recuperado el 30 de agosto de 2026, de <https://nvd.nist.gov/vuln/detail/CVE-2025-29927>
- Neifer, T., Lawo, D., Bossauer, P., y Gadatsch, A. (2021). Decoding IPaaS: Investigation of user requirements for Integration Platforms as a Service. En *Proceedings of the 18th International Conference on e-Business* (pp. 47–55). SCITEPRESS. <https://doi.org/10.5220/0010626000470055>
- Notion. (s. f.). *Notion API reference (versión 2022-06-28)*. Recuperado el 17 de agosto de 2026, de <https://developers.notion.com/reference/intro>
- Ntaliani, M., y Costopoulou, C. (2017). Reduction of administrative burdens for SMEs. *Administration & Society*, 49(8), 1143–1164. <https://doi.org/10.1177/0095399714558715>
- Odoo. (s. f.). *Odoo Enterprise vs Community — Editions comparison*. Recuperado el 30 de agosto de 2026, de <https://www.odoo.com/page/editions>
- Open Source Initiative. (2024). *The open source definition*. Recuperado el 17 de agosto de 2026, de <https://opensource.org/osd>
- Organización Internacional del Trabajo [OIT]. (2025). *Encuesta sobre trabajadores en plataformas digitales basadas en la web* [Informe]. OIT. https://www.ilo.org/sites/default/files/2025-04/Informe%20Plataformas%20Digitales_ESP_web.pdf

- OWASP Foundation. (2025). *OWASP Top 10:2025*. Recuperado el 17 de agosto de 2026, de <https://owasp.org/Top10/>
- Peppers, K., Tuunanen, T., Rothenberger, M. A., y Chatterjee, S. (2007). A design science research methodology for information systems research. *Journal of Management Information Systems*, 24(3), 45–77. <https://doi.org/10.2753/MIS0742-1222240302>
- Pipedrive. (s. f.). *Pipedrive pricing*. Recuperado el 17 de agosto de 2026, de <https://www.pipedrive.com/en/pricing>
- PostgreSQL Global Development Group. (2024). *PostgreSQL 15 documentation*. Recuperado el 17 de agosto de 2026, de <https://www.postgresql.org/docs/15/index.html>
- Pressman, R. S., y Maxim, B. R. (2020). *Software engineering: A practitioner's approach* (9.^a ed.). McGraw-Hill.
- Rahmati, A., Fernandes, E., Jung, J., y Prakash, A. (2017). *IFTTT vs. Zapier: A comparative study of trigger-action programming frameworks* [Preprint]. arXiv. <https://doi.org/10.48550/arXiv.1709.02788>
- Richardson, C. (2018). *Microservices patterns: With examples in Java*. Manning.
- Sommerville, I. (2016). *Software engineering* (10.^a ed.). Pearson.
- Stripe. (s. f.). *Testing: test card numbers and payment methods*. Recuperado el 31 de agosto de 2026, de <https://docs.stripe.com/testing>
- Supabase. (s. f.). *Supabase documentation*. Recuperado el 17 de agosto de 2026, de <https://supabase.com/docs>
- Vercel. (s. f.). *Next.js documentation*. Recuperado el 17 de agosto de 2026, de <https://nextjs.org/docs>
- Wu, M., Andreev, P., y Benyoucef, M. (2024). The state of lead scoring models and their impact on sales performance. *Information Technology and Management*, 25(1), 69–98. <https://doi.org/10.1007/s10799-023-00388-w>
- Zapier. (s. f.). *Zapier pricing*. Recuperado el 17 de agosto de 2026, de <https://zapier.com/pricing>

Anexos

Anexo A. Evidencias de la validación

Este anexo reúne la evidencia del funcionamiento del sistema: las capturas de la aplicación y de los flujos configurados en n8n, y la salida de la corrida completa de la suite de escenarios (Tabla 32). Cada captura se numera y se titula de forma individual, de modo que toda remisión del texto pueda resolverse sin ambigüedad. Los diagramas de arquitectura, del modelo de datos, de las máquinas de estados y de la secuencia de aceptación y facturación (Figuras 1 a 4) no se incluyen aquí sino en el cuerpo del Capítulo 4, junto al texto que los explica.

Figura 5. Formulario público de captación de leads, con el control deslizante de presupuesto y las seis opciones de servicio (§4.2.1, escenario E1).

Formulario de leads

CONSULTAS: 2026

NUEVA CONSULTA

Hablemos de tu próximo proyecto.

Completa el formulario y te respondemos en menos de 24 horas.

⌚ Respuesta en menos de 24 horas

I CONTACTO

NOMBRE *
María González

EMAIL *
tu@email.com

TELÉFONO
+54 11 1234-5678

II PROYECTO

SERVICIO *

☐ Desarrollo Web ☐ Diseño UX/UI

☐ Marketing Digital ☐ SEO / Posicionamiento

☐ Consultoría ☐ Otro

URGENCIA

☐ Baja – puedo esperar ☐ Media – próximas semanas

☐ Alta – lo antes posible

III PRESUPUESTO

ESTIMADO EN USD \$1.000

300 \$20.000

IV DESCRIPCIÓN

CONTAMOS TU PROYECTO *

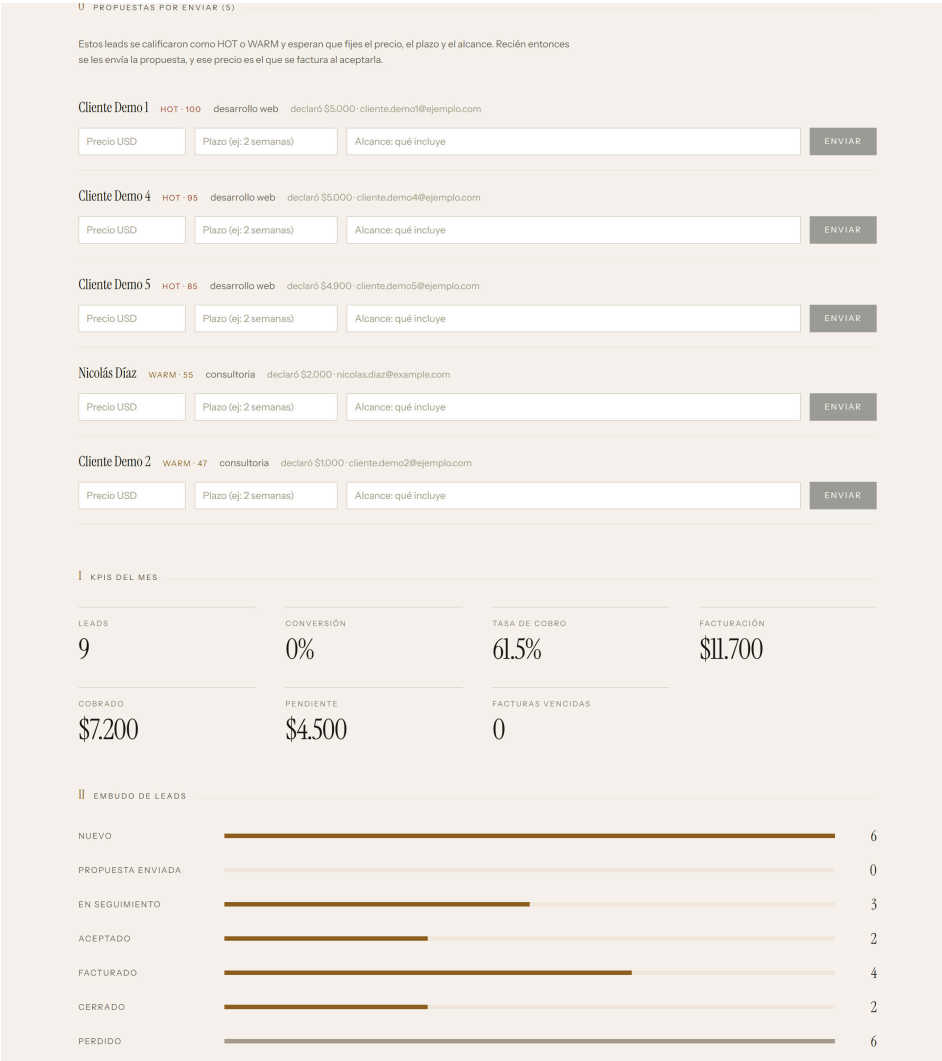
Describe brevemente en qué consiste tu proyecto...

0/2000 caracteres

ENVIAR CONSULTA

Nota. Captura propia de la aplicación desplegada.

Figura 6. Tablero de control interno: la bandeja de propuestas por enviar, desde la cual el profesional fija el precio, el plazo y el alcance antes del envío (§4.3.2); los indicadores del mes y el embudo por estado (§4.2.5, escenario E7).

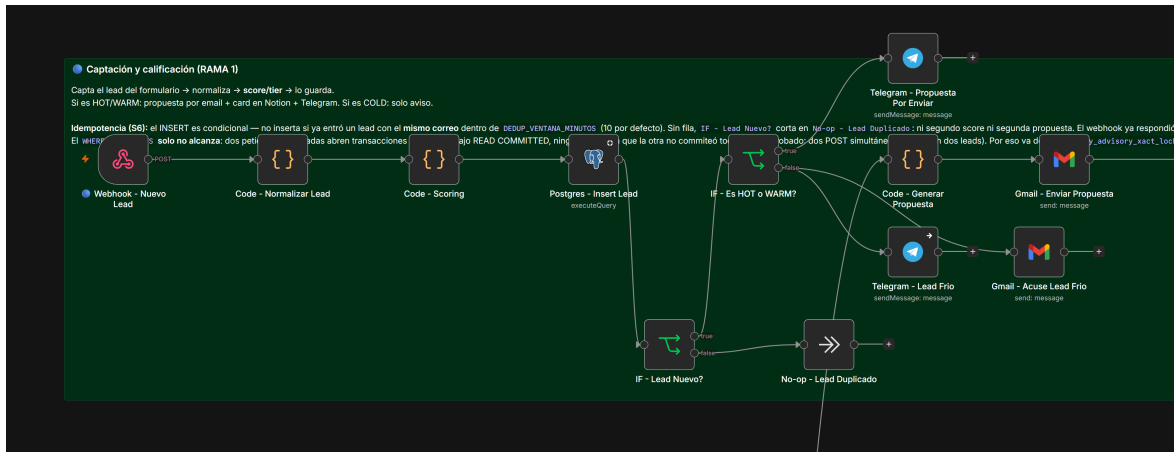


Nota. Captura propia, tomada el 29 de agosto de 2026 sobre el sistema en ejecución, con datos de prueba ficticios. Cuatro de las cinco filas de la bandeja siguen el patrón Cliente Demo N que fija la Tabla 14; la restante conserva un nombre distinto y el dominio example.com —reservado para ejemplos por el RFC 2606, de modo que no corresponde a una dirección real—, y su regeneración con los datos del protocolo queda pendiente. Las facturas pendientes, los trabajos activos y los pedidos de cambio continúan por debajo del corte de la imagen. El indicador «LEADS» (9) y la suma del embudo (23) no coinciden porque responden a alcances temporales distintos (Tabla 19).

Figura 7. Página pública de aceptación de propuesta, protegida por token de un solo uso (§4.2.6).

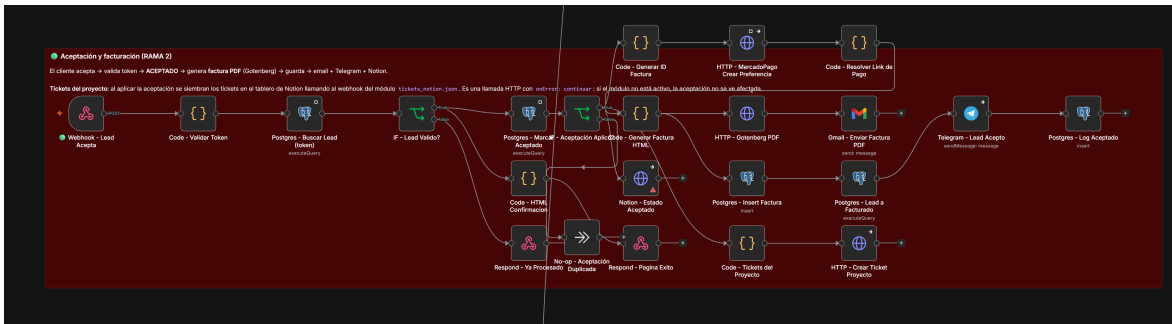
Nota. Captura propia.

Figura 8. Flujo de captación y calificación en n8n: webhook de nuevo lead, normalización, verificación de duplicados, scoring inserción ya calificada y el aviso de que la propuesta queda pendiente de que el profesional la envíe (§4.3.1).



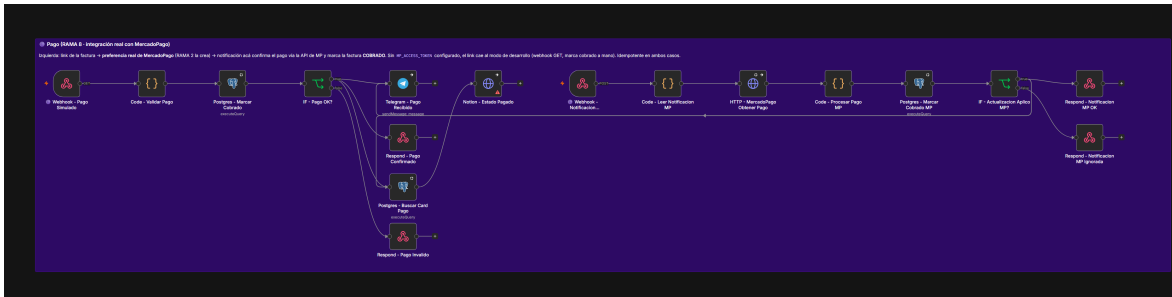
Nota. Captura propia del lienzo de n8n, tomada sobre el flujo exportado en el commit d1abe08 (§5.6) importado en una instancia limpia de la versión que declara la Tabla 12. El condicional IF - Es HOT o WARM? tiene exactamente dos salidas: la verdadera lleva a Telegram - Propuesta Por Enviar, que avisa al profesional de que la propuesta queda pendiente, y la falsa a la rama fría (Telegram - Lead Frio y Gmail - Acuse Lead Frio). Los nodos Code - Generar Propuesta y Gmail - Enviar Propuesta que aparecen a la derecha comparten esa zona del lienzo pero pertenecen al proceso /propuesta-enviar y los dispara IF - Terminos Aplicaron?, situado fuera del encuadre: no cuelgan del condicional de nivel (§4.3.2).

Figura 9. Flujo de aceptación y facturación en n8n: validación del token, actualización condicional atómica, generación de la preferencia de pago con MercadoPago, generación del PDF y persistencia de la factura (§4.3.2, §4.3.3).



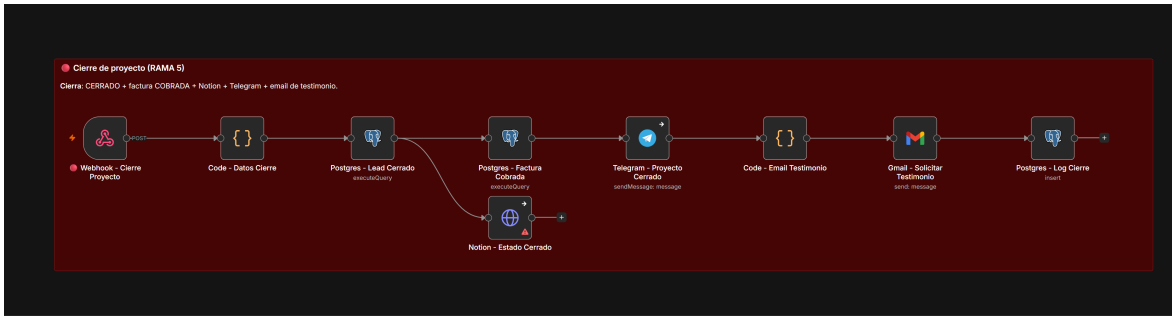
Nota. Captura propia del lienzo de n8n, tomada sobre el flujo exportado en el commit d1abe08 (§5.6) importado en una instancia limpia de la versión que declara la Tabla 12. La rama incorpora la integración con MercadoPago descrita en el §4.3.3.

Figura 10. Flujo de confirmación de pago en n8n: modo de desarrollo y confirmación real con MercadoPago (§4.3.3).



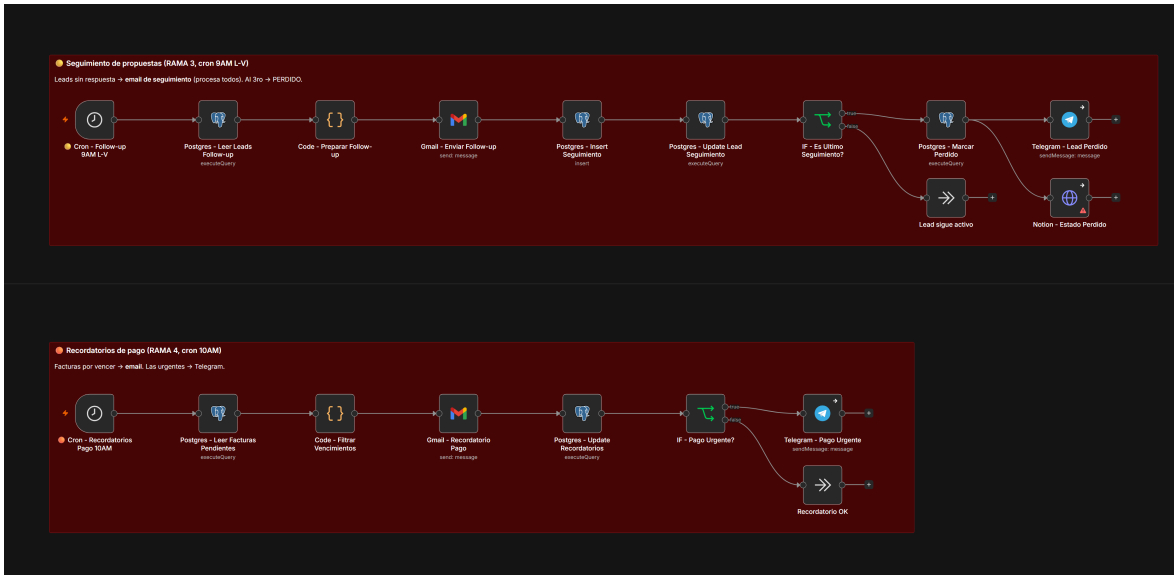
Nota. Captura propia del lienzo de n8n, tomada sobre el flujo exportado en el commit d1abe08 (§5.6) importado en una instancia limpia de la versión que declara la Tabla 12. Se ven las dos vías del §4.3.3: el webhook GET del modo de desarrollo y la notificación servidor a servidor de MercadoPago.

Figura 11. Flujo de cierre de proyecto y solicitud de testimonio en n8n (§4.3.4).



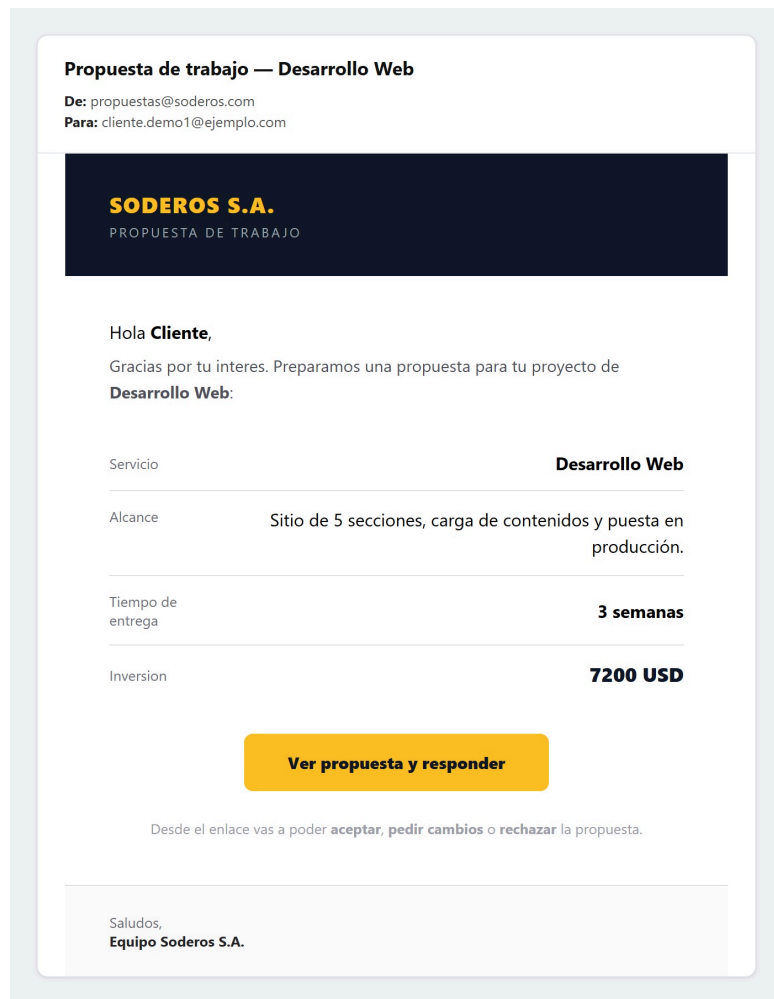
Nota. Captura propia del lienzo de n8n, tomada sobre el flujo exportado en el commit d1abe08 (§5.6) importado en una instancia limpia de la versión que declara la Tabla 12.

Figura 12. Procesos programados en n8n: seguimiento de propuestas y recordatorios de pago (§4.3.5, escenarios E11 y E12).



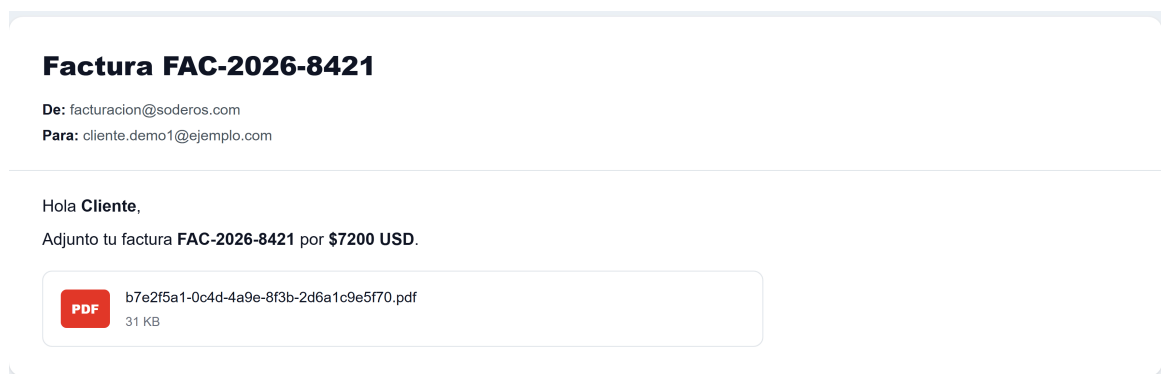
Nota. Captura propia del lienzo de n8n, tomada sobre el flujo exportado en el commit d1abe08 (§5.6) importado en una instancia limpia de la versión que declara la Tabla 12. Reúne las dos ramas a la misma escala, tomadas por separado y compuestas para suprimir el espacio vacío que las separa en el lienzo. Los otros dos procesos programados —el reporte diario de métricas y la reconciliación de facturas— no cuentan con captura del lienzo en este anexo: su verificación se reporta en el escenario E13 y en tests/idempotencia.mjs, respectivamente (§4.3.5).

Figura 13. Correo de propuesta generado por el sistema, con los términos fijados por el profesional (§4.3.2, escenario E5).



Nota. Esta figura se generó ejecutando el nodo real Code - Generar Propuesta con datos neutros conforme al protocolo de la Tabla 14 (Cliente Demo 1, dominio ejemplo.com) y los valores reales de E5 (precio 7200, plazo 3 semanas), renderizando el HTML resultante. Reproduce la salida del nodo y no la captura de un correo enviado, de modo que el contenido es exactamente el que el sistema produce.

Figura 14. Correo de envío de la factura con el comprobante adjunto en PDF (§4.3.2, escenario E5).



Nota. Esta figura se generó del mismo modo que la Figura 13, con los mismos datos neutros y los valores del escenario E5, ejecutando el mensaje real del nodo Gmail - Enviar Factura PDF; la identificación del comprobante y el importe son los mismos que consigna la Figura 15. El nombre del adjunto es un identificador que el nodo HTTP Request de n8n asigna automáticamente a la respuesta binaria de Gotenberg cuando no se declara un nombre de archivo explícito; se verificó contra el código del flujo que no deriva del accept_token del lead ni de ningún otro dato sensible.

Figura 15. Comprobante en PDF generado por el sistema (§4.3.2, escenario E5).

SODEROS S.A.
facturacion@soderos.com

FACTURA
N FAC-2026-8421
Emisión: 3/9/2026
Vence: 18/9/2026

Cliente: Cliente Demo 1
Email: cliente.demo1@ejemplo.com

Descripción	Cant.	Total
Desarrollo Web	1	\$7200
TOTAL		7200 USD

Pagar ahora

Datos de pago:
Alias: soderos.mp

Nota. Esta figura se generó ejecutando el nodo real Code - Generar Factura HTML del flujo, con la plantilla del emisor completa, los datos de prueba neutros que fija el protocolo de la Tabla 14 (Cliente Demo 1, dominio ejemplo.com) y los valores del escenario E5. El importe es el precio_propuesto de 7.200 USD que fija el profesional, y no el presupuesto de 5.000 que declaró el interesado (§4.3.2), de modo que el comprobante acredita la misma cifra que la Tabla 25 reporta para E5. El vencimiento cae a quince días de la emisión, conforme a la regla del nodo Code - Generar ID Factura, y la identificación del comprobante es la misma que muestra la Figura 14. El HTML resultante se convirtió a PDF con el mismo servicio Gotenberg que usa el sistema.

Figura 16. Clasificación de leads por score y nivel: ejemplos HOT, WARM y COLD registrados en la tabla leads (§4.3.1, escenarios E1 a E3).

lead_id	nombre	servicio	presupuesto	urgencia	score	tier
LD-1787973167150-531K	Cliente Demo 1	desarrollo_web	5000.00	alta	100	HOT
LD-1787973167434-COML	Cliente Demo 2	consultoria	1000.00	media	47	WARM
LD-1787973168547-37PS	Cliente Demo 3	soporte	300.00	baja	20	COLD

Nota. Esta figura se generó enviando tres leads reales con los datos de la Tabla 14 (Cliente Demo 1 a 3, dominio ejemplo.com) y las entradas exactas de E1, E2 y E3, y consultando la tabla leads resultante: los tres puntajes (100, 47, 20) coinciden con la Tabla 25.

Figura 17. Validación del formulario ante una entrada inválida (§4.2.1, escenario E4).

Nota. Captura propia. La captura documenta una sola de las tres condiciones inválidas que declara el escenario E4.

Figura 18. Los cuatro desenlaces de la página pública de aceptación de propuesta: (a) aceptación satisfactoria (§4.2.6, escenario E5); (b) enlace ya utilizado, rechazado sin reprocesar ni duplicar la factura (§4.3.2, escenario E6); (c) rechazo de la propuesta por el cliente, el lead pasa a PERDIDO (§4.3.7, escenario E8); (d) pedido de cambios del cliente, el lead vuelve a EN_SEGUIMIENTO (§4.3.7, escenario E9).



Nota. Captura propia, tomada el 3 de septiembre de 2026 sobre el frontend del commit d1abe08 (§5.6) ejecutado en local, con los datos neutros que fija la Tabla 14. Reúne los cuatro paneles a la misma escala, capturados por separado. Los mensajes son los que devuelven literalmente los nodos del flujo exportado en ese mismo commit: Respond - Pagina Exitosa en (a), Respond - Ya Procesado en (b), Respond - Rechazo OK en (c) y Respond - Modifica OK en (d); se sirvieron desde un doble local para no depender de la instancia de producción. El panel (b) es la segunda aceptación sobre el mismo enlace: la actualización condicional atómica del §4.3.2 no encuentra el estado previo que exige y el flujo deriva al nodo de operación nula, de modo que no se emite una segunda factura.

La Tabla 32 reproduce la corrida completa de tests/escenarios.mjs, ejecutada el 31 de agosto de 2026 a las 18:16:30 UTC sobre el commit d1abe08 y contra el sistema levantado. Es la corrida acreditante del estado del artefacto que este documento describe (§4.1), y es la misma corrida que el evaluador externo del Anexo K cotejó de forma independiente, de modo que lo verificado por un tercero y lo que el documento acredita coinciden. Su salida íntegra está archivada en docs/evidencia-validacion.md. Las doce filas terminan en OK y cubren los diez escenarios que enumera la Tabla 25 —E5 y E6 comparten una sola fila— más tres garantías sin escenario propio: la lectura de la propuesta con el token correcto, la idempotencia del pago y el rechazo de un token vencido, descritas en §4.3.2 y §4.3.3. Los escenarios E1, E2 y E3 verifican el desglose del puntaje contra la Tabla 16 y no sólo el total, y las cuatro entradas inválidas de la Tabla 25 se verifican en una sola corrida.

Tabla 32. Salida de la corrida completa de la suite de escenarios de validación, sobre el commit del corte documental

Escenario	Descripción	Resultado	Tiempo
E1	Un lead de presupuesto alto se califica HOT y queda esperando propuesta	OK	6009 ms
E1b	El profesional fija precio, plazo y alcance, y recién ahí sale la propuesta	OK	3413 ms
E2	Un lead de valor medio se califica WARM y también espera propuesta	OK	4119 ms
E3	Un lead de presupuesto bajo se califica COLD y NO recibe propuesta	OK	4553 ms
E4	Un lead inválido se rechaza sin persistirse y queda registrado	OK	4997 ms
—	La propuesta se lee sólo con el token correcto	OK	618 ms
E5/E6	Aceptación válida y reutilización del enlace: una sola factura	OK	9257 ms
—	El pago simulado es idempotente	OK	4540 ms
E8	El rechazo de la propuesta deja el lead en PERDIDO	OK	5095 ms
E9	El pedido de cambios vuelve el lead a EN_SEGUIMIENTO y guarda el mensaje	OK	4595 ms
E10	El estado del trabajo se actualiza desde el panel y se sincroniza	OK	709 ms
—	Un token vencido no permite aceptar la propuesta	OK	7538 ms

Nota. Elaboración propia a partir de la salida de tests/escenarios.mjs, archivada en docs/evidencia-validacion.md, que este mismo comando regenera en cada corrida. Esta matriz reemplaza a la del 30 de agosto de 2026 que reproducían las versiones previas: aquella corrida era anterior al cierre de S1, S3, S4 y S8, de modo que acreditaba un estado del artefacto que ya no era el vigente. El detalle completo, con cada aserción verificada, vive en ese mismo archivo. E7 no forma parte de esta suite: se mide por separado con scripts/medir-realtime.mjs (§5.2). Tampoco E11 a E14: los tres primeros corresponden a procesos programados que se disparan desde el editor de n8n y el último, a la rama de cobro (§5.3).

Anexo B. Flujos de trabajo de n8n

El backend de automatización del ciclo comercial se compone de un único flujo de 161 nodos funcionales —178 en total, incluidas 17 notas de documentación—, exportado en `workflow/crm_postgres.json`. La Tabla 33 resume su composición por tipo de nodo y la Tabla 34, la organización por proceso, con el disparador y los nodos principales de cada uno. El repositorio contiene además un segundo flujo independiente, `workflow/tickets_notion.json` (39 nodos, 3 webhooks y un proceso programado), que gestiona un tablero de tareas sobre Notion y alimenta la ruta `/dashboard/tickets`: no comparte nodos con el flujo del CRM, no interviene en ninguno de los escenarios de validación y queda fuera del alcance declarado en §1.9. Se lo consigna para que quien abra el repositorio sepa qué es y por qué no se lo describe; las pruebas automatizadas que recorren «ambos flujos» (§5.4) lo incluyen por estar versionado junto al principal.

Tabla 33. Composición del flujo por tipo de nodo

Tipo de nodo	Cantidad	Función
<code>n8n-nodes-base.postgres</code>	35	Consultas y actualizaciones sobre la base de datos.
<code>n8n-nodes-base.code</code>	31	Lógica de negocio en JavaScript (normalización, scoring, validación de token, generación de documentos, integración con MercadoPago).
<code>n8n-nodes-base.respondToWebhook</code>	20	Respuestas HTTP a los llamadores.
<code>n8n-nodes-base.stickyNote</code>	17	Notas de documentación en el lienzo (no funcionales).
<code>n8n-nodes-base.if</code>	16	Bifurcaciones condicionales.
<code>n8n-nodes-base.telegram</code>	15	Notificaciones al profesional.
<code>n8n-nodes-base.webhook</code>	13	Puntos de entrada (véase la Tabla 21).
<code>n8n-nodes-base.httpRequest</code>	13	Llamadas a Gotenberg, a la API de Notion y a la API de MercadoPago.
<code>n8n-nodes-base.gmail</code>	8	Envío de propuestas, facturas, recordatorios y testimonios.
<code>n8n-nodes-base.noOp</code>	5	Operación nula (ramas de aceptación duplicada, lead duplicado y factura ya emitida).
<code>n8n-nodes-base.scheduleTrigger</code>	4	Disparadores por calendario.
<code>n8n-nodes-base.errorTrigger</code>	1	Disparador de la rama de manejo de errores.
Total	178	161 funcionales + 17 notas de documentación.

Nota. Elaboración propia a partir del recuento de nodos de `workflow/crm_postgres.json`.

Tabla 34. Procesos del flujo, disparador y nodos principales

Proceso	Disparador	Nodos principales
Captación y calificación	POST /lead/nuevo	Normalizar Lead → Scoring → Insert Lead (INSERT condicional: descarta el correo repetido dentro de la ventana) → IF - Lead Nuevo? → IF por nivel. Rama HOT/WARM: Telegram de propuesta por enviar. Rama COLD: Telegram + acuse por correo. Rama duplicada: operación nula.
Envío de la propuesta	POST /propuesta-enviar	Leer Términos Propuesta → Guardar Términos (UPDATE condicional) → IF - Términos Aplicaron? → Generar Propuesta → Gmail → Estado Propuesta Enviada → Preparar Card Notion → Telegram.
Lectura de la propuesta	GET /lead-propuesta	Leer Query → consulta a <code>leads</code> → Armar Respuesta.
Aceptación y facturación	POST /lead-acepta	Validar Token → Buscar Lead → Marcar Aceptado (UPDATE condicional) → IF - Aceptación Aplicó? → Generar Factura HTML → Gotenberg → Gmail → Insert Factura → Lead a Facturado → Telegram y Notion. Rama alternativa: No-op - Aceptación Duplicada.
Rechazo de la propuesta	POST /lead-rechaza	Leer Rechazo → UPDATE a PERDIDO → Telegram → Notion.
Pedido de cambios	POST /lead-modifica	Leer Modifica → UPDATE a EN_SEGUIMIENTO con el mensaje → Telegram.
Resolución de pedidos de cambio	POST /cambio-aceptar · POST /cambio-rechazar	Leer Aceptar Cambio → Re-Propuesta → Gmail; o Leer Rechazar Cambio → Email No Cambios.
Pago (modo desarrollo)	GET /pago-confirmado	Validar Pago → Marcar Cobrado (solo desde PENDIENTE) → Telegram → respuesta al cliente. Activo únicamente cuando <code>MP_ACCESS_TOKEN</code> está vacío.
Confirmación de pago con MercadoPago	POST /mp/notificacion	Leer Notificación MP → Obtener Pago (API de MercadoPago) → Procesar Pago MP (solo si approved) → Marcar Cobrado MP (UPDATE idempotente) → Telegram y Notion.

Proceso	Disparador	Nodos principales
Cierre de proyecto	POST /proyecto-cerrado	Datos Cierre → Lead Cerrado → Factura Cobrada → Telegram → Email Testimonio.
Estado del trabajo	POST /trabajo-estado	Validar Estado Trabajo → UPDATE de estado_trabajo → sincronización con Notion.
Cancelación desde el tablero	POST /lead-cancelar	Leer Cancelar → UPDATE a PERDIDO (si no está CERRADO ni PERDIDO) → Telegram → Notion CANCELADO.
Seguimiento de propuestas	Cron 0 9 * * 1-5	Consulta de leads elegibles → Preparar Follow-up → Gmail → Insert seguimiento → UPDATE del lead.
Recordatorios de pago	Cron 0 10 * * *	Leer Facturas Pendientes → Filtrar Vencimientos (niveles RECORDATORIO, HOY, VENCIDA, URGENTE) → Gmail → Telegram en el caso urgente.
Reporte diario de métricas	Cron 59 23 * * *	Consulta de metrics mensuales → Telegram.
Reconciliación de facturas	Cron */30 * * * *	Leer Aceptado Sin Factura (ACEPTADO, sin factura y fuera del período de gracia, LIMIT 1) → Preparar Factura Reconciliada → preferencia de MercadoPago → Resolver Link → Insert Factura Reconciliada (INSERT condicional) → IF - Factura Reconciliada? → Lead a FACTURADO → Telegram; si ya existía, operación nula.
Manejo de errores	errorTrigger global	Formatear Error → Insert en logs → Telegram de error crítico.

Nota. Elaboración propia a partir de workflow/crm_postgres.json. El flujo exportado se incluye íntegro en el repositorio y puede importarse en una instancia de n8n sin modificaciones. En el lienzo, estos procesos se agrupan bajo notas adhesivas tituladas «RAMA n», rotulación visual que no se corresponde con un orden ni se usa en el cuerpo del texto: RAMA 1 es la captación y calificación; RAMA 2, la aceptación y facturación; RAMA 3, el seguimiento de propuestas; RAMA 4, los recordatorios de pago; RAMA 5, el cierre de proyecto; RAMA 6, el reporte diario; RAMA 7, el manejo de errores; RAMA 8, la confirmación de pago con MercadoPago; y RAMA 9, la reconciliación de facturas. Las figuras del Anexo A que muestran el lienzo conservan esa rotulación.

B.2 Clientes de Supabase

La conexión con Supabase se estructura en tres clientes especializados, en línea con el modelo de renderizado híbrido de Next.js, que la Tabla 35 detalla.

Tabla 35. Clientes de Supabase y su ámbito de ejecución

Archivo	Función
client.ts	cliente de navegador, para los componentes de cliente.
server.ts	cliente de servidor, que gestiona la sesión mediante cookies.
lib/supabase/middleware.ts	cliente para la comprobación de sesión en la capa de proxy, que se ejecuta en el entorno de Node.js y no en el borde (§4.2.3). El nombre de este archivo conserva la denominación histórica y no debe confundirse con el archivo de enrutamiento, que en Next.js 16 pasó a llamarse proxy.ts.

Nota. Elaboración propia.

Los tres comparten un criterio de degradación defensiva: si faltan las variables de entorno, devuelven un valor nulo en lugar de interrumpir la construcción de la aplicación, lo que permite desplegar sin configurar y evita caídas del despliegue (RNF3b).

B.3 Rutas del App Router

El frontend se organiza con el App Router de Next.js. La Tabla 36 enumera sus rutas.

Tabla 36. Rutas del App Router del frontend

Ruta	Función
/	Formulario público de captación.
/login, /register	Pantallas de autenticación.
/dashboard	Tablero de control protegido; exige sesión y rol admin.
/aceptar/[leadId]	Página pública de aceptación de propuesta, protegida por token de un solo uso.
/auth/confirm, /auth/signout	Manejadores de confirmación y de cierre de sesión.
/dashboard/tickets	Tablero del módulo de tickets, fuera del alcance declarado (§1.9, Anexo B).
/pago-exitoso, /pago-pendiente, /pago-fallido	Páginas de retorno del checkout de MercadoPago (back_urls de la preferencia de pago, §4.3.3).

Nota. Elaboración propia a partir de la estructura de directorios de src/app/ del repositorio declarado en §4.1.

Anexo C. Seguridad a nivel de fila (esquema de políticas)

Se reproduce a continuación el fragmento de `db/schema.sql` que implementa la seguridad a nivel de fila descrita en §4.6, de modo que el modelo de seguridad pueda auditarse sin recurrir al repositorio. El criterio es de denegación por omisión: habilitada la RLS y ausente una política aplicable, el acceso queda denegado.

```
-- 1) Habilitar RLS en todas las tablas. Con RLS activa y sin política
--   aplicable, el acceso queda denegado por defecto.
ALTER TABLE leads      ENABLE ROW LEVEL SECURITY;
ALTER TABLE facturas   ENABLE ROW LEVEL SECURITY;
ALTER TABLE seguimientos ENABLE ROW LEVEL SECURITY;
ALTER TABLE logs       ENABLE ROW LEVEL SECURITY;
ALTER TABLE profiles   ENABLE ROW LEVEL SECURITY;
-- 2) Lectura de las tablas que alimentan el tablero. No alcanza con estar
--   autenticado: se exige además profiles.role = 'admin'. Las políticas
--   de escritura de n8n se definen aparte, en la sección 5.1, sobre el
--   rol acotado n8n_writer (S4 de la Tabla 22).
DROP POLICY IF EXISTS leads_select_authenticated ON leads;
CREATE POLICY leads_select_authenticated ON leads
  FOR SELECT TO authenticated
  USING (EXISTS (SELECT 1 FROM profiles WHERE id = auth.uid() AND role = 'admin'));
DROP POLICY IF EXISTS facturas_select_authenticated ON facturas;
CREATE POLICY facturas_select_authenticated ON facturas
  FOR SELECT TO authenticated
  USING (EXISTS (SELECT 1 FROM profiles WHERE id = auth.uid() AND role = 'admin'));
DROP POLICY IF EXISTS seguimientos_select_authenticated ON seguimientos;
CREATE POLICY seguimientos_select_authenticated ON seguimientos
  FOR SELECT TO authenticated
  USING (EXISTS (SELECT 1 FROM profiles WHERE id = auth.uid() AND role = 'admin'));
-- 2.1) profiles: cada usuario solo puede leer su propia fila. Nadie puede
--   escribir su propio rol: solo la service_role o el trigger.
DROP POLICY IF EXISTS profiles_select_own ON profiles;
CREATE POLICY profiles_select_own ON profiles
  FOR SELECT TO authenticated USING (id = auth.uid());
-- 3) logs (auditoría y errores) no tiene política para authenticated:
--   service_role y n8n_writer (5.1) escriben y consultan; la
--   auditoría no se expone al tablero.
-- 4) Privilegios de tabla. La RLS filtra filas, pero el rol necesita
--   además el privilegio SELECT sobre el objeto.
GRANT SELECT ON leads, facturas, seguimientos, profiles TO authenticated;
GRANT SELECT ON metrics_mensuales, facturas_pendientes TO authenticated;
-- 5) La service_role se conserva solo para acceso administrativo (véase 5.1).
GRANT SELECT, INSERT, UPDATE, DELETE
  ON leads, facturas, seguimientos, logs, profiles TO service_role;
GRANT SELECT ON metrics_mensuales, facturas_pendientes TO service_role;
-- 5.1) Rol acotado para la conexión de n8n (cierra S4 de la Tabla 22).
--   Sin BYPASSRLS; sin acceso a profiles ni al esquema auth; sin DELETE.
CREATE ROLE n8n_writer NOLOGIN;
GRANT USAGE ON SCHEMA public TO n8n_writer;
GRANT SELECT, INSERT, UPDATE
  ON leads, facturas, seguimientos, logs TO n8n_writer;
GRANT SELECT ON metrics_mensuales, facturas_pendientes TO n8n_writer;
REVOKE ALL ON profiles FROM n8n_writer;
DROP POLICY IF EXISTS leads_rw_n8n_writer ON leads;
```

```

CREATE POLICY leads_rw_n8n_writer ON leads
FOR ALL TO n8n_writer USING (true) WITH CHECK (true);
-- (análogas sobre facturas, seguimientos y logs: facturas_rw_n8n_writer,
-- seguimientos_rw_n8n_writer, logs_rw_n8n_writer)
-- 6) El rol público (anon) no debe leer las tablas de negocio ni las vistas.
REVOKE ALL ON leads, facturas, seguimientos, logs, profiles FROM anon;
REVOKE ALL ON metrics_mensuales, facturas_pendientes FROM anon;
-- 7) Vistas declaradas con security_invoker: respetan las políticas de las
-- tablas subyacentes en lugar de las del propietario de la vista.
-- CREATE OR REPLACE VIEW metrics_mensuales WITH (security_invoker = true) AS ...
-- CREATE OR REPLACE VIEW facturas_pendientes WITH (security_invoker = true) AS ...

```

Obsérvese la consecuencia señalada en §4.6: hasta el 31 de agosto de 2026 no existían políticas de INSERT ni UPDATE, y toda la protección de escritura dependía por completo de la custodia de service_role. La sección 5.1 —agregada esa fecha— es la mitigación de S4: un rol sin BYPASSRLS, con políticas propias, acotado a las cuatro tablas y a las tres operaciones que el flujo realmente necesita.

Anexo D. Contrato de datos del formulario

El formulario envía al webhook de captación POST /lead/nuevo un objeto JSON con la estructura que se detalla a continuación. La Tabla 37 especifica el contrato de entrada campo por campo, con su tipo, obligatoriedad y restricciones; después se reproducen un ejemplo de petición y las tres respuestas posibles.

Tabla 37. Contrato de datos de entrada del webhook de captación

Campo	Tipo	Obligatorio	Restricción y normalización aplicada
nombre	string	Sí	Se recortan los espacios sobrantes. Debe tener al menos 2 caracteres.
email	string	Sí	Se convierte a minúsculas y se recorta. Debe contener «@».
telefono	string	No	Se recorta. Suma 5 puntos al score si supera los 6 caracteres.
servicio	string	Sí	Se traduce a la lista blanca de nueve valores (§4.3.1). Valor de respaldo, el mismo al que traduce la opción «Otro»: consultoria .
presupuesto	number	Sí	Se interpreta como entero. Debe ser mayor que cero. El formulario lo acota entre 100 y 20.000.
urgencia	string	Sí	Uno de alta, media o baja. Valor de respaldo: media .
descripcion	string	No	Se recorta y se trunca a 2000 caracteres. Suma 5 puntos si supera los 50.
fuelle	string	No	Metadato de origen. Valor por omisión: webhook .

Nota. Elaboración propia a partir de src/app/components/lead-form.tsx y del nodo Code - Normalizar Lead del flujo exportado.

Ejemplo de petición:

```

POST /webhook/lead/nuevo HTTP/1.1
Content-Type: application/json
{
  "nombre":      "Cliente Demo 1",
  "email":       "cliente.demo1@ejemplo.com",
  "telefono":    "+54 261 555 0100",
  "servicio":    "Desarrollo Web",
  "presupuesto": 5000,
  "urgencia":    "alta",
  "descripcion": "Necesito una tienda en línea con pasarela de pago y panel de gestión.",
  "fuente":      "formulario_web"
}

```

Respuesta satisfactoria (200):

```
{ "status": "ok", "lead_id": "LD-1785931200000-A1B2", "tier": "HOT" }
```

Respuesta ante datos inválidos: el nodo de normalización interrumpe el flujo con un error, no se inserta ninguna fila y la rama de manejo de errores registra el evento en la tabla logs y notifica por Telegram.

El webhook de aceptación POST /lead-acepta recibe { lead_id, token } y devuelve uno de tres desenlaces, que la página pública traduce a un mensaje para el cliente:

```
{ "status": "ok", "mensaje": "Propuesta aceptada." } // token válido, primera vez
{ "status": "duplicado", "mensaje": "El enlace ya fue procesado." } // 0 filas afectadas
{ "status": "invalido", "mensaje": "Enlace inválido o vencido." } // token inexistente
```

Sobre el segundo desenlace, véase la observación de §4.3.2: hoy agrupa tres situaciones distintas —enlace ya usado, oportunidad expirada y token rotado por un reenvío posterior de la propuesta (S3 de la Tabla 22)— que convendría distinguir.

Anexo E. Matriz de trazabilidad

Este anexo reúne la trazabilidad completa del trabajo en tres matrices. La Tabla 38 vincula el objetivo general, la pregunta de investigación y los nueve objetivos específicos con la sección de diseño que los implementa, el escenario que los valida y la conclusión que los retoma. La Tabla 39 traza los once requisitos funcionales y la Tabla 40, los nueve requisitos no funcionales, resultado de desagregar en dos requisitos atómicos los dos originales que reunían dos atributos de calidad bajo una misma etiqueta (§3.5). La columna de resultado remite al Capítulo 5 y al Capítulo 7, donde cada verificación se desarrolla.

Tabla 38. Matriz de trazabilidad objetivos → diseño → validación → conclusión

Elemento	Diseño	Validación	Resultado	Conclusión
Objetivo general (§1.5): viabilidad técnica de la automatización de extremo a extremo	§4.1 a §4.8 (sistema completo)	E1–E16 (E1b incluido)	Alcanzado en cuanto a viabilidad técnica y trazabilidad. Ninguno de los dos componentes de la carga administrativa se computa como resultado: el tiempo y el esfuerzo quedan fuera del alcance (§1.9), y la cobertura de acciones del §1.7 es un descriptor del alcance del artefacto —su denominador se deriva del propio artefacto— y no un resultado de la validación.	Cap. 7
Pregunta de investigación (§1.3): automatización sin sacrificar trazabilidad ni control	§4.1, §4.4, §4.6, §4.2.5, §4.3.7	E1–E16 (E1b incluido), E15 y Tabla 19	Respondida afirmativamente en viabilidad y trazabilidad. El control del profesional se materializa en el tablero interno y en la fijación de los términos comerciales; E15 reconstruye el ciclo de una oportunidad leyendo sólo lo que el sistema persiste. La magnitud del ahorro queda fuera de lo acreditado.	Cap. 7
OE1 — captación	§4.2.1	E1–E4	Verificado. E1–E4 en verde en la corrida automatizada que registra docs/evidencia-validacion.md: el alta válida persiste el lead y la	Cap. 7

Elemento	Diseño	Validación	Resultado	Conclusión
			inválida se rechaza sin escribir en leads.	
OE2 — normalización y scoring	§4.3.1 y Tabla 16	E1–E3	Verificado con salvedad. E1–E3 en verde, con el desglose del score coincidiendo con la Tabla 16 (100 HOT, 47 WARM, 20 COLD). La salvedad es la de la Tabla 30: la validez del scoring es de diseño y no empírica (§3.3, Capítulo 6).	Cap. 7
OE3a y OE3b — ciclo comercial y cobro	§4.3.2 a §4.3.4, §4.3.7, §4.5	E1b, E5, E6, E8, E9, E10	OE3a verificado: E1b, E5, E6, E8, E9 y E10 en verde; E1b acredita además que se factura el precio que fija el profesional y no el que declaró el interesado (§4.3.2). OE3b verificado parcialmente: el cobro se verifica contra el contrato documentado de la API (E14), no contra el servicio del proveedor (§4.3.3).	Cap. 7
OE4a — modelo de datos relacional y RLS	§4.2.5, §4.4, §4.6, Anexo C, Anexo F	npm run test:rls	Verificado por inspección del esquema. npm run test:rls aplica el esquema versionado sobre un PostgreSQL real y ejecuta 33 casos rol por rol, nueve bajo n8n_writer. S4 cerrada el 31-ago-2026 con verificación contra el proyecto real (§4.9).	Cap. 7
OE4b — tablero autenticado con métricas en tiempo real	§4.2.5, §4.4	E7	Verificado por medición de latencia (3-sep-2026, commit c09008d, posterior al corte documental del §5.6): 20 de 20 eventos, p95 de 0,56 s y peor caso de 0,81 s, por debajo del umbral de 3 s.	Cap. 7
OE5 — procesos programados	§4.3.5	E11, E12, E13	Verificado. E11 y E12 se dispararon el 23-ago-2026 y se reejecutaron sobre lotes de más de un caso el 2-sep-2026, lo que reveló y permitió corregir el defecto de propagación que la Tabla 22 registra como S9; E13 se ejecutó el 28-ago-2026 contra el sistema real (ok = true, message_id = 594). El cuarto proceso programado se verifica por separado en tests/idempotencia.mjs.	Cap. 7
OE6a — controles de acceso implementados	§4.2.3, §4.6, Anexo C	RNF1, RNF2, npm run test:rls	Verificado. npm run test:rls ejecuta 33 casos rol por rol: anon sin acceso, autenticado sin rol admin con 0 filas, admin con lectura, logs cerrada incluso para admin y escritura denegada, más nueve casos bajo n8n_writer. S4 cerrada el 31-ago-2026 (§4.9).	Cap. 7
OE6b — superficie sin proteger, medida y declarada	§4.6, §4.9, Tabla 22, Anexo C	E16	Cumplido en cuanto a medición, no a cierre. E16 mide el reparto de la Tabla 22 —6 webhooks atienden sin credencial, 6 del panel responden 403— y falla si ese reparto cambia en cualquiera de las dos direcciones. El remanente de S1 impide el uso con clientes reales.	Cap. 7

Nota. Elaboración propia.

Tabla 39. Trazabilidad de los requisitos funcionales

Requisito funcional	Diseño	Escenario que lo verifica	Resultado
RF1 — formulario público con validación	§4.2.1	E1, E4	Verificado (E1, E4).
RF2 — normalización y rechazo de datos inválidos	§4.3.1	E1, E4	Verificado (E1, E4): la entrada inválida no persiste y queda en logs.
RF3 — cálculo de score y clasificación en niveles	§4.3.1, Tabla 16	E1, E2, E3	Verificado (E1–E3): 100 HOT, 47 WARM y 20 COLD, con el desglose de la Tabla 16.
RF4 — envío de propuesta a HOT y WARM	§4.3.1, §4.3.2	E1b (y E1, E2, E3 como casos negativos)	Verificado (E1b): el lead HOT recibe la propuesta una vez que el profesional fija los términos. E1 y E2 acreditan el caso negativo —el lead calificado no la recibe antes de esa decisión— y E3, que el COLD no la recibe en ningún caso.
RF5 — aceptación con token de un solo uso	§4.3.2, §4.2.6	E5, E6	Verificado (E5, E6): cuatro invocaciones del mismo enlace dejan una sola factura.
RF6 — generación y envío de la factura en PDF	§4.3.2	E5	Verificado (E5): se emite la factura con vencimiento a 15 días de la emisión.
RF7 — registro del cobro y cierre del proyecto	§4.3.3, §4.3.4	E5 y verificación manual del cierre	Verificado parcialmente. El cobro se acredita por las dos vías que el sistema contempla: el modo de desarrollo (E5) y la rama real de MercadoPago, ejercitada en E14 contra un doble del contrato documentado y un adaptador sobre una pasarela real. Queda sin acreditar la validación contra el servicio del proveedor (Capítulo 8, punto 6). El cierre de proyecto sigue sin escenario propio: se verificó manualmente.
RF8 — seguimientos automáticos de propuestas sin respuesta	§4.3.5	E11	Verificado (E11), con reserva sobre su historia. El correo, la fila de seguimiento y el incremento del contador se confirmaron desde la primera corrida; la transición a PERDIDO del tercer intento estuvo fallando hasta el 2-sep-2026, porque el 23-ago se aplicó una corrección que no la resolvía y no se reejecutó el escenario. Establecida la causa real (§5.2), la reejecución sobre un lote de dos leads dejó las trece comprobaciones en verde.
RF9 — recordatorios de facturas por vencer o vencidas	§4.3.5	E12	Verificado (E12) en sus dos escalones. El nivel URGENTE se acreditó el 27-ago-2026; el nivel RECORDATORIO, el 2-sep-2026, sembrada la factura a dos días del vencimiento que nunca había existido. Esa corrida obligó a corregir el defecto

Requisito funcional	Diseño	Escenario que lo verifica	Resultado
			de propagación que hacía que la rama urgente notificara una sola factura por lote (§5.2).
RF10 — notificaciones por Telegram y reporte diario	§4.3.5, §4.3.6	E13	Verificado (E13): el reporte diario se ejecutó el 28-ago-2026 contra el sistema real y la API de Telegram confirmó la entrega (ok = true, message_id = 594), con indicadores coincidentes con la vista metrics_mensuales. Antes, al ejercitar E12 se detectó y corrigió que las notificaciones de Telegram estaban rotas.
RF11 — tablero autenticado con métricas en tiempo real	§4.2.5, §4.4	E7	Verificado: el tablero lee y muestra los siete conjuntos de datos, y la actualización en vivo (E7) se comprobó sobre veinte repeticiones con p95 de 0,56 s y peor caso de 0,81 s (véase OE4b).

Nota. Elaboración propia.

Tabla 40. Trazabilidad de los requisitos no funcionales

Requisito no funcional	Diseño	Método de verificación (Tabla 11)	Resultado
RNF1 — acceso restringido y RLS	§4.2.3, §4.6, Anexo C	Tres intentos de lectura con distinto rol; intento de escritura con clave pública.	Verificado. npm run test:rls ejecuta 33 casos rol por rol sobre un PostgreSQL real: anon sin acceso, autenticado sin rol admin con 0 filas, admin con lectura, logs cerrada incluso para admin y escritura denegada desde el navegador, más nueve casos bajo n8n_writer (S4 de la Tabla 22).
RNF2 — token de un solo uso	§4.3.2	Dos invocaciones secuenciales y dos concurrentes (E6).	Verificado (E6): cuatro invocaciones — dos concurrentes y dos consecutivas — del mismo enlace producen exactamente una fila en facturas.
RNF3a — detención controlada y registro del error	§4.3.1, §4.3.6	Cuatro entradas inválidas (E4).	Verificado (E4): el flujo se detiene, no persiste el lead y deja el registro en logs.
RNF3b — degradación defensiva del frontend	§4.2.4, Anexo B.2	Construcción del frontend sin variables de entorno definidas.	Verificado: los tres clientes de Supabase devuelven un valor nulo en lugar de interrumpir la construcción, y el build del frontend sin variables de entorno termina con código de salida 0.
RNF4 — lógica concentrada en el orquestador	§4.1, §4.5	Inspección estática del directorio src/.	Verificado por inspección estática de src/: 0

Requisito no funcional	Diseño	Método de verificación (Tabla 11)	Resultado
			llamadas a insert, update, upsert o delete sobre las tablas de negocio; las únicas consultas son de lectura sobre leads, metrics_mensuales, facturas_pendientes y perfiles.
RNF5a — interacción sin desborde horizontal	§4.2.1	Revisión en cuatro anchos de pantalla —incluido el extremo superior de 1920 px que fija el requisito—.	Verificado el 30-ago-2026: la interfaz no desborda horizontalmente en ninguno de los cuatro anchos revisados, incluido el extremo de 1920 px que fija el requisito.
RNF5b — mensajes de error de autenticación en español	§4.2.2	Recuento automatizado de los códigos de error traducidos.	Verificado. tests/auth_errores.js declara los veintidós códigos que la plataforma puede devolver en signUp, signInWithPassword y verifyOtp, y falla si alguno no tiene mensaje en español. Su primera ejecución acreditó que el requisito no se cumplía —once de veintidós, entre los faltantes los del enlace de confirmación vencido—; incorporadas las traducciones, la cobertura es 22/22.
RNF6 — actualidad del dato en el tablero	§4.2.5, §4.7	20 escrituras consecutivas midiendo la latencia hasta la recepción del evento (E7).	Verificado. La medición (scripts/medir-realtime.mjs) recibió los veinte eventos que el método exige, con p95 de 0,56 s y peor caso de 0,81 s, por debajo del umbral de 3 s. El script termina con código distinto de cero si el umbral no se cumple.
RNF7 — puesta en marcha reproducible	§4.1, Anexo G	Levantar el entorno desde cero con docker compose up -d.	Verificado: el entorno de orquestación y el conversor de documentos se levantan con docker compose up -d y los webhooks quedan operativos sin editar archivos versionados.

Nota. Elaboración propia.

Anexo F. Esquema de la base de datos (DDL)

Se reproduce el fragmento del archivo db/schema.sql del repositorio declarado en §4.1 que define la extensión pgcrypto, los siete tipos enumerados, las dos tablas centrales del modelo —leads y facturas— y las columnas que se incorporaron después por migración, entre ellas las que soportan los términos que fija el profesional y la integración de pagos. Ese archivo es la fuente de verdad del modelo descrito en §4.4 y representado en la Figura 3, y es idempotente: puede ejecutarse varias veces sobre la misma base sin efectos colaterales. Las tres tablas restantes —seguimientos, logs y perfiles, de estructura sencilla y descrita en §4.4—, las funciones, los disparadores, las vistas, los once índices y la configuración de tiempo real se consultan allí; las políticas de seguridad a nivel de fila se reproducen en el Anexo C.

```
CREATE EXTENSION IF NOT EXISTS pgcrypto;

-- -----
-- Tipos enumerados
-- -----

CREATE TYPE urgencia_tipo AS ENUM ('alta','media','baja');
CREATE TYPE servicio_tipo AS ENUM

('desarrollo_web','ecommerce','app_movil','automatizacion','diseno_ui','consultoria','soporte','marketing','se
o');
CREATE TYPE tier_tipo AS ENUM ('HOT','WARM','COLD');
CREATE TYPE lead_estado AS ENUM
('NUEVO','PROPUESTA_ENVIADA','EN_SEGUIMIENTO','ACEPTADO','FACTURADO','CERRADO','PERDIDO');
CREATE TYPE pago_estado AS ENUM ('PENDIENTE','COBRADO','VENCIDA','ANULADA');
CREATE TYPE log_nivel AS ENUM
('INFO','RECORDATORIO','HOY','VENCIDA','URGENTE','WARN','ERROR');
-- Estado del TRABAJO (ejecución del proyecto, distinto del estado del lead/venta)
CREATE TYPE trabajo_estado AS ENUM ('PENDIENTE','EN_PROGRESO','EN_REVISION','ENTREGADO');
-- -----
-- Tablas
-- -----

CREATE TABLE IF NOT EXISTS leads (
  id                BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  lead_id           TEXT UNIQUE NOT NULL,
  nombre            TEXT NOT NULL,
  email             TEXT NOT NULL CHECK (position('@' in email) > 1),
  telefono          TEXT,
  presupuesto       NUMERIC(12,2) NOT NULL DEFAULT 0 CHECK (presupuesto >= 0),
  urgencia          urgencia_tipo NOT NULL DEFAULT 'media',
  servicio          servicio_tipo NOT NULL DEFAULT 'desarrollo_web',
  descripcion       TEXT,
  fuente            TEXT DEFAULT 'webhook',
  estado            lead_estado NOT NULL DEFAULT 'NUEVO',
  estado_trabajo    trabajo_estado NOT NULL DEFAULT 'PENDIENTE',
  score             INT NOT NULL DEFAULT 0,
  tier               tier_tipo,
  seguimientos      INT NOT NULL DEFAULT 0,
  operador_asignado TEXT,
  notas            TEXT,
```

```

card_id            TEXT,
accept_token       UUID NOT NULL DEFAULT gen_random_uuid(),
token_expira_en    TIMESTAMPTZ,
fecha_ingreso      TIMESTAMPTZ NOT NULL DEFAULT now(),
fecha_propuesta    TIMESTAMPTZ,
fecha_ultimo_seguimiento TIMESTAMPTZ,
fecha_aceptacion   TIMESTAMPTZ,
fecha_cierre       TIMESTAMPTZ,
dias_ciclo_completo INT,
creado_en          TIMESTAMPTZ NOT NULL DEFAULT now(),
actualizado_en     TIMESTAMPTZ NOT NULL DEFAULT now()
);
CREATE TABLE IF NOT EXISTS facturas (
    id              BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    factura_id      TEXT UNIQUE NOT NULL,
    lead_id         TEXT NOT NULL REFERENCES leads(lead_id) ON DELETE CASCADE,
    cliente         TEXT NOT NULL,
    email           TEXT NOT NULL,
    servicio        servicio_tipo,
    monto           NUMERIC(12,2) NOT NULL CHECK (monto >= 0),
    moneda          TEXT NOT NULL DEFAULT 'USD',
    estado_pago     pago_estado NOT NULL DEFAULT 'PENDIENTE',
    recordatorios_enviados INT NOT NULL DEFAULT 0,
    fecha_emision   TIMESTAMPTZ NOT NULL DEFAULT now(),
    fecha_vencimiento TIMESTAMPTZ NOT NULL,
    fecha_cobro     TIMESTAMPTZ,
    mp_preference_id TEXT,
    mp_payment_id   TEXT,
    comision_plataforma NUMERIC(12,2) NOT NULL DEFAULT 0,
    creado_en       TIMESTAMPTZ NOT NULL DEFAULT now()
);
-- Columna de estado del trabajo (para bases creadas antes de agregarla).
ALTER TABLE leads ADD COLUMN IF NOT EXISTS estado_trabajo trabajo_estado NOT NULL DEFAULT 'PENDIENTE';
ALTER TABLE leads ADD COLUMN IF NOT EXISTS token_expira_en TIMESTAMPTZ;
-- Servicios marketing/seo que ofrece el formulario (para bases ya creadas).
ALTER TYPE servicio_tipo ADD VALUE IF NOT EXISTS 'marketing';
ALTER TYPE servicio_tipo ADD VALUE IF NOT EXISTS 'seo';
ALTER TABLE facturas ADD CONSTRAINT chk_facturas_fechas
    CHECK (fecha_vencimiento >= fecha_emision) NOT VALID;

```

-- Cobro real con MercadoPago (para bases ya creadas). Véase §4.3.3.

```

ALTER TABLE facturas ADD COLUMN IF NOT EXISTS mp_preference_id TEXT;
ALTER TABLE facturas ADD COLUMN IF NOT EXISTS mp_payment_id TEXT;
ALTER TABLE facturas ADD COLUMN IF NOT EXISTS comision_plataforma NUMERIC(12,2) NOT NULL DEFAULT 0;
ALTER TABLE facturas ADD CONSTRAINT chk_facturas_comision
    CHECK (comision_plataforma >= 0) NOT VALID;
ALTER TABLE facturas ADD COLUMN IF NOT EXISTS pay_url TEXT;
ALTER TABLE facturas ADD COLUMN IF NOT EXISTS pago_token UUID NOT NULL DEFAULT gen_random_uuid();
ALTER TABLE leads ADD COLUMN IF NOT EXISTS precio_propuesto NUMERIC(12,2);
ALTER TABLE leads ADD COLUMN IF NOT EXISTS plazo_propuesto TEXT;
ALTER TABLE leads ADD COLUMN IF NOT EXISTS alcance_propuesto TEXT;
ALTER TABLE leads ADD CONSTRAINT chk_leads_precio_propuesto
    CHECK (precio_propuesto IS NULL OR precio_propuesto > 0) NOT VALID;

```

Nota. Se reproduce el fragmento de db/schema.sql correspondiente a la extensión pgcrypto, los tipos enumerados, las tablas leads y facturas y las columnas incorporadas por migración, que es la parte del esquema que sostiene el modelo de datos descrito en §4.4. El resto del archivo —las tablas seguimientos, logs y perfiles; los índices; la función y el disparador que mantienen

actualizado_en; la función, el disparador y el backfill que crean el perfil de cada usuario; las vistas metrics_mensuales y facturas_pendientes; las políticas de seguridad a nivel de fila; los GRANT y REVOKE; y el alta de leads en la publicación de tiempo real— se consulta en db/schema.sql. Las políticas se reproducen además en el Anexo C. Se sustituye por un marcador el correo del usuario administrador que el archivo original consigna en claro, y se omiten las cláusulas defensivas que hacen idempotente la creación de los tipos enumerados.

Anexo G. Puesta en marcha del entorno

Este anexo reúne el procedimiento mínimo para levantar el sistema en una máquina limpia, requisito del RNF7 y condición para que un tercero reproduzca los escenarios del Capítulo 5.

1. Base de datos. Crear un proyecto de Supabase y ejecutar el contenido de db/schema.sql (Anexos C y F) en el editor SQL. El archivo es idempotente.

2. Orquestador y conversor de documentos. Desde la raíz del repositorio, ejecutar docker compose up -d. El archivo docker-compose.yml levanta n8n (puerto 5678) y Gotenberg en una red interna común, de modo que n8n alcance a Gotenberg por nombre de host. La base de datos no forma parte de la composición: reside en Supabase.

3. Flujo de trabajo. Importar workflow/crm_postgres.json en la instancia de n8n y configurar las credenciales de PostgreSQL, Gmail, Telegram y Notion.

4. Frontend. Instalar dependencias, crear el archivo de variables de entorno a partir de .env.example y ejecutar el proyecto. La Tabla 41 detalla las variables requeridas.

Tabla 41. Variables de entorno requeridas

Variable	Ámbito	Descripción
NEXT_PUBLIC_N8N_BASE	Frontend (pública)	URL base de la instancia de n8n. En local, <code>http://localhost:5678</code> .
NEXT_PUBLIC_SUPABASE_URL	Frontend (pública)	URL del proyecto de Supabase.
NEXT_PUBLIC_SUPABASE_ANON_KEY	Frontend (pública)	Clave pública de Supabase. Es la única que emplea el navegador; el acceso queda acotado por las políticas RLS.
SUPABASE_SERVICE_ROLE_KEY	Scripts de verificación	Secreta. Evade la RLS por diseño. La consumen únicamente la suite de escenarios y la medición de tiempo real; ningún componente del frontend la usa, en línea con el RNF4. No debe versionarse ni exponerse al navegador.
N8N_PUBLIC_URL	Contenedor de n8n	URL pública con la que n8n se anuncia a sí mismo; necesaria para que los webhooks sean alcanzables desde fuera de la máquina.
TELEGRAM_CHAT_ID	Contenedor de n8n	Identificador del chat de destino de las notificaciones.
NOTION_DATABASE_ID	Contenedor de n8n	Identificador de la base de Notion sincronizada.
MP_API_BASE	Contenedor de n8n	Base de la API de MercadoPago. Por omisión <code>https://api.mercadopago.com</code> , el servicio real: no cambia el comportamiento de producción. Se reapunta sólo para ejercitar E14 contra el doble de prueba (§5.3).
MP_ACCESS_TOKEN	Contenedor de n8n	Credencial de la cuenta de MercadoPago que cobra. Vacía por defecto: sin ella el sistema opera en modo de desarrollo (§4.3.3).
MP_CURRENCY	Contenedor de n8n	ARS por defecto. Moneda de la preferencia de checkout; debe coincidir con el país de la cuenta.
MP_COMISION_PORCENTAJE	Contenedor de n8n	1 por defecto. Porcentaje del monto que se registra como comisión de la plataforma; es contable, no una transferencia real.
MP_WEBHOOK_SECRET	Contenedor de n8n	Secreto para validar la firma del webhook de notificación. Obligatorio (Tabla 22, S8): sin este valor, o con firma inválida, el nodo rechaza la notificación. En producción hay que configurar acá el secreto real que entrega el panel de MercadoPago.

Variable	Ámbito	Descripción
CRM_PANEL_TOKEN	Contenedor de n8n y frontend (servidor)	Secreta. Valor del header que exigen los seis webhooks invocados desde el tablero interno (Tabla 21). Sin ella, esas acciones responden 403.
TOKEN_VIGENCIA_DIAS	Contenedor de n8n	14 por defecto. Días de vigencia del enlace de aceptación; fija token_expira_en (Tabla 22, S3).
DEDUP_VENTANA_MINUTOS	Contenedor de n8n	10 por defecto. Ventana de deduplicación de la captación: dentro de esos minutos, un segundo envío con el mismo correo no crea otro lead ni dispara otro aviso (S6, §4.3.1).
RECONCILIACION_GRACIA_MINUTOS	Contenedor de n8n	15 por defecto. Minutos que el proceso de reconciliación concede a la cadena de aceptación antes de dar por perdida la factura de un lead en ACEPTADO (S5, §4.3.5).
FACTURA_VENCIMIENTO_DIAS	Contenedor de n8n	15 por defecto. Días de vigencia de la factura que emite la reconciliación, alineados con los de la rama de aceptación.
CORS_ORIGINS	Contenedor de n8n	Lista blanca de orígenes autorizados a invocar los webhooks desde el navegador (Tabla 21). Es una restricción del navegador y no una autenticación.
CRM_PANEL_HEADER	Contenedor de n8n y frontend (servidor)	Nombre del header de autenticación del panel interno (x-crm-token por defecto). Debe coincidir con la credencial de n8n y con el frontend; su valor es CRM_PANEL_TOKEN.
FRONTEND_URL	Contenedor de n8n	Base de los enlaces que viajan en los correos de propuesta y aceptación.
GOTENBERG_URL	Contenedor de n8n	Punto de conversión de HTML a PDF de la factura; en la composición local se resuelve por nombre de host dentro de la red interna.
RESENAS_FORM_URL	Contenedor de n8n	Formulario de testimonio que se enlaza en el correo de cierre del proyecto.
SCORING_UMBRAL_HOT	Contenedor de n8n	70 por defecto. Puntaje mínimo del nivel HOT (Tabla 16).
SCORING_UMBRAL_WARM	Contenedor de n8n	40 por defecto. Puntaje mínimo del nivel WARM (Tabla 16).
SCORING_PRESUPUESTO	Contenedor de n8n	Tramos de puntaje por presupuesto, en pares umbral:puntos (Tabla 16).
SCORING_URGENCIA	Contenedor de n8n	Puntos asignados a cada nivel de urgencia (Tabla 16).
SCORING_SERVICIO	Contenedor de n8n	Ponderación de cada tipo de servicio (Tabla 16).
SCORING_BONUS_TELEFONO	Contenedor de n8n	5 por defecto. Bonificación cuando el lead declara teléfono.
SCORING_BONUS_DESCRIPCION	Contenedor de n8n	5 por defecto. Bonificación por una descripción suficientemente extensa.

Nota. Elaboración propia a partir de .env.example y docker-compose.yml. Ninguna variable secreta se versiona en el repositorio. Se omiten las del módulo de tickets, fuera de alcance (Anexo B), y las que sólo hacen falta para ejecutar la suite de escenarios.

5. Verificación. Enviar un lead desde el formulario y comprobar que se persiste en leads con su score y su nivel, y que el tablero lo muestra al recargarlo; ese recorrido corresponde al escenario E1. La actualización sin recarga (E7) exige que leads esté incluida en la publicación supabase_realtime, que db/schema.sql resuelve de forma idempotente; cumplida esa condición, node scripts/medir-realtime.mjs mide la latencia de propagación y verifica el umbral de 3 s del RNF6.

Anexo H. Glosario de siglas y acrónimos

Se listan aquí, por orden alfabético, las siglas y acrónimos empleados a lo largo del documento; cuando el término tiene en el texto un lugar de definición o de uso principal, se lo consigna entre paréntesis. La Tabla 42 los reúne.

Tabla 42. Glosario de siglas y acrónimos empleados en el documento

Sigla	Significado y lugar de uso principal
AAIP	Agencia de Acceso a la Información Pública, órgano de control de la Ley 25.326 (§2.7).
AFIP	Administración Federal de Ingresos Públicos, organismo antecesor de ARCA (§2.7).
AGPL	Affero General Public License, licencia de software libre (§2.10).
APA	American Psychological Association, normas de estilo bibliográfico (Capítulo 3).
API	Application Programming Interface, interfaz de programación de aplicaciones.
ARCA	Agencia de Recaudación y Control Aduanero (§2.7, §4.3.2).
B2B	Business to Business (§2.10).

Sigla	Significado y lugar de uso principal
BaaS	Backend as a Service, patrón descrito en el §2.4.
CAE	Código de Autorización Electrónico, exigido por el régimen de facturación electrónica (§2.7).
CORS	Cross-Origin Resource Sharing, control del navegador (Tabla 21, §4.6).
CRM	Customer Relationship Management, sistema de gestión de relaciones con clientes.
CSS	Cascading Style Sheets, hojas de estilo en cascada (Tabla 12).
CUIT	Clave Única de Identificación Tributaria (§2.7).
CVE	Common Vulnerabilities and Exposures (§4.2.3).
CVSS	Common Vulnerability Scoring System (§4.2.3).
DDL	Data Definition Language, lenguaje de definición de datos (Anexo F).
DOI	Digital Object Identifier, identificador persistente citado en las Referencias.
ERP	Enterprise Resource Planning, categoría excluida del universo de comparación (§2.10).
FURPS	Functionality, Usability, Reliability, Performance, Supportability; modelo descartado en el §2.9.
HMAC	Hash-based Message Authentication Code (§4.6, Tabla 22).
HTML	HyperText Markup Language, formato en que se generan la propuesta y la factura antes de convertirlas a PDF (§4.3.2).
HTTP / HTTPS	HyperText Transfer Protocol (Secure), protocolo de transferencia de hipertexto.
iPaaS	Integration Platform as a Service, categoría de plataformas discutida en el §2.10.
ISO/IEC	International Organization for Standardization / International Electrotechnical Commission.
IVA	Impuesto al Valor Agregado; la condición del emisor y del receptor determina el tipo de comprobante (§2.7).
JSON	JavaScript Object Notation, formato de los contratos de integración (§4.5, Anexo D).
JWT	JSON Web Token, mecanismo de credenciales mencionado en el §6.3.
LGPL	Lesser General Public License, licencia de software libre (§2.10).
mTLS	mutual TLS, mecanismo de autenticación mutua mencionado en la Tabla 22.
MVP	Minimum Viable Product / Producto Mínimo Viable (§1.9).
OE	Objetivo Específico (§1.8).
OIT	Organización Internacional del Trabajo (§1.1).
OWASP	Open Web Application Security Project.
PDF	Portable Document Format, formato del comprobante que emite el sistema (§4.3.2).
PRISMA	Preferred Reporting Items for Systematic Reviews and Meta-Analyses, guía cuyo esquema reducido sigue el cribado del §2.10.1.
PyME	Pequeña y Mediana Empresa, segmento con el que el §2.10 asimila al profesional independiente.
RF / RNF	Requisito Funcional / Requisito No Funcional (§3.4, §3.5).
RLS	Row Level Security, seguridad a nivel de fila (§2.6, §4.6).
SaaS	Software as a Service (§2.10).
SEO	Search Engine Optimization, una de las seis opciones de servicio del formulario (§4.2.1).
SQL	Structured Query Language, lenguaje de consulta del motor relacional (§2.4 y Anexo F).
SQuaRE	Systems and software Quality Requirements and Evaluation, marco de ISO/IEC 25010:2023 (Tabla 11).
URL	Uniform Resource Locator, localizador uniforme de recursos.
UTN	Universidad Tecnológica Nacional.
UUID	Universally Unique Identifier, formato del token de aceptación y del de pago (§4.3.2).
UX / UI	User Experience / User Interface; nombra una de las seis opciones de servicio del formulario (§4.2.1).

Nota. Elaboración propia.

Nota terminológica. Tres pares de términos se emplean con un criterio fijo. «Nivel» y «tier» designan lo mismo —la clasificación HOT, WARM o COLD—: se usa «nivel» en el cuerpo y «tier» sólo al nombrar la columna homónima de leads (§2.3). «Presupuesto» es el importe que declara el interesado en el formulario y que alimenta el scoring, mientras que «precio propuesto» es el que fija el profesional y que se factura: son columnas distintas y no sinónimos (§4.3.2). Y la capa de enrutamiento se nombra «capa de proxy» y se implementa en proxy.ts, convención vigente en Next.js 16 que reemplaza al histórico middleware.ts; el documento menciona el nombre antiguo sólo al explicar ese cambio y su motivo (§4.2.3).

Anexo I. Bitácora de correcciones durante el desarrollo

Este anexo consolida las correcciones funcionales y documentales introducidas durante el desarrollo, de modo que el cuerpo describa el estado del artefacto en el corte documental (§5.6) sin competir con el detalle de cuándo se llegó a él. No repite las fechas de cierre de las nueve deudas de seguridad, que la Tabla 22 consigna una por una, ni las de las corridas de validación de la Tabla 25 y la Tabla 32: esas fechas son evidenciales y no

narrativas. La Tabla 43 reúne las correcciones en orden cronológico, con la sección afectada por cada una.

Tabla 43. Bitácora de correcciones durante el desarrollo

Fecha	Corrección	Sección afectada
23-ago-2026	Se corrige el sesgo de la opción «Otro» en el scoring: pasa de mapear a soporte (5 puntos, la ponderación mínima) a mapear a consultoria (12, neutra).	\$4.3.1, Tabla 16
23-ago-2026	La corrida manual de E11 a E13 revela y permite corregir tres defectos de expresión en los procesos programados, invisibles para la lectura estática y para la suite automatizada.	\$5.2
24-ago-2026	Se agrega el nodo Gmail - Acuse Lead Frío: los leads COLD no recibían ningún correo pese a que el formulario promete respuesta dentro de las 24 horas.	\$4.3.1
24-ago-2026	Se incorpora el paso manual de fijación de términos (webhook /propuesta-enviar). Hasta entonces la propuesta y la factura tomaban el presupuesto que el propio interesado había declarado en el formulario.	\$4.3.2, §1.3, §1.5, Capítulo 7
24-ago-2026	El valor de respaldo de la normalización de servicio se alinea con la traducción de «Otro»: de desarrollo_web (la ponderación máxima) a consultoria.	\$4.3.1
24-ago-2026	El tope del control deslizante de presupuesto pasa de 5.000 a 20.000, para que deje de coincidir con el inicio del tramo más alto del puntaje.	\$4.3.1
24 y 28-ago-2026	La consulta del reporte diario pasa a partir de una fila sintética del mes en curso, de modo que el reporte se emite aunque el mes no registre actividad. Reejecutado el 28-ago contra el sistema real: la API de Telegram confirma la entrega y los indicadores coinciden con metrics_mensuales.	\$4.3.5, Tabla 25 (E13)
24-ago-2026	El cierre de proyecto deja de depender de una petición directa del profesional y pasa a tener la acción «Cerrar proyecto» en el tablero interno.	\$4.3.4
23-ago a 2-sep-2026	Reejecuciones de la suite que la Tabla 14 exige tras cada corrección del camino principal: tres corridas seguidas el 23-ago y otras tres el 30-ago; la corrida acreditante es la del 31-ago sobre el commit d1abe08, que es la que reproduce la Tabla 32 y la que cotejó el evaluador externo del Anexo K.	\$5.2, Tabla 26, Tabla 32
26-ago-2026	Se habilita la publicación supabase_realtime para la tabla leads, condición pendiente para poder medir el RNF6 (escenario E7).	\$5.2
27-ago-2026	Al ejercitar E12 se descubre que las notificaciones de Telegram estaban rotas —primero por una variable de entorno vacía y, corregida ésa, por una credencial de bot vencida—; se corrigen ambas causas.	\$5.2
30-ago-2026	Se verifica en vivo el RNF5a en los cuatro anchos de pantalla que fija el requisito y se ejecutan por primera vez E15 y E16 contra el sistema levantado.	\$5.1, Tabla 25
31-ago-2026	Se ejecuta E14 contra un segundo instrumento: un adaptador que llama a la	\$5.3, Tabla 25

Fecha	Corrección	Sección afectada
	API real de Stripe en modo de prueba, complementario al doble de contrato.	
1-sep-2026	Sebastián Díaz Moyano, ajeno al equipo, ejecuta el protocolo de la Tabla 14 sobre el repositorio y cierra la constancia del Anexo K cotejando lo observado contra la evidencia archivada, de fecha anterior. La primera amenaza a la validez del §3.7 queda atendida, no eliminada.	§3.7, Anexo K
2-sep-2026	Se instrumentan test:rls, test:idempotencia y test:realtime para que escriban su salida en docs/ con marca temporal, commit y estado del árbol, como ya hacían los otros tres. Levanta la primera observación del evaluador externo.	§5.4, Anexo K
2-sep-2026	Al ejecutar test:rls sobre PostgreSQL 14 se descubre que db/schema.sql exige la versión 15: las vistas usan WITH (security_invoker = true). La versión declarada provenía de info.version, que informa la de PostgREST y no la del motor. Se corrige la Tabla 12 y se deriva la versión de una restricción verificable del esquema.	§3.6, Tabla 12
2-sep-2026	Se corrige una condición de carrera de tests/trazabilidad.mjs que dejaba E15 en 9 de 10 por sincronización y no por una falla del artefacto; corregida, vuelve a 10 de 10.	§7.1, Anexo K
2-sep-2026	Se siembran las precondiciones que E11 y E12 nunca habían tenido —un lead en su tercer intento y una factura a dos días del vencimiento— y se reejecutan ambos procesos. E11 vuelve a fallar: la transición a PERDIDO que el documento daba por corregida desde el 23-ago seguía sin alcanzarse. Establecida la causa con los conteos de ítems del registro de ejecución y corregida (S9), la reejecución sobre lotes con un señuelo sembrado a propósito deja trece comprobaciones en verde; RF8 y RF9 dejan de estar verificados parcialmente.	§5.2, §5.5, Tabla 25, Anexo E

Nota. Elaboración propia. Cada corrección se verificó contra el artefacto en el momento de aplicarla. Las fechas de cierre de las nueve deudas de seguridad constan en la Tabla 22, y las de las corridas de validación, en la Tabla 25 y en la Tabla 32; no se repiten aquí por no ser correcciones sino evidencia de ejecución.

Anexo J. Instrumentos del estudio propuesto para medir la carga administrativa

Este anexo reúne los instrumentos del estudio con usuarios que el punto 4 del Capítulo 8 propone para medir el componente temporal de la carga administrativa. Son instrumentos propuestos y no aplicados: este trabajo no ejecutó ese estudio ni lo sometió a evaluación ética. Su administración a personas exige, como condición previa e indelegable, el dictamen favorable de un comité de ética institucional; el consentimiento del apartado J.3 es un instrumento de ese proceso y no un sustituto del aval. Se los incluye para que la línea futura quede especificada al nivel de detalle que permitiría llevarla a cabo sin volver a diseñarla. El diseño del estudio se describe en el Capítulo 8 y no se repite aquí.

J.1 Planilla de registro de acciones por lead

La planilla, que reproduce la Tabla 45, se completa por observación directa. El observador registra una fila por cada acción manual del profesional, en el orden en que ocurre, durante la gestión de cada lead del lote.

Definición operacional de «acción manual», compartida por los observadores. Acción manual es toda intervención discreta del profesional sobre una herramienta digital o sobre un documento, con inicio y fin identificables, orientada a hacer avanzar el trámite del lead y no a producir el servicio contratado. Se cuenta una acción nueva cada vez que cambia la herramienta usada o el propósito de la intervención. No se cuentan las esperas, las interrupciones ajenas a la tarea ni la lectura sin registro ni decisión posterior. Esta definición es la misma que fija el §1.7 (Definición operacional de la carga administrativa), detallada aquí para su uso en campo.

Cada acción se clasifica en una y sólo una de las categorías de la Tabla 44.

Tabla 44. Categorías de acción manual del instrumento J.1

Categoría	Definición
Comunicación	redactar, revisar o enviar un correo, un mensaje o una llamada dirigidos al cliente o a un tercero.
Transferencia de datos	copiar, transcribir o volcar datos de un sistema o documento a otro.
Generación documental	crear, completar o exportar un documento, como una propuesta, un comprobante o un recibo.
Actualización de registros	modificar el estado de una oportunidad en una planilla, un CRM, una agenda o un tablero.
Consulta y verificación	buscar o cotejar información necesaria para continuar, por ejemplo confirmar si un pago ingresó.
Decisión de negocio	fijar el precio, el plazo o el alcance de una propuesta, o dar por cerrado el proyecto.

Nota. Instrumento de elaboración propia.

Acuerdo entre observadores. Dos observadores registran de forma independiente la misma submuestra, no menor al 20 % de los lotes y elegida al azar. Se calculan el porcentaje de acuerdo sobre el número de acciones y el coeficiente kappa de Cohen sobre la categoría asignada; un valor de kappa inferior a 0,70 obliga a revisar la definición operacional y a repetir el registro de esa submuestra antes de analizar los datos. Las acciones se numeran de forma correlativa dentro de cada lead y las horas se anotan en formato de 24 horas.

Tabla 45. Planilla de registro de acciones por lead (instrumento J.1)

ID de lead	N.º de acción	Descripción de la acción	Categoría	Hora de inicio	Hora de fin	Duración	Observaciones

Nota. Instrumento de elaboración propia. Categorías admitidas: comunicación, transferencia de datos, generación documental, actualización de registros, consulta y verificación, decisión de negocio (véase la definición operacional de este anexo). Se imprime una planilla por lead; las filas en blanco se reproducen según se necesiten.

J.2 Cuestionario de familiaridad previa

El cuestionario se administra a cada participante una sola vez, antes del primer bloque. Mide la familiaridad previa con herramientas de automatización, que el Capítulo 8 declara como variable interviniente principal. Se identifica con el mismo código que las planillas del participante y no con su nombre. La Tabla 46 reproduce sus diez ítems.

Tabla 46. Cuestionario de familiaridad previa (instrumento J.2)

Nº	Pregunta	Formato de respuesta
1	¿Cuántos años hace que trabaja de manera independiente (freelance)?	Respuesta abierta, en años.
2	¿Cuál es su área principal de servicio?	Opción única: desarrollo de software · diseño · marketing · redacción · consultoría · otra (indicar).
3	¿Cuántos clientes o consultas nuevas gestiona en un mes típico?	Respuesta abierta, numérica.
4	¿Utilizó alguna vez un CRM, como HubSpot, Pipedrive o Zoho?	Sí / No. Si es afirmativa, indicar cuál y por cuánto tiempo.
5	¿Utilizó alguna vez una herramienta de automatización de flujos, como Zapier, Make, n8n o IFTTT?	Sí / No. Si es afirmativa, indicar cuál.
6	¿Con qué frecuencia automatiza tareas administrativas de su trabajo?	Opción única: nunca · rara vez · a veces · a menudo · siempre.
7	Autoevalúe su habilidad técnica general con herramientas digitales.	Escala de 1 a 5, donde 1 es muy baja y 5 es muy alta.
8	Autoevalúe su comodidad para aprender una herramienta nueva sin ayuda.	Escala de 1 a 5.
9	¿Qué utiliza hoy para llevar el registro de sus oportunidades comerciales?	Opción única: planilla de cálculo · CRM · bandeja de correo · agenda en papel · nada en particular · otra (indicar).
10	¿Tiene formación formal en informática o programación?	Opción única: ninguna · curso corto o taller · tecnicatura o carrera superior.

Nota. Instrumento de elaboración propia. Se administra una sola vez, antes del primer bloque.

J.3 Formulario de consentimiento informado (Ley 25.326)

Se entrega y se firma antes del primer bloque, en dos ejemplares de igual tenor; uno queda para el participante. Los campos entre corchetes se completan al momento de administrar el estudio. Este formulario no puede administrarse antes de que un comité de ética institucional apruebe el protocolo del estudio: la referencia a ese dictamen —comité interviniente, número de acta y fecha— se incorpora al encabezado del formulario una vez obtenida.

Título del estudio: Medición del número de acciones administrativas en la gestión de clientes de profesionales independientes, con y sin un sistema de automatización.

Responsables. El estudio está a cargo de Mateo Morgui y Tobías Rivas, estudiantes de la Tecnicatura Universitaria en Programación de la Universidad Tecnológica Nacional, Facultad Regional Mendoza, bajo la dirección de Alberto Cortez. Contacto: [correo electrónico de contacto].

1. Finalidad. Se lo invita a participar de un estudio que busca contar cuántas acciones administrativas requiere la gestión de una oportunidad comercial y comparar el método manual habitual con el uso de un sistema de automatización. Los datos se usan con fines exclusivamente académicos.

2. En qué consiste su participación. Completar un cuestionario breve sobre su experiencia previa, gestionar dos lotes equivalentes de oportunidades ficticias —uno con su método habitual y otro con el sistema, en el orden que se le indique— y permitir que un observador registre en una planilla las acciones que realiza y el tiempo de cada una. Cada bloque se extiende por hasta dos semanas.

3. Datos que se recogen. Sus respuestas al cuestionario, el número, la categoría y la duración de las acciones observadas, y notas de contexto del observador. Las oportunidades son ficticias: no se recogen datos de clientes reales ni datos sensibles en el sentido del art. 7 de la Ley 25.326.

4. Carácter voluntario. Su participación es voluntaria: puede negarse a participar y retirarse en cualquier momento, sin expresar motivo y sin consecuencia alguna, y pedir que los datos ya recogidos se eliminen.

5. Confidencialidad. La planilla y el cuestionario se identifican con un código y no con su nombre; los resultados se informan de forma agregada o con un código que no permite identificarlo.

6. Tratamiento de datos personales. El responsable de la base es [nombre del responsable], con domicilio en [domicilio]. Usted tiene derecho a acceder a sus datos, rectificarlos y solicitar su supresión, de forma gratuita, escribiendo a [correo electrónico de contacto] (arts. 6, 14, 15 y 16 de la Ley 25.326). Los datos se conservan sólo mientras son necesarios para el análisis y la defensa del trabajo final y se suprimen dentro de los seis meses posteriores a su aprobación (art. 4, inc. 7). No se ceden a terceros ni se transfieren fuera del país. Puede reclamar ante la Agencia de Acceso a la Información Pública.

7. Riesgos y beneficios. El estudio no implica riesgos previsibles más allá del cansancio propio de la tarea, y no se prevé un beneficio directo para usted.

8. Declaración. Declaro que leí este formulario, que se me explicó su contenido, que pude hacer preguntas y que recibí respuestas satisfactorias. Acepto participar del estudio en los términos descritos y acepto el tratamiento de mis datos con la finalidad indicada.

Participante. Nombre y apellido: ____ Firma: ____ Fecha: ____ / ____ / ____

Responsable que toma el consentimiento. Nombre y apellido: ____ Firma: ____ Fecha: ____ / ____ / ____

Anexo K. Constancia de verificación por un evaluador externo

El §3.7 declara como primera amenaza a la validez el sesgo del evaluador: que quienes construyen el artefacto sean también quienes lo validan. Para atenderla, una persona

ajena al equipo ejecutó el protocolo de la Tabla 14 sobre el repositorio declarado en §4.1 y cotejó lo observado contra la evidencia que el propio repositorio tenía archivada. La Tabla 47 reproduce esa constancia, del 1 de septiembre de 2026. Su alcance decide qué puede leerse en ella: el evaluador constató que los instrumentos son ejecutables por un tercero y que sus resultados coinciden con la evidencia preexistente —del 30 y el 31 de agosto, y del 26 de agosto para la medición de propagación—, no que las corridas acreditantes se hubieran generado en su presencia.

Las tres observaciones de método que el evaluador dejó asentadas se atendieron al día siguiente, con posterioridad al corte documental que fija el §5.6, y conviene ser explícito sobre quién hizo qué: ese trabajo lo hicieron los autores, de modo que no constituye una segunda verificación externa. Se instrumentaron los tres verificadores que informaban sólo por consola (§5.4) y se confirmó en un commit el cambio que db/schema.sql conservaba sin registrar. Las dos primeras observaciones dejan de aplicar al repositorio posterior, no al estado que este documento acredita, en el que siguen describiendo con exactitud lo que el evaluador encontró; la constancia, por su parte, sigue siendo la del 1 de septiembre y no se actualiza retroactivamente.

Conviene declarar qué acredita esta constancia y qué no. Acredita que el protocolo de la Tabla 14 es ejecutable por un tercero a partir del material versionado y que los instrumentos no dependen, para arrojar su resultado, de que los ejecuten los autores. No acredita cuatro cosas: una evaluación ciega, porque el evaluador disponía del documento y con él del resultado esperado de cada escenario; la independencia de la corrida acreditante, porque cotejó evidencia archivada con anterioridad; la auditabilidad posterior de tres de los seis instrumentos, que en esa fecha informaban sólo por consola; y un árbol de trabajo plenamente limpio. Las tres últimas se atendieron al día siguiente, pero por obra de los autores y con posterioridad al corte documental (§5.6), de modo que sobre el estado que este documento acredita siguen describiendo lo que el evaluador encontró. Cerrarlas también del lado externo exigiría una nueva verificación con procedimiento ciego, que se registra como línea pendiente.

Tabla 47. Constancia de verificación del protocolo de validación por un evaluador externo

Elemento de la constancia	Contenido
Evaluador	Sebastián Díaz Moyano.
Vínculo con el equipo	Ajeno al equipo de trabajo: no participó del diseño, de la implementación ni de la redacción de este trabajo final.
Declaración de conflicto de interés	El evaluador declara no tener con los autores ni con el director vínculo de parentesco, relación laboral, dependencia académica ni interés económico en el resultado de esta verificación, no haber recibido contraprestación alguna por realizarla y no haber participado del diseño, de la implementación ni de la redacción del trabajo final.

Elemento de la constancia	Contenido
Fecha de la verificación	1 de septiembre de 2026. Ese día el evaluador ejecutó los seis instrumentos sobre el material versionado y cotejó sus resultados contra la evidencia archivada en el repositorio, cuyas marcas temporales eran anteriores: 30 y 31 de agosto de 2026, y 26 de agosto para la corrida de test:realtime entonces vigente. Los valores que esta constancia reporta provienen de esa evidencia preexistente y no de corridas generadas en presencia del evaluador.
Material entregado	El repositorio declarado en §4.1, el procedimiento de puesta en marcha del Anexo G y el protocolo de ejecución y registro de la Tabla 14. Sin acompañamiento de los autores durante la ejecución.
Versión del repositorio	d1abe08b5cf896f986b92f646c44719755f88661 (forma abreviada d1abe08), del 31 de agosto de 2026 a las 16:13:20 (UTC-3). El árbol de trabajo no estaba limpio: db/schema.sql conservaba tres líneas modificadas sin registrar, correspondientes a un comentario, sin cambio alguno en el SQL ejecutable. Ese cambio quedó confirmado el 2 de septiembre de 2026 en el commit 177bc9c.
Entorno de ejecución	Microsoft Windows 11 Pro 24H2, versión 10.0.26100 (compilación 26100.8875); Docker 29.6.2 (build dfc4efb) con Docker Compose v5.3.1; Node.js v24.18.0 con npm 11.16.0.
Instrumentos ejecutados	Los seis previstos: npm run test:escenarios, test:rls, test:idempotencia, test:realtime, test:trazabilidad y test:exposicion. Tres dejaban evidencia fechada en archivo (test:escenarios en docs/evidencia-validacion.md; test:exposicion y test:trazabilidad en docs/evidencia-E15-E16.md). Los otros tres informaban sólo por consola: test:rls ejecutó los 33 casos rol por rol sobre una imagen postgres:16-alpine desechable y test:idempotencia, las 22 comprobaciones sobre S5 y S6; test:realtime no dejó evidencia archivada y sus valores correspondían a corridas exploratorias de cinco repeticiones, por debajo de la muestra de veinte que fija el método.
Valores observados por escenario	Corrida de tests/escenarios.mjs registrada en docs/evidencia-validacion.md con marca temporal 2026-08-31T18:16:30.224Z: 12 de 12 filas en OK. Tiempos por escenario: E1, 6009 ms; E1b, 3413 ms; E2, 4119 ms; E3, 4553 ms; E4, 4997 ms; lectura de la propuesta con el token correcto, 618 ms; E5/E6, 9257 ms; idempotencia del pago simulado, 4540 ms; E8, 5095 ms; E9, 4595 ms; E10, 709 ms; rechazo de un token vencido, 7538 ms. Son exactamente los que reproduce la Tabla 32, porque esa tabla acredita esta misma corrida: lo que el evaluador cotejó de forma independiente y lo que el documento presenta como evidencia de E1 a E10 son el mismo registro sobre el mismo commit.
Observaciones del evaluador	Los seis instrumentos devolvieron el resultado que el documento declaraba. Tres observaciones de método. Primera: sólo tres de los seis dejan evidencia fechada en archivo; los otros informan por consola y no resultan auditables sin reejecutarlos. Segunda: la evidencia archivada lleva marcas temporales anteriores a esta constancia, de modo que lo verificado es evidencia preexistente y no una corrida generada en el acto. Tercera: el árbol de trabajo no estaba limpio. Subsiste, además, que la verificación no es ciega.

Nota. Elaboración propia. Los campos de registro los completó el evaluador con los valores que observó, de modo que la ejecución resulta auditable por un tercero, que es precisamente lo que esta constancia se propone habilitar. Restan las dos firmas, que se así entan sobre el ejemplar impreso.